Binarization

2025-02-18

이진화 방식

1. 전역 임계값 기반 이진화

   (1) 단순 이진화 (Basic Thresholding)

   한 개의 임계값(threshold)을 기준으로 픽셀을 0(검은색) 또는 255(흰색)으로 변환

   (2) Otsu 이진화 (Otsu's Binarization)

   이미지 히스토그램 분석을 통해 최적의 임계값을 자동으로 결정,

   전경과 배경이 분명한 이미지에 효과적

2. 지역(적응형) 임계값 기반 이진화

   (3) 적응형 이진화 (Adaptive Thresholding)

   이미지의 국소적인 영역마다 다른 임계값을 적용,

   조명 변화가 있는 이미지에서 효과적

3. 기타 특수한 이진화 기법

   (4) 반전 이진화 (Inverse Binarization)

   기본 이진화의 반대: 밝은 부분을 0, 어두운 부분을 255로 변환

   (5) 삼진화 (Trinary Thresholding)

   세 가지 레벨로 픽셀 값을 나누는 방식 (흑, 백, 회색 중간값)

   (6) 적응형 Otsu 이진화 (Adaptive Otsu)

   지역별로 Otsu 이진화를 적용하는 방식

사용할 필터의 목록

1. 공간 도메인 필터

(1) 평균 필터 (Mean Filter)

- 주변 픽셀의 평균값을 계산하여 중앙 픽셀을 새로운 값으로 설정하는 방식
- 단순하지만 엣지가 흐려지는 단점이 있음

(2) 가우시안 필터 (Gaussian Filter)

- 가우시안 분포를 기반으로 가중치를 적용한 블러링 필터
- 평균 필터보다 부드럽고 자연스러운 효과 제공

(3) 중간값 필터 (Median Filter)

- 주변 픽셀의 중간값을 중앙 픽셀로 설정
- 소금-후추 노이즈(Salt & Pepper Noise) 제거에 효과적

(4) 양방향 필터 (Bilateral Filter)

- 공간 가중치와 픽셀 값 차이에 기반한 가중치를 결합하여 블러 효과 최소화
- 엣지를 보존하면서 노이즈 제거 가능

(5) 비등방성 확산 필터 (Anisotropic Diffusion Filter, Perona-Malik Filter)

- 이미지의 경계 부분을 보존하면서 내부 노이즈를 효과적으로 제거하는 방식
- PDE(편미분방정식) 기반의 기법

(6) 비국소적 평균 필터 (Non-Local Means Filter, NLM)

- 유사한 패치를 찾아 평균을 계산하여 노이즈 제거
- 자연스러운 결과를 제공하지만 계산량이 많음

2. 주파수 도메인 필터

(7) 저주파 필터 (Low-Pass Filter)

- 고주파(엣지 및 노이즈)를 제거하고 저주파(부드러운 영역)만 남김
- Fourier Transform을 이용하여 필터링 가능

(8) 위너 필터 (Wiener Filter)

- 신호와 노이즈의 파워 스펙트럼을 분석하여 최적의 필터링 수행
- 블러 제거에도 효과적임

(9) 파동 변환 필터 (Wavelet Transform Denoising)

- 이미지 데이터를 다중 해상도로 분석하여 노이즈를 제거하는 방식
- Haar, Daubechies 같은 웨이블렛 기반 필터 사용

3. 머신러닝 및 신경망 기반 필터

(10) BM3D (Block-Matching and 3D Filtering)

- 비선형 필터링 기법으로, 비슷한 패치를 그룹화하여 3D 변환 후 노이즈 제거
- 현재까지도 강력한 노이즈 제거 알고리즘 중 하나

(11) 딥러닝 기반 필터 (CNN Denoising, DnCNN)

- 컨볼루션 신경망(CNN)을 사용하여 노이즈 제거
- 데이터 학습을 통해 최적의 필터를 자동 생성

(12) GAN 기반 필터 (Denoising Autoencoder, Noise2Noise, Noise2Void)

- 노이즈가 포함된 이미지를 학습하여 클린 이미지를 복원하는 방식
- 지도학습 없이도 강력한 성능을 발휘

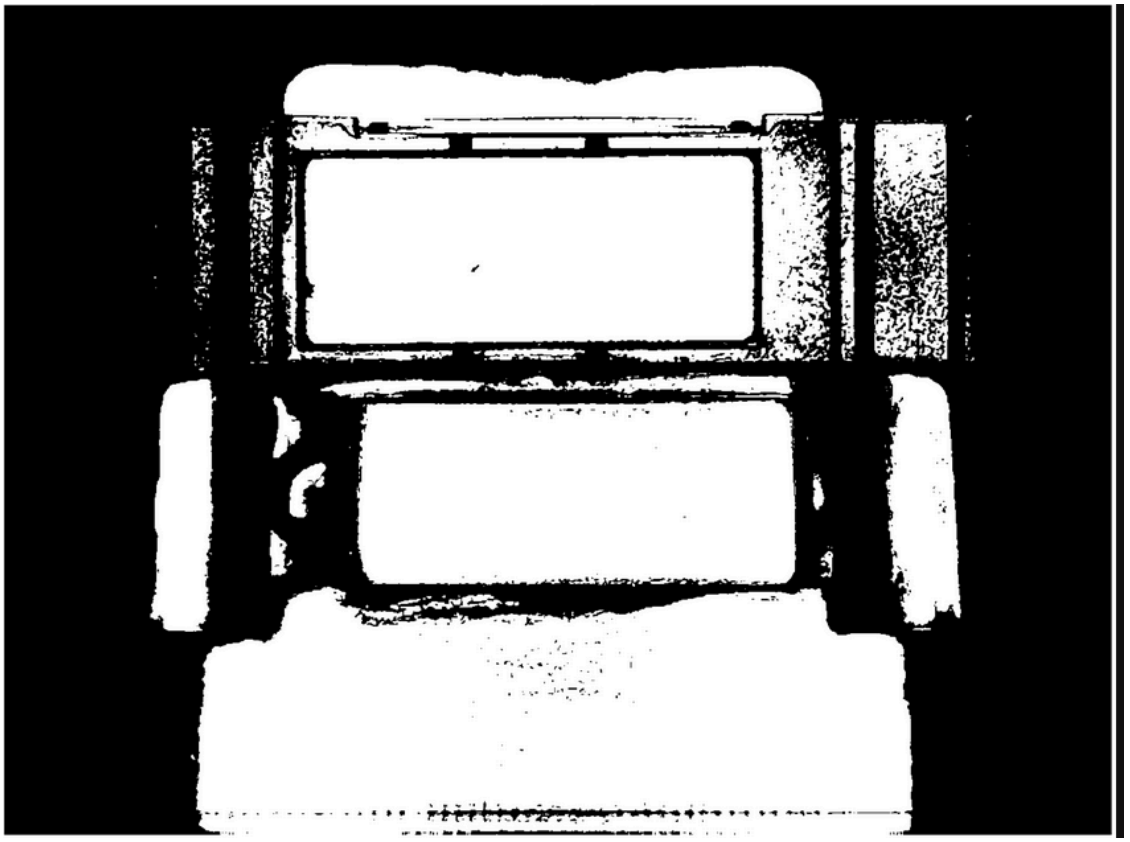
진행 방향

필터 하나, 두개(순서 바꿔서) 를 적용한 이미지에 단순 이진화를 적용시켜서 경계선의 차이를 확인해봄

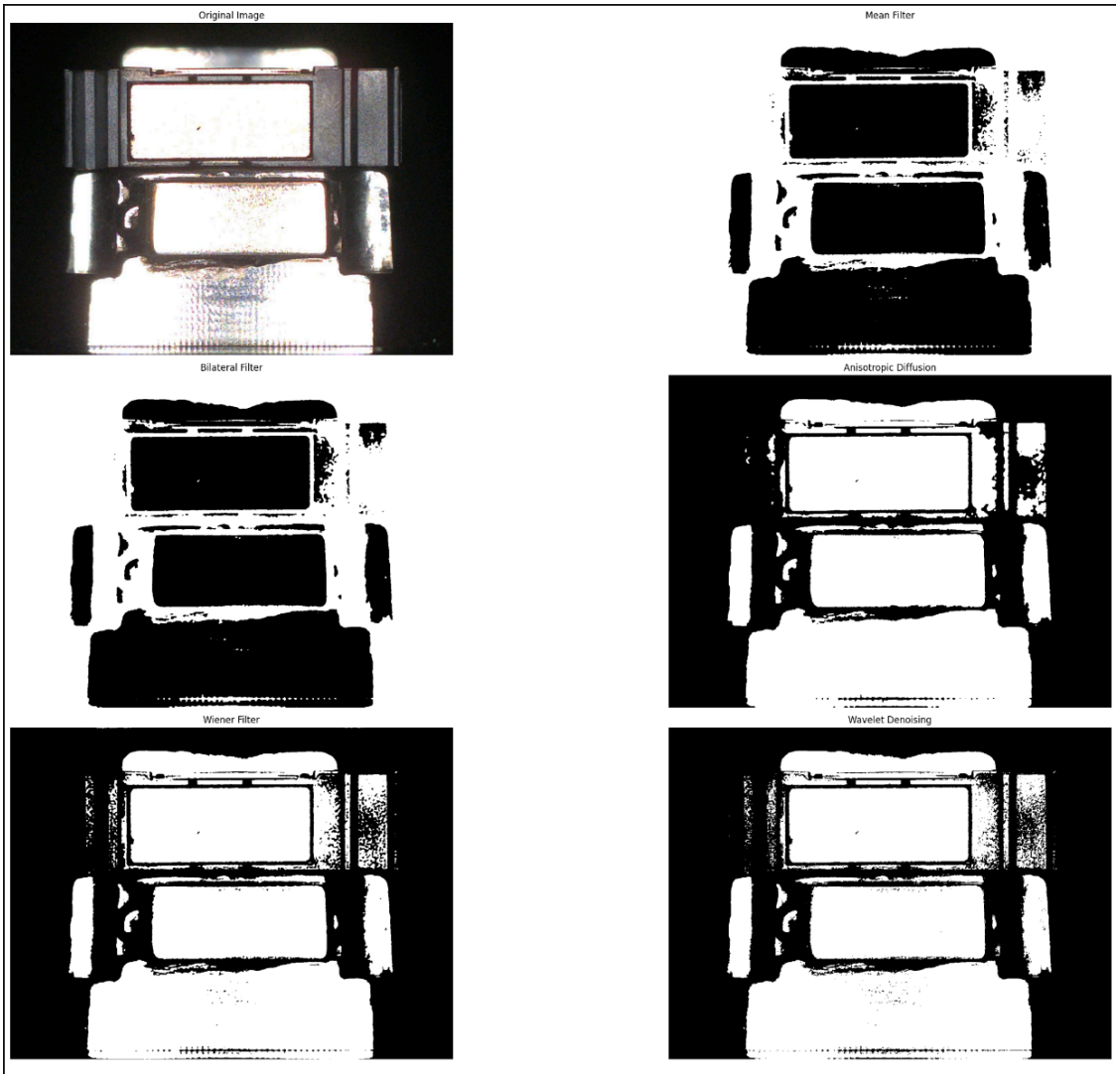
2025-02-19

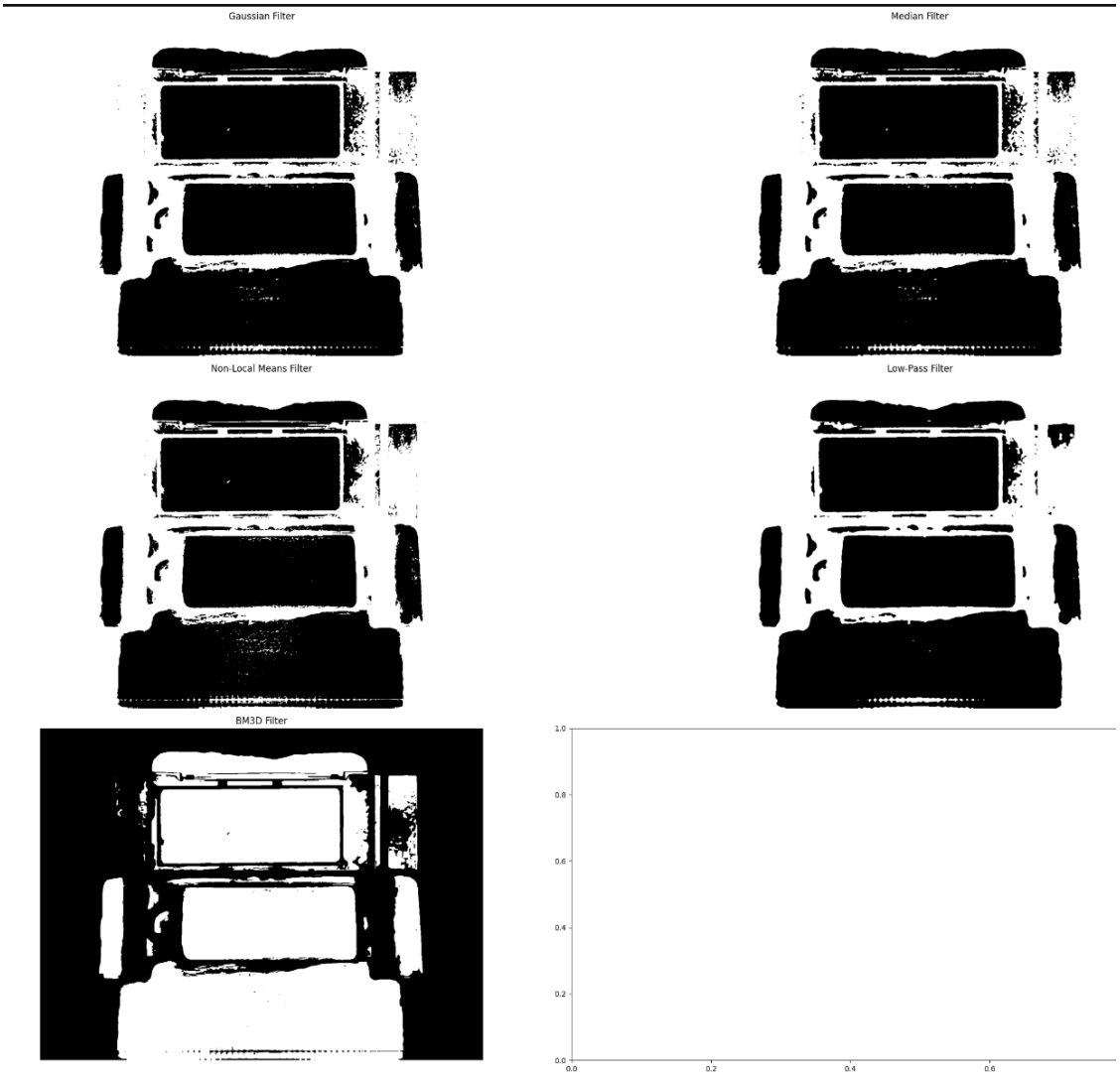
환경 : 각 필터 적용시키고 단순이진화(임계값 120으로 진행)

이미지에 필터 없이 이진화 적용



단일 필터 적용


Original Image
Mean Filter
Bilateral Filter
Anisotropic Diffusion
Wiener Filter
Wavelet Denoising



방식 변경:

필터 적용하고 이진화된 사진 눈으로 보는 방식보다

먼저 임의의 필터 하나 선정하고 각 위치 갭 측정하는 코드 만들어서 필터마다 갭 측정 후 그 갭의 움직임을 보는 방식

먼저 픽셀 사이 갭을 측정하는 방식부터 생각!

방식 1: 1D Projection (가로 또는 세로 방향으로 흰색 픽셀 간 거리 측정)

ROI 내부의 흰색 픽셀을 **가로(X) 방향 또는 세로(Y) 방향으로 투영(Projection)하여 분석**.

`np.where()`를 사용해 흰색 픽셀 좌표를 찾고, 픽셀 간 거리를 계산.

방식 1 ,

5픽셀 미만은 무시하고 각 6개의 위치에서 검정색 픽셀들을 그룹화함.

그룹이 1개이면 그 검정색 그룹의 최상단점 최하단점 기준 갭 측정

그룹이 2개 이상이면 최상단 검정 그룹에서의 최상단점 최하단점 기준 갭 측정

굉장해, 엄청나! 중요한 가정 : 최상단 검정 그룹은 내가 측정하는 그 검정 부분이여야 한다!

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import bm3d

# 이미지 로드
image_path = "data/3560_Magnet Check/P001/IV00052/0000/00512_001_OK.jpeg"
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

# BM3D 필터 적용 (노이즈 제거)
gray = image.astype(np.float32) / 255.0  # BM3D는 float 입력 필요
bm3d_filtered = bm3d.bm3d(gray, sigma_psd=0.1, stage_arg=bm3d.BM3DStages.HARD_THRES
HOLDING)

# 단순 이진화 적용 (Thresholding, 임계값 120)
threshold_value = 120
_, binary = cv2.threshold((bm3d_filtered * 255).astype(np.uint8), threshold_value, 255, cv2.THRE
SH_BINARY)

# ROI 설정 (고정)
x, y, w, h = 570, 380, 80, 30  # 원하는 ROI 좌표 (x, y, width, height)
roi = binary[y:y+h, x:x+w]  # ROI 내부의 이진화된 영역 추출

# ROI 내부 6개의 위치에서 검정 픽셀 찾기
num_samples = 6
step = w // (num_samples + 1)  # 균일한 간격 계산
sample_x_positions = [x + step * (i + 1) for i in range(num_samples)]  # 6개의 X 좌표 계산

black_pixel_lengths = []
for center_x in sample_x_positions:
    black_pixels_y = np.where(binary[y:y+h, center_x] == 0)[0] + y  # 전체 이미지에서 Y 좌표 반영

    # 검정색 픽셀 그룹 찾기
    min_black_length = 5  # 노이즈로 간주할 검정색 길이 기준 (5 픽셀 이하 무시)
    filtered_black_pixels = []

    if len(black_pixels_y) > 1:
        groups = np.split(black_pixels_y, np.where(np.diff(black_pixels_y) > 1)[0] + 1)
        long_black_groups = [group for group in groups if len(group) > min_black_length]

        if len(long_black_groups) >= 1:
            # 가장 위쪽 검정색 그룹을 선택
            top_black_group = long_black_groups[0]
```

```
            start_y = min(top_black_group)  # 해당 그룹에서 가장 위쪽 픽셀
            end_y = max(top_black_group)  # 해당 그룹에서 가장 아래쪽 픽셀

            black_pixel_length = end_y - start_y
            black_pixel_lengths.append((center_x, start_y, end_y, black_pixel_length))
            print(f"X={center_x}, 검정색 시작점: {start_y}, 끝점: {end_y}, 길이: {black_pixel_length} px")

# ROI 및 측정된 검정 픽셀 선을 이진화된 이미지에 추가
binary_colored = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)  # 이진화된 이미지에 색상을 추가할 수 있도록 변환
cv2.rectangle(binary_colored, (x, y), (x+w, y+h), (0, 255, 0), 2)  # ROI 박스 그리기

# 검출된 6개의 위치에 대해 선을 그리기
for center_x, start_y, end_y, _ in black_pixel_lengths:
    cv2.line(binary_colored, (center_x, start_y), (center_x, end_y), (0, 0, 255), 2)  # 검정 픽셀의 시작점과 끝점을 잇는 선 그리기

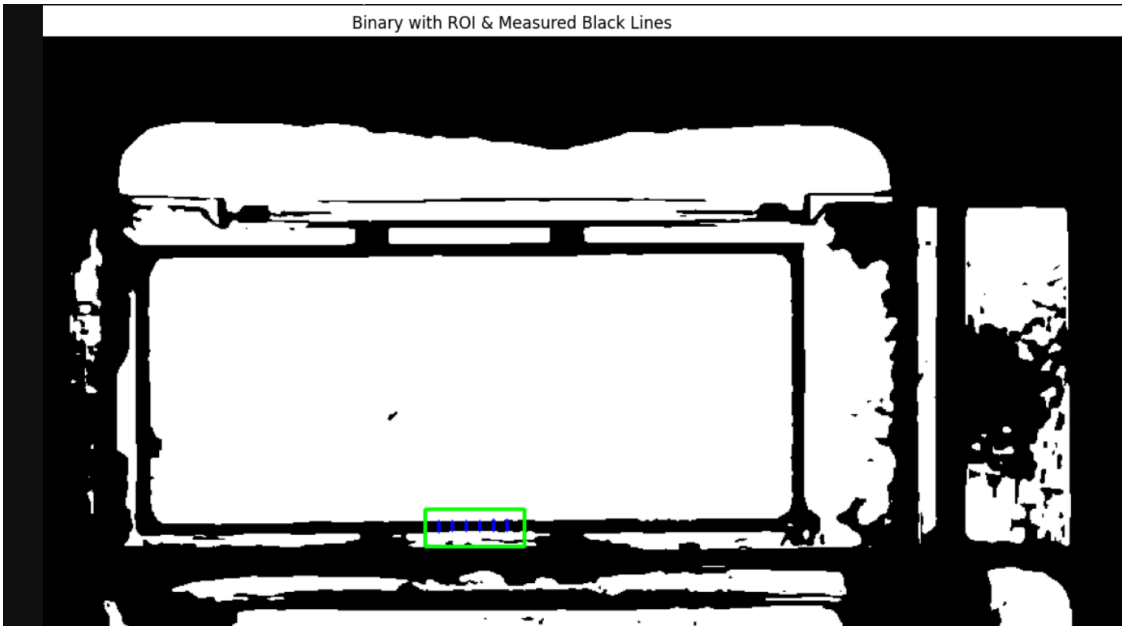
# 결과 시각화
fig, axes = plt.subplots(1, 3, figsize=(50, 20))
axes[0].imshow(image, cmap='gray')
axes[0].set_title("Original Image")
axes[0].axis("off")

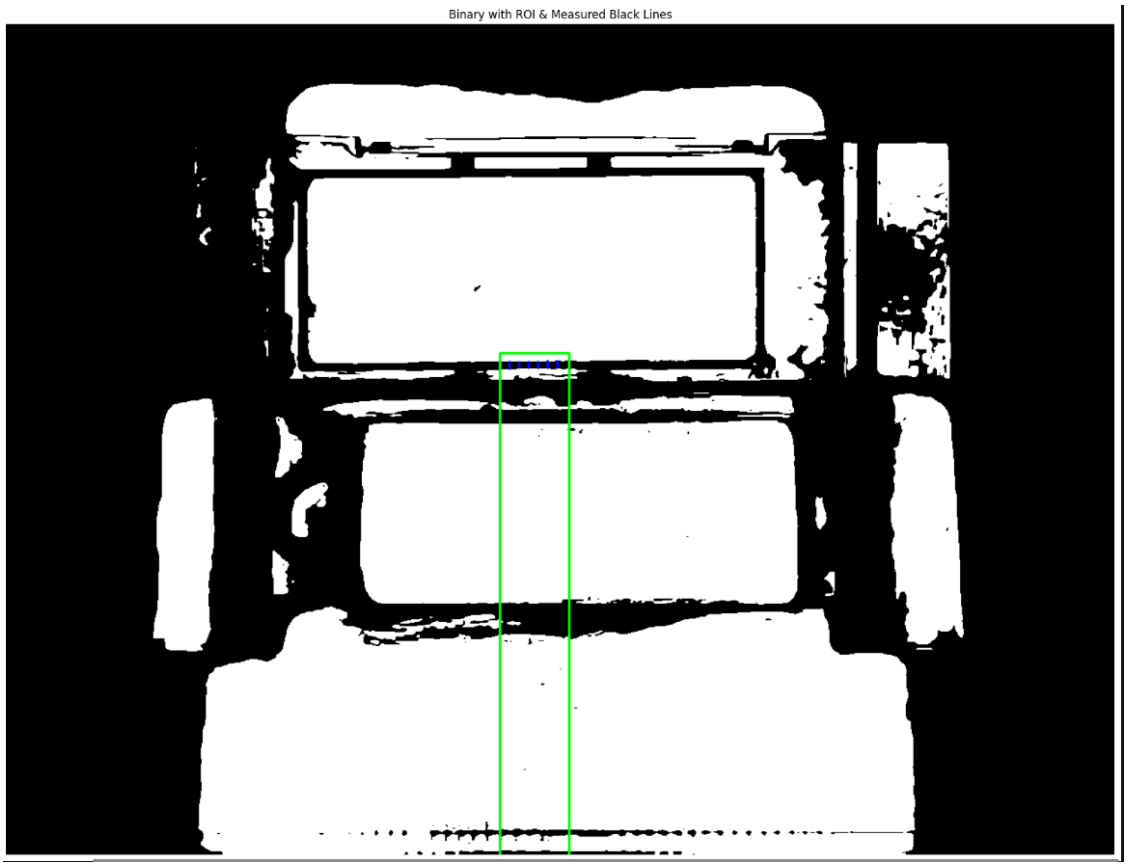
axes[1].imshow(bm3d_filtered, cmap='gray')
axes[1].set_title("BM3D Filter Applied")
axes[1].axis("off")

axes[2].imshow(binary_colored)
axes[2].set_title("Binary with ROI & Measured Black Lines")
axes[2].axis("off")

plt.tight_layout()
plt.show()
```

```
X=581, 검정색 시작점: 390, 끝점: 398, 길이: 8 px
X=592, 검정색 시작점: 390, 끝점: 397, 길이: 7 px
X=603, 검정색 시작점: 390, 끝점: 397, 길이: 7 px
X=614, 검정색 시작점: 390, 끝점: 397, 길이: 7 px
X=625, 검정색 시작점: 389, 끝점: 397, 길이: 8 px
X=636, 검정색 시작점: 389, 끝점: 397, 길이: 8 px
```





각 ROI마다 특성이 다르고 측정할 검정색 그룹의 위치가 달라질 수 있으므로 ROI마다 그 특성을 고려해서 어느 위치에 있는 검정색 그룹을 측정할지 적용하는 코드로 수정 해야 할 듯..?

2025-02-20

6개의 roi(위치는 정적으로 생성)를 생성하고

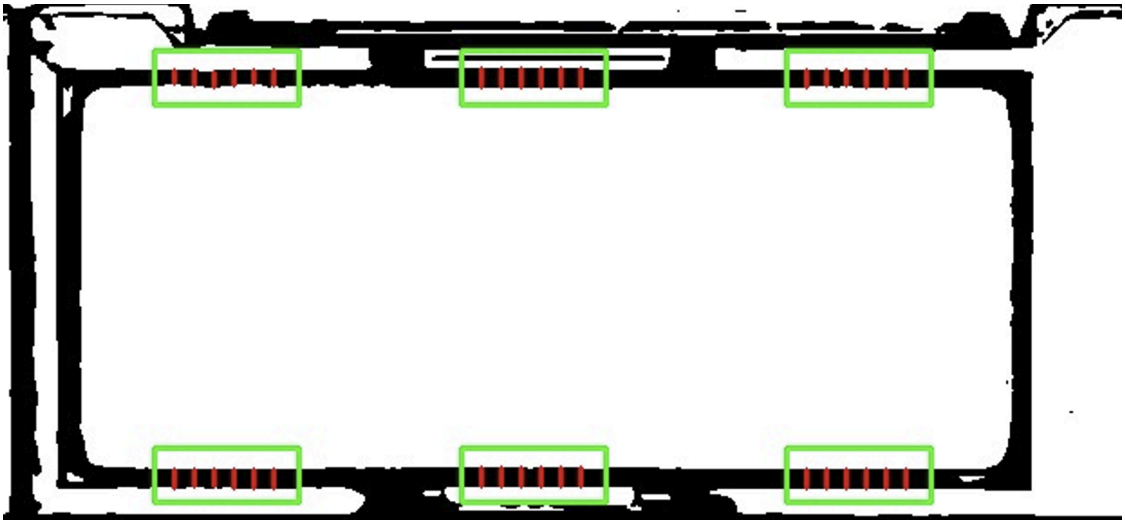
1. 각 roi에서 연속된 픽셀을 찾아 그룹으로 묶는다
2. 그룹에서 5픽셀 이하는 노이즈로 보고 그룹에서 제외
3. 그룹중에서 옵션(0:최상단 1:두번째)을 주어 최상단에 있는 그룹에서 최상단,최하단점의 갭을 측정

03812_001_OK.jpeg

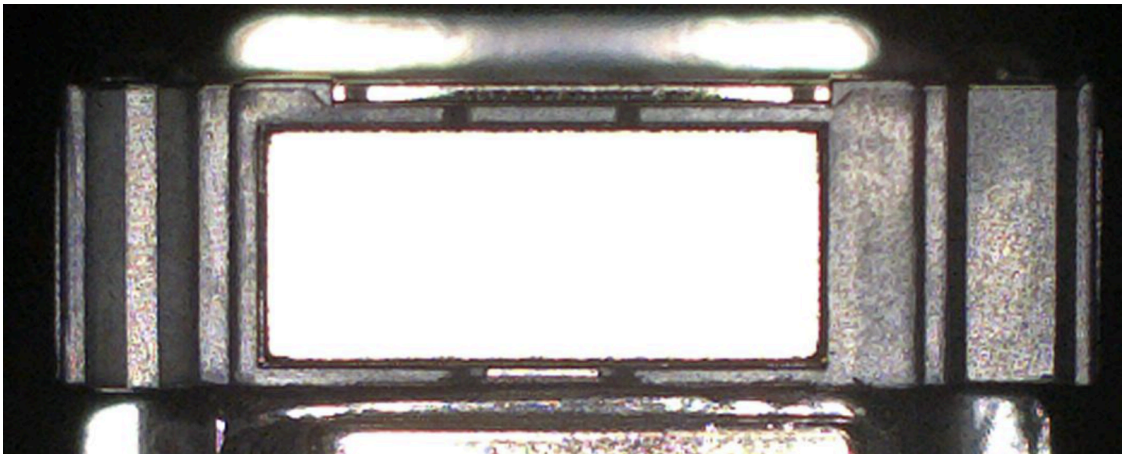


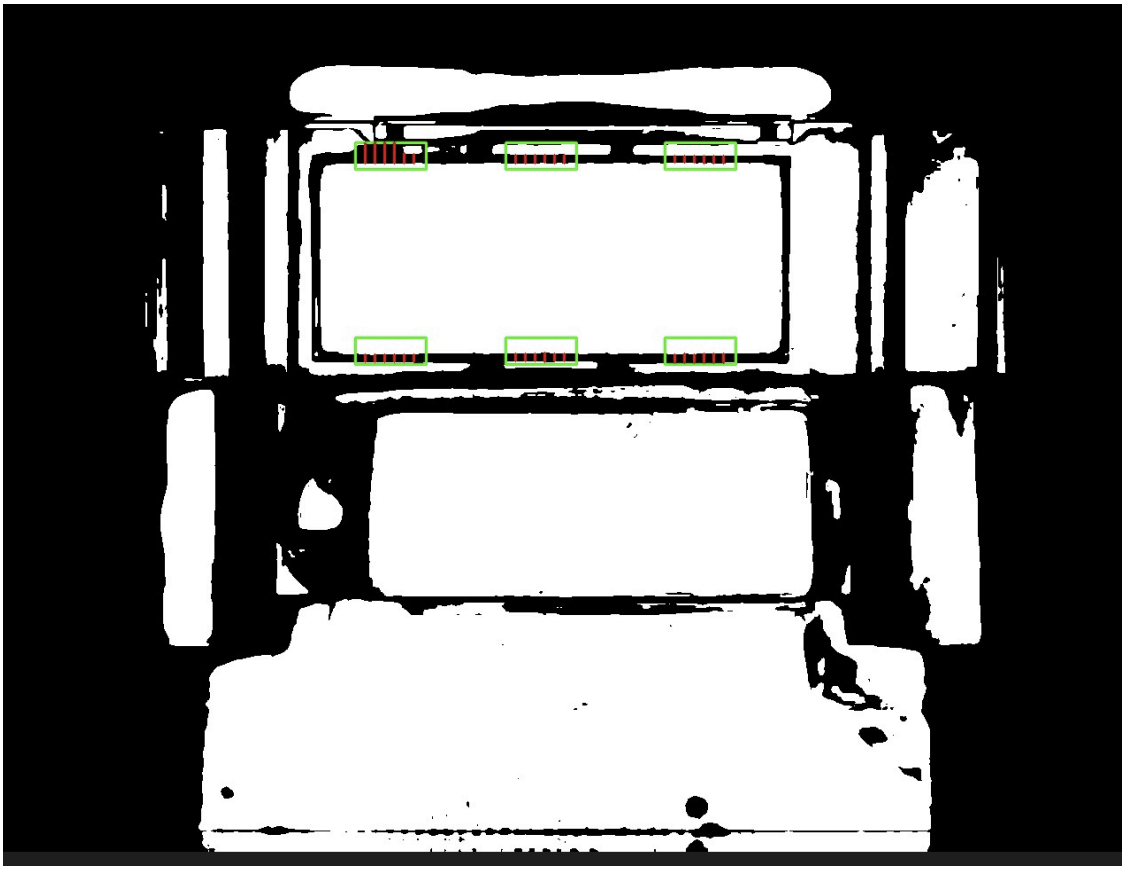
중간 노이즈가 관측되어 측정이 안된 경우 존재

02439_001_OK.jpeg

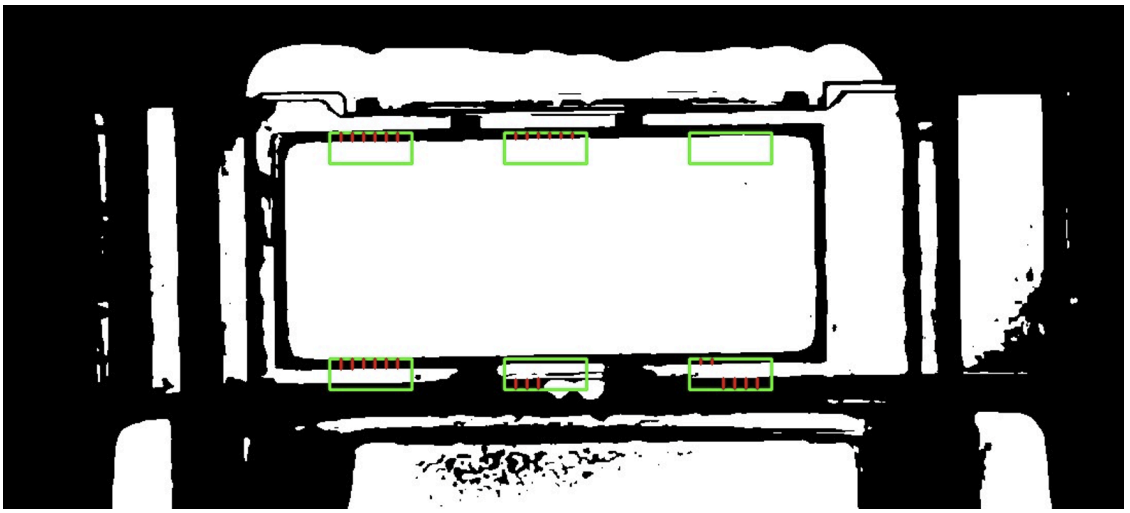


01518_001_OK.jpeg





02390_001_OK.jpeg



현재 발견된 문제점

1. 사진이 흔들림이 있어 정적 ROI 시 문제 발생
2. 이진화한 결과 문제, 중간에 흰색 노이즈 그대로 나오거나 아예 이진화 임계값이 문제됨.

2025-02-24

폴더 내 임계값(500장)
이미지 회전 + 평행이동 후
동적 roi생성(위치는 객체의 좌상단점 기준, 각 roi당 측정 할 선 6개)하여 각 roi에 검정색 갭 찍어서 저장하는 코드

```
import cv2
import torch
import numpy as np
from ultralytics import YOLO
import os
import bm3d

def order_points(pts):
    """
    4개의 꼭짓점을 좌상, 우상, 우하, 좌하 순서로 정렬하는 함수 (PCA 없이 중심점을 기준으로 정렬)
    """
    # 1️⃣ 4개 점의 중심점(center) 계산
    center = np.mean(pts, axis=0)

    # 2️⃣ 좌우를 나눔 (x 좌표 기준)
    left_pts = pts[pts[:, 0] < center[0]]  # 중심보다 왼쪽
    right_pts = pts[pts[:, 0] >= center[0]]  # 중심보다 오른쪽

    # 3️⃣ 좌측(L) 그룹 정렬
    left_pts = sorted(left_pts, key=lambda p: p[1])  # y값 기준 정렬 (위 → 아래)
    top_left, bottom_left = left_pts  # 위쪽이 좌상, 아래쪽이 좌하

    # 4️⃣ 우측(R) 그룹 정렬
    right_pts = sorted(right_pts, key=lambda p: p[1])  # y값 기준 정렬 (위 → 아래)
    top_right, bottom_right = right_pts  # 위쪽이 우상, 아래쪽이 우하

    # 5️⃣ 최종 정렬된 좌표 반환 (좌상, 우상, 우하, 좌하)
    return np.array([top_left, top_right, bottom_right, bottom_left], dtype=np.float32)

def find_boundary_points(mask, min_area=100, epsilon_factor=0.02):
    """
    마스크에서 4개의 코너 포인트를 찾고 좌상단(left_top)과 우상단(right_top)을 반환
    """
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    if not contours:
        return None, None  # 컨투어가 없으면 None 반환

    largest_contour = max(contours, key=cv2.contourArea)
```

```
    if cv2.contourArea(largest_contour) < min_area:
        return None, None  # 너무 작은 컨투어는 무시

    epsilon = epsilon_factor * cv2.arcLength(largest_contour, True)
    approx = cv2.approxPolyDP(largest_contour, epsilon, True)

    if len(approx) > 4:
        rect = cv2.minAreaRect(largest_contour)
        box = cv2.boxPoints(rect)
        approx = np.array(box, dtype=np.float32)
    elif len(approx) < 4:
        x, y, w, h = cv2.boundingRect(largest_contour)
        approx = np.array([[x, y], [x + w, y], [x + w, y + h], [x, y + h]], dtype=np.float32)

    if len(approx) != 4:
        return None, None

    ordered_points = order_points(approx.reshape(-1, 2))

    left_top = tuple(ordered_points[0])  # 좌상단
    right_top = tuple(ordered_points[1])  # 우상단

    return left_top, right_top

def detect_top_keypoints_with_convex_hull(image, model):
    """
    YOLO 모델을 이용해 객체의 키포인트를 감지하고, 좌상단(left_top)과 우상단(right_top)을 반환
    """
    results = model.predict(image)
    result = results[0]

    if not result.masks:
        return [], None, None

    all_points = []

    for mask in result.masks.xy:
        mask_points = np.array(mask, dtype=np.float32)
        all_points.extend(mask_points)

    all_points = np.array(all_points, dtype=np.float32)

    if len(all_points) < 3:
        return [], None, None

    mask = np.zeros(image.shape[:2], dtype=np.uint8)
    cv2.fillPoly(mask, [all_points.astype(np.int32)], 255)
```

```
    left_top, right_top = find_boundary_points(mask)

    return all_points.tolist(), left_top, right_top

def rotate_and_translate_image(image, left_point, right_point, all_points, target_x=350, target_y=1
70):
    """
    두 점을 기준으로 이미지를 회전하고, 모든 키포인트도 함께 변환 후 지정한 위치로 이동
    """
    dx, dy = right_point[0] - left_point[0], right_point[1] - left_point[1]
    angle = np.degrees(np.arctan2(dy, dx))

    if np.abs(dy) < 1e-6:
        angle = 0.0

    h, w = image.shape[:2]
    center_x = (left_point[0] + right_point[0]) / 2
    center_y = (left_point[1] + right_point[1]) / 2
    center = (int(center_x), int(center_y))

    rotation_matrix = cv2.getRotationMatrix2D(center, angle, 1.0)
    rotated_image = cv2.warpAffine(image, rotation_matrix, (w, h), flags=cv2.INTER_LINEAR, bord
erMode=cv2.BORDER_REPLICATE)

    rotated_all_points = [tuple(np.dot(rotation_matrix[:, :2], np.array([p[0], p[1]])) + rotation_matrix
[:, 2]) for p in all_points]
    rotated_left_top = np.dot(rotation_matrix[:, :2], np.array([left_point[0], left_point[1]])) + rotation
_matrix[:, 2]
    rotated_right_top = np.dot(rotation_matrix[:, :2], np.array([right_point[0], right_point[1]])) + rota
tion_matrix[:, 2]

    dx = int(target_x - rotated_left_top[0])
    dy = int(target_y - rotated_left_top[1])
    translation_matrix = np.float32([[1, 0, dx], [0, 1, dy]])

    translated_image = cv2.warpAffine(rotated_image, translation_matrix, (w, h), flags=cv2.INTER_
LINEAR, borderMode=cv2.BORDER_REPLICATE)

    translated_all_points = [(p[0] + dx, p[1] + dy) for p in rotated_all_points]
    translated_left_top = (rotated_left_top[0] + dx, rotated_left_top[1] + dy)
    translated_right_top = (rotated_right_top[0] + dx, rotated_right_top[1] + dy)

    return translated_image, translated_all_points, translated_left_top, translated_right_top

def draw_keypoints(image, points, color, size=3):
    """ 이미지에 점들을 그리는 함수 """
    for x, y in points:
        cv2.circle(image, (int(x), int(y)), size, color, -1)
```

```
def apply_bm3d_and_threshold(image, threshold_value=120):
    """ BM3D 필터 적용 후 이진화 """
    if image is None:
        return None

    if len(image.shape) == 3:  # 컬러 이미지를 Grayscale로 변환
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    gray = image.astype(np.float32) / 255.0
    bm3d_filtered = bm3d.bm3d(gray, sigma_psd=0.1, stage_arg=bm3d.BM3DStages.HARD_THRESHOLDING)

    bm3d_filtered = (bm3d_filtered * 255).astype(np.uint8)
    _, binary = cv2.threshold(bm3d_filtered, threshold_value, 255, cv2.THRESH_BINARY)
    return binary

def analyze_black_pixels(binary, transformed_left_top):
    """ 변환된 좌상단(transformed_left_top)을 기준으로 ROI에서 검정 픽셀 분석 """
    base_x, base_y = map(int, transformed_left_top)  # 변환된 좌상단을 정수형으로 변환

    # ROI 위치 설정 (transformed_left_top을 기준)
    rois = [
        {"x": base_x + 50, "y": base_y - 10, "w": 80, "h": 30, "group_index": 0},
        {"x": base_x + 220, "y": base_y - 10, "w": 80, "h": 30, "group_index": 0},
        {"x": base_x + 400, "y": base_y - 10, "w": 80, "h": 30, "group_index": 0},
        {"x": base_x + 50, "y": base_y + 210, "w": 80, "h": 30, "group_index": 0},
        {"x": base_x + 220, "y": base_y + 210, "w": 80, "h": 30, "group_index": 0},
        {"x": base_x + 400, "y": base_y + 210, "w": 80, "h": 30, "group_index": 0},
    ]

    binary_colored = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
    black_pixel_results = []

    for roi in rois:
        x, y, w, h, group_index = map(int, [roi["x"], roi["y"], roi["w"], roi["h"], roi["group_index"]])

        # ROI가 이미지 크기를 초과하지 않도록 보정
        x = max(0, x)
        y = max(0, y)
        w = min(binary.shape[1] - x, w)
        h = min(binary.shape[0] - y, h)

        # ROI 내부에서 6개 샘플 X 좌표를 선택 (정수 변환)
        num_samples = 6
        step = max(1, w // (num_samples + 1))  # step이 0이 되는 것을 방지
        sample_x_positions = np.round(np.linspace(x + step, x + w - step, num_samples)).astype(int)

        for center_x in sample_x_positions:
```

```
            black_pixels_y = np.where(binary[y:y+h, center_x] == 0)[0] + y
            min_black_length = 5  # 너무 짧은 노이즈 필터링

            if len(black_pixels_y) > 1:
                groups = np.split(black_pixels_y, np.where(np.diff(black_pixels_y) > 1)[0] + 1)
                long_black_groups = [group for group in groups if len(group) > min_black_length]

                if len(long_black_groups) > group_index:
                    selected_group = long_black_groups[group_index]
                    start_y, end_y = int(min(selected_group)), int(max(selected_group))
                    black_pixel_length = end_y - start_y
                    black_pixel_results.append((x, center_x, start_y, end_y, black_pixel_length))

    # ROI 및 검정 픽셀 길이 표시
    for roi in rois:
        x, y, w, h = map(int, [roi["x"], roi["y"], roi["w"], roi["h"]])
        cv2.rectangle(binary_colored, (x, y), (x+w, y+h), (0, 255, 0), 2)  # ROI 표시

    for x, center_x, start_y, end_y, _ in black_pixel_results:
        cv2.line(binary_colored, (center_x, start_y), (center_x, end_y), (0, 0, 255), 2)  # 검정 픽셀 길이
표시

    return binary_colored

def process_and_save_images(image_dir, output_dir, model_path, max_images=500):
    """
    주어진 폴더 내 임계값 개수만큼의 이미지에 YOLO 객체 탐지 및 정렬 적용 후 저장.
    """
    model = YOLO(model_path)
    os.makedirs(output_dir, exist_ok=True)

    image_files = [f for f in os.listdir(image_dir) if f.endswith(('jpg', 'jpeg', 'png'))][:max_images]

    for image_file in image_files:
        image_path = os.path.join(image_dir, image_file)
        image = cv2.imread(image_path)

        if image is None:
            print(f"❌ 이미지를 불러올 수 없음: {image_path}")
            continue

        try:
            # 🔵 YOLO를 이용해 객체의 키포인트 탐지
            all_points, left_top, right_top = detect_top_keypoints_with_convex_hull(image, model)

            if left_top is None or right_top is None:
                print(f"⚠️ 코너 포인트를 찾을 수 없음: {image_file}")
                continue
```

```
            # 🔵 이미지 정렬 및 변환
            final_image, transformed_all_points, transformed_left_top, transformed_right_top = rotate_
and_translate_image(
                image, left_top, right_top, all_points
            )

            # 🟢 BM3D 필터 적용 후 이진화
            binary = apply_bm3d_and_threshold(final_image)
            if binary is None:
                print(f"❌ 필터링 및 이진화 실패: {image_file}")
                continue

            # 🔴 ROI에서 검정 픽셀 분석 및 시각화
            binary_colored = analyze_black_pixels(binary, transformed_left_top)

            # 📌 결과 저장
            output_path = os.path.join(output_dir, image_file)
            cv2.imwrite(output_path, binary_colored)
            print(f"✅ 처리 완료: {output_path}")

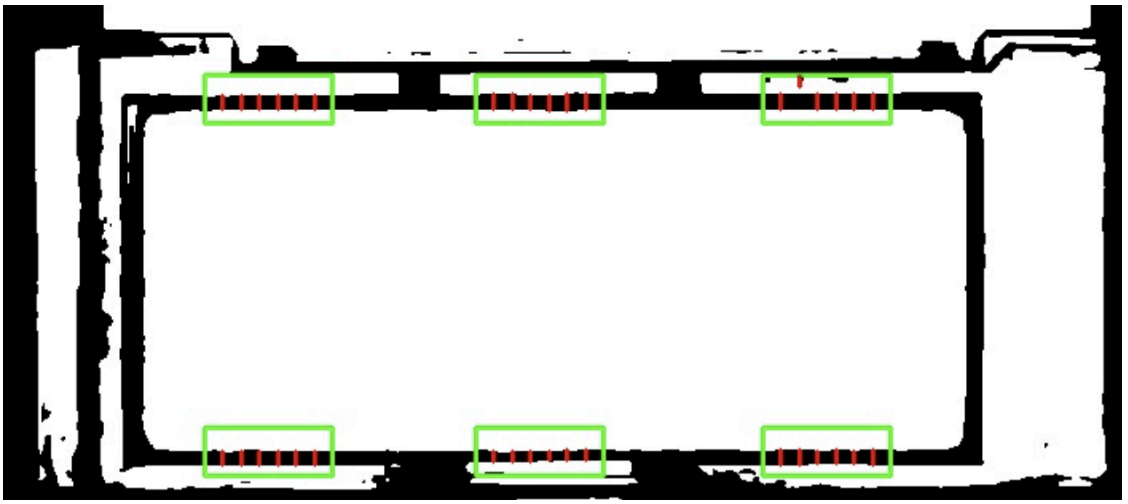
        except Exception as e:
            print(f"❌ 오류 발생 [{image_file}]: {e}")

    print("🔵 Batch processing complete.")

if __name__ == "__main__":
    image_dir = "data/3560_Magnet Check/P001/IV00052/0000"
    output_dir = "./output"
    model_path = "yolo/3560_MagnetCheck_v8s_white.pt"
    max_images = 500

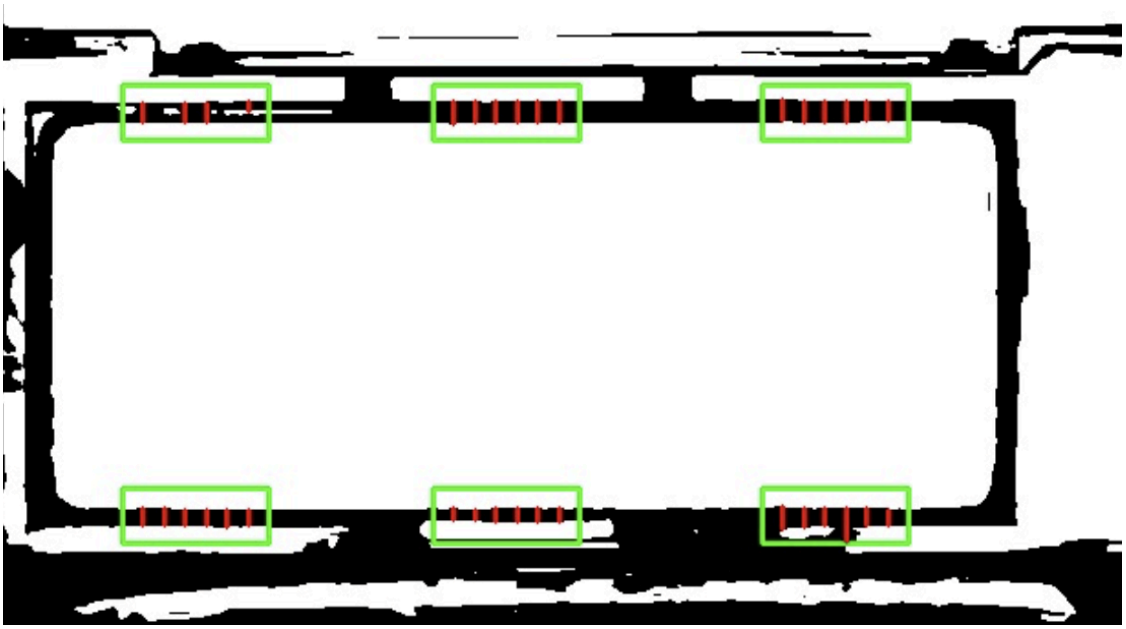
    process_and_save_images(image_dir, output_dir, model_path, max_images)
```

04210_001_OK.jpeg



노이즈로 픽셀 크기 바꿀지 , 필터가 문제인건지

03748_001_OK



2025-02-25

이진화한 결과 문제

→ 필터를 변경해 보자

근데 좋은 필터의 기준을 뭘로 잡을지?

각 ROI 내의 갭들의 편차가 큰 것?

편차가 크다는건? ,

이상치 값이 있는지?

어떻게 확인 할 건데?

필터 적용 걸리는 시간

SNR(신호 대 잡음 비율, Signal-to-Noise Ratio)

이거 이진화 전에 적용하고 확인해 봐야함.

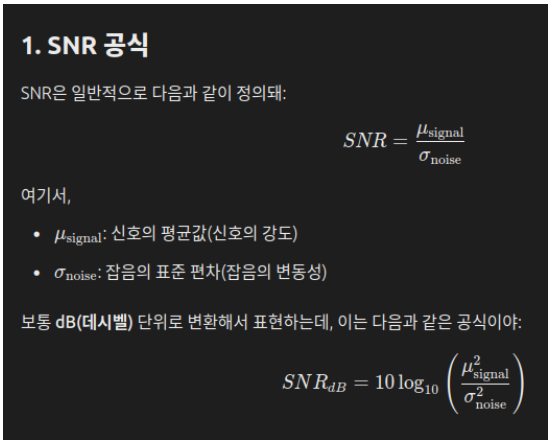
이진화 하면 0 아니면 255만 남아서 잡음의 표준편차가 의미가 없어짐

→내가 지금 갭까지 구해놨는데 다시 이렇게 한다?,

갭을 활용하는 방식으로 가는게...

SNR은 신호(유용한 데이터)와 잡음(원하지 않는 변동 요소)의 비율을 나타내는 값

높을수록 신호가 명확하고 잡음이 적다는 의미고, **낮을수록** 신호가 약하고 잡음이 많다는 뜻



(1) 신호(Signal)와 잡음(Noise) 정의

이미지에서 SNR을 계산하려면 **신호**와 **잡음**을 정의해야 해.

- **신호 (Signal)**: 원하는 객체의 픽셀 값

  → 예를 들어, ROI 내 검은색 픽셀(필요한 특징)을 신호로 볼 수 있어.

- **잡음 (Noise)**: 불필요한 픽셀 값의 변동성

  → 배경 영역(흰색, 회색)에서의 픽셀 값 변동을 잡음으로 간주할 수 있어.

2025-02-26

오늘 할 일 : 필터 적용해서 갭 측정하는 코드 만들고 , 필터 적용해보기

필터를 씌웠을 때 , 각 ROI 내에서 이상치 있는지 확인하고 필터 당 이상치있는 ROI 개수 출력+ 결과 사진 이미지 필터별 저장, 이상치 있는 사진 로그 출력

검정색 그룹 선택할 때, 가장 긴 그룹? 하니면 제일 첫번째 그룹?

```
import cv2
import numpy as np
import bm3d
import os

def apply_filter(image, filter_type):
    """ 필터 적용 """
    if filter_type == 'blur':
        return cv2.GaussianBlur(image, (15, 15), 0)
    elif filter_type == 'sharpen':
        kernel = np.array([[0, -1, 0],
                           [-1, 5, -1],
```

```
                    [0, -1, 0]])
        return cv2.filter2D(image, -1, kernel)
    elif filter_type == 'grayscale':
        return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    elif filter_type == 'bm3d':
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY).astype(np.float32) / 255.0
        bm3d_filtered = bm3d.bm3d(gray, sigma_psd=0.1, stage_arg=bm3d.BM3DStages.HARD_TH
RESHOLDING)
        return (bm3d_filtered * 255).astype(np.uint8)
    else:
        print("지원되지 않는 필터 유형입니다.")
        return None

def apply_threshold(image, threshold_value=120):
    """ 이진화 처리 """
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(image, threshold_value, 255, cv2.THRESH_BINARY)
    return binary

def analyze_black_pixels(binary):
    """ ROI에서 연속된 검정 픽셀의 평균과 이상치 탐지 """
    rois = [
        {"x": 400, "y": 160, "w": 80, "h": 30},
        {"x": 570, "y": 160, "w": 80, "h": 30},
        {"x": 750, "y": 160, "w": 80, "h": 30},
        {"x": 400, "y": 380, "w": 80, "h": 30},
        {"x": 570, "y": 380, "w": 80, "h": 30},
        {"x": 750, "y": 380, "w": 80, "h": 30},
    ]

    binary_colored = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
    all_gaps = []
    roi_results = []
    threshold_factor = 1.5  # 평균에서 벗어날 경우 이상치 판별

    for roi in rois:
        x, y, w, h = roi["x"], roi["y"], roi["w"], roi["h"]
        num_samples = 6
        step = max(1, w // (num_samples + 1))
        sample_x_positions = np.round(np.linspace(x + step, x + w - step, num_samples)).astype(in
t)
        roi_gaps = []

        for center_x in sample_x_positions:
            black_pixels_y = np.where(binary[y:y+h, center_x] == 0)[0] + y
            if len(black_pixels_y) > 1:
                groups = np.split(black_pixels_y, np.where(np.diff(black_pixels_y) > 1)[0] + 1)
                long_black_groups = [group for group in groups if len(group) > 5]
```

```
            if long_black_groups:
                longest_group = max(long_black_groups, key=len)
                start_y, end_y = min(longest_group), max(longest_group)
                black_pixel_length = end_y - start_y
                roi_gaps.append(black_pixel_length)

                # 검출된 갭에 선을 긋기
                cv2.line(binary_colored, (center_x, start_y), (center_x, end_y), (0, 0, 255), 2)

        all_gaps.extend(roi_gaps)
        roi_results.append((roi, roi_gaps))
        cv2.rectangle(binary_colored, (x, y), (x + w, y + h), (0, 255, 0), 2)  # ROI 그리기

    missing_gaps = sum(1 for roi, gaps in roi_results if not gaps)
    outlier_gaps = sum(1 for gap in all_gaps if gap < np.mean(all_gaps) - threshold_factor * np.std
(all_gaps) or gap > np.mean(all_gaps) + threshold_factor * np.std(all_gaps))
    small_gaps = sum(1 for gap in all_gaps if gap <= 5)

    return binary_colored, missing_gaps, outlier_gaps, small_gaps

def process_images_in_folder(folder_path, output_prefix, max_images=500):
    """ 폴더 내 최대 500개의 이미지 처리 및 분석 통계 계산 """
    base_folder = "filter_observation"
    filter_types = ['blur', 'sharpen', 'grayscale', 'bm3d']

    summary = {filter_type: {'missing': 0, 'outlier': 0, 'small': 0} for filter_type in filter_types}

    for filter_type in filter_types:
        os.makedirs(os.path.join(base_folder, filter_type), exist_ok=True)
        for category in ["missing", "outlier", "small"]:
            os.makedirs(os.path.join(base_folder, filter_type, category), exist_ok=True)

    image_files = [f for f in os.listdir(folder_path) if f.lower().endswith(('png', 'jpg', 'jpeg'))][:max_i
mages]

    for image_file in image_files:
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path)
        if image is None:
            continue

        for filter_type in filter_types:
            filtered_image = apply_filter(image, filter_type)
            if filtered_image is not None:
                binary_image = apply_threshold(filtered_image)
                analyzed_image, missing, outlier, small = analyze_black_pixels(binary_image)

                base_path = os.path.join(base_folder, filter_type, f"{output_prefix}_{image_file}")
```

```
            cv2.imwrite(base_path, analyzed_image, [cv2.IMWRITE_JPEG_QUALITY, 95])

            if missing:
                summary[filter_type]['missing'] += 1
                cv2.imwrite(os.path.join(base_folder, filter_type, "missing", f"{output_prefix}_{image_
file}"), analyzed_image, [cv2.IMWRITE_JPEG_QUALITY, 95])
            if outlier:
                summary[filter_type]['outlier'] += 1
                cv2.imwrite(os.path.join(base_folder, filter_type, "outlier", f"{output_prefix}_{image_fi
le}"), analyzed_image, [cv2.IMWRITE_JPEG_QUALITY, 95])
            if small:
                summary[filter_type]['small'] += 1
                cv2.imwrite(os.path.join(base_folder, filter_type, "small", f"{output_prefix}_{image_fil
e}"), analyzed_image, [cv2.IMWRITE_JPEG_QUALITY, 95])

    for filter_type, counts in summary.items():
        print(f"필터: {filter_type} | 갭 없음: {counts['missing']}, 이상치 갭: {counts['outlier']}, 5픽셀 미만
갭: {counts['small']}")

# 사용 예시
process_images_in_folder('rotated_translated_images', 'output',20)
```

```
필터: blur | 갭 없음: 4, 이상치 갭: 20, 5픽셀 미만 갭: 18
필터: sharpen | 갭 없음: 8, 이상치 갭: 19, 5픽셀 미만 갭: 19
필터: grayscale | 갭 없음: 0, 이상치 갭: 20, 5픽셀 미만 갭: 11
필터: bm3d | 갭 없음: 0, 이상치 갭: 20, 5픽셀 미만 갭: 7
```

2025-02-27

threshold_factor = 3.5 # 표준편차 기준
std_threshold = 2.0 # 표준편차가 너무 작으면 필터링 비활성화

500장 기준

필터: blur | 갭 없음: 93, 이상치 갭: 59
필터: sharpen | 갭 없음: 136, 이상치 갭: 51
필터: grayscale | 갭 없음: 0, 이상치 갭: 99
필터: bm3d | 갭 없음: 2, 이상치 갭: 84
필터: mean | 갭 없음: 12, 이상치 갭: 76
필터: gaussian | 갭 없음: 1, 이상치 갭: 88
필터: median | 갭 없음: 0, 이상치 갭: 108
필터: bilateral | 갭 없음: 0, 이상치 갭: 127
필터: anisotropic | 갭 없음: 0, 이상치 갭: 103

필터: wiener | 갭 없음: 500, 이상치 갭: 0

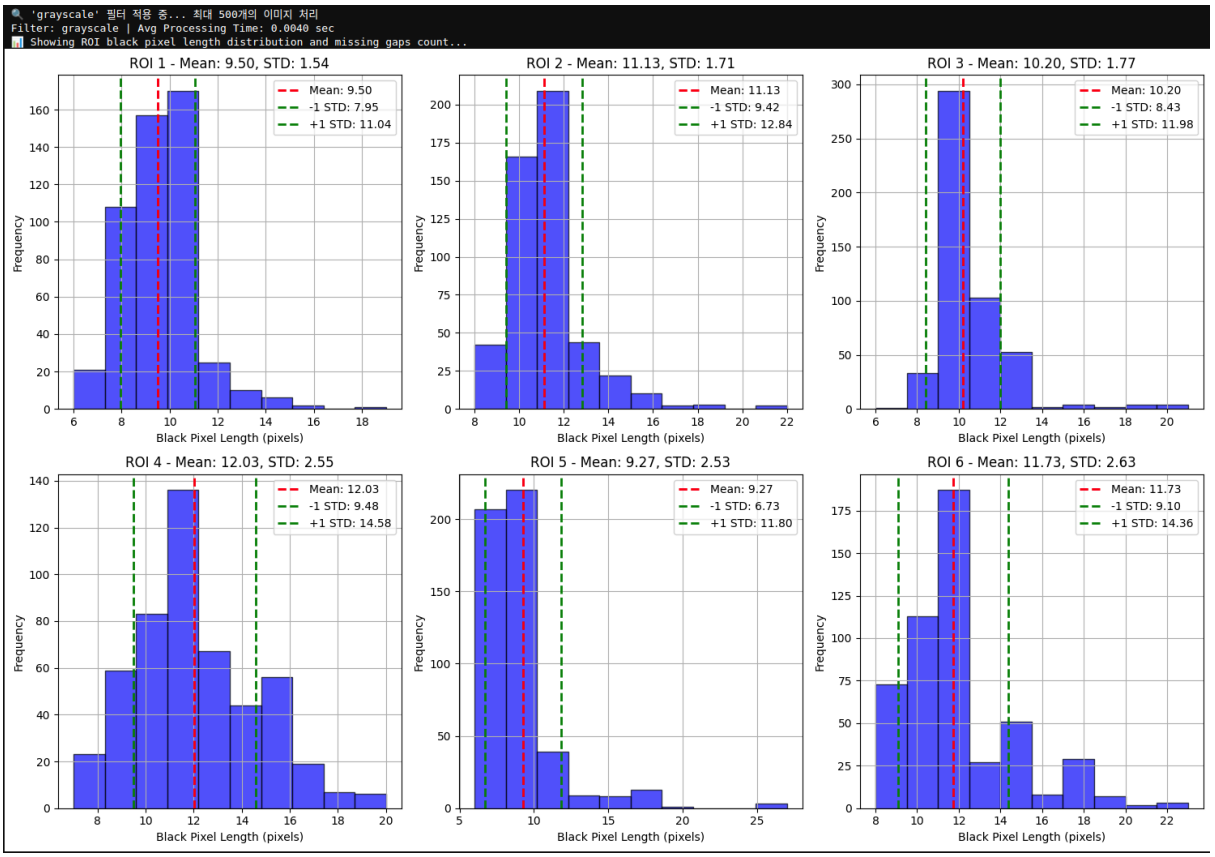
2025-02-28

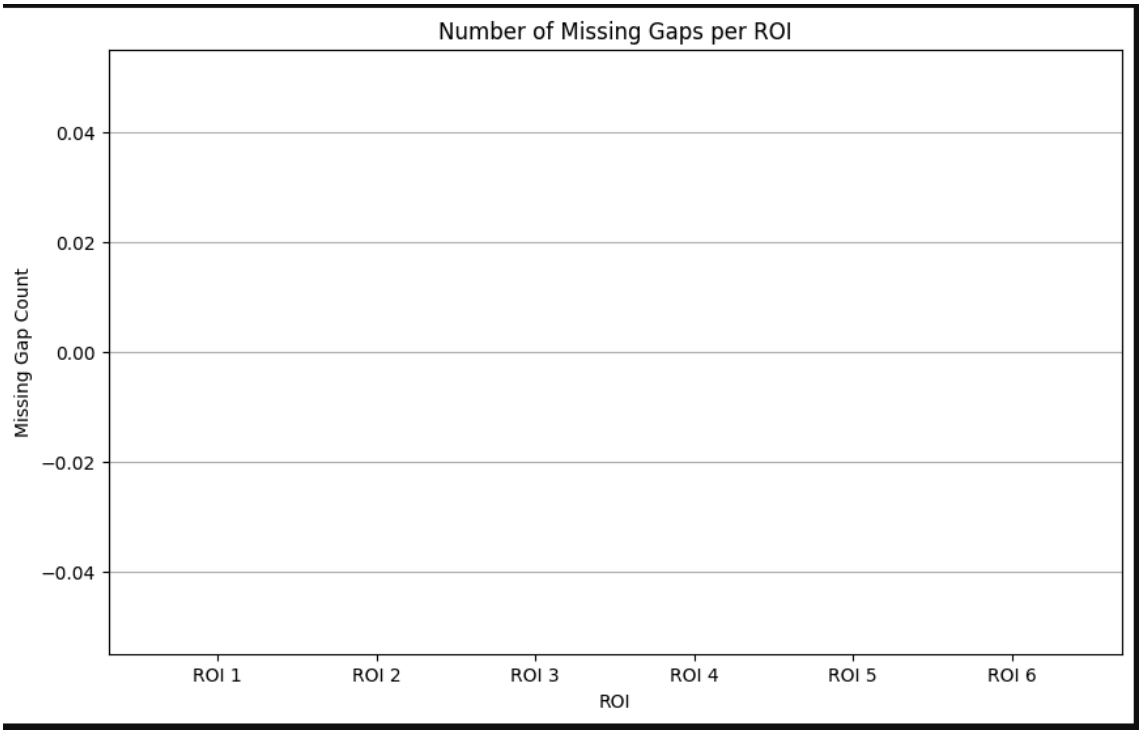
각 필터마다 갭의 분포 확인

grayscale

500장 , 필터 평균 전체 처리시간 0.0040sec, 4.0ms

return cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # 바로 반환 (BGR 변환 필요 X)

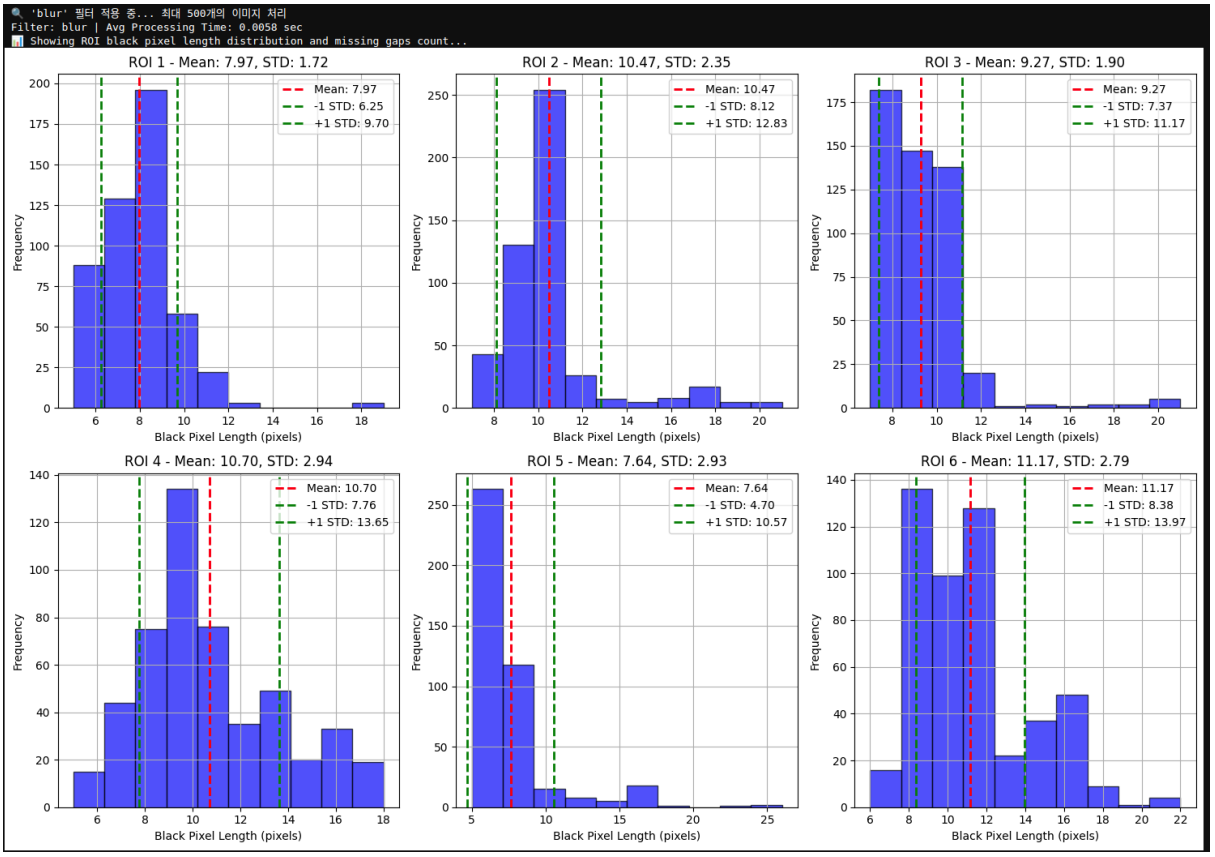


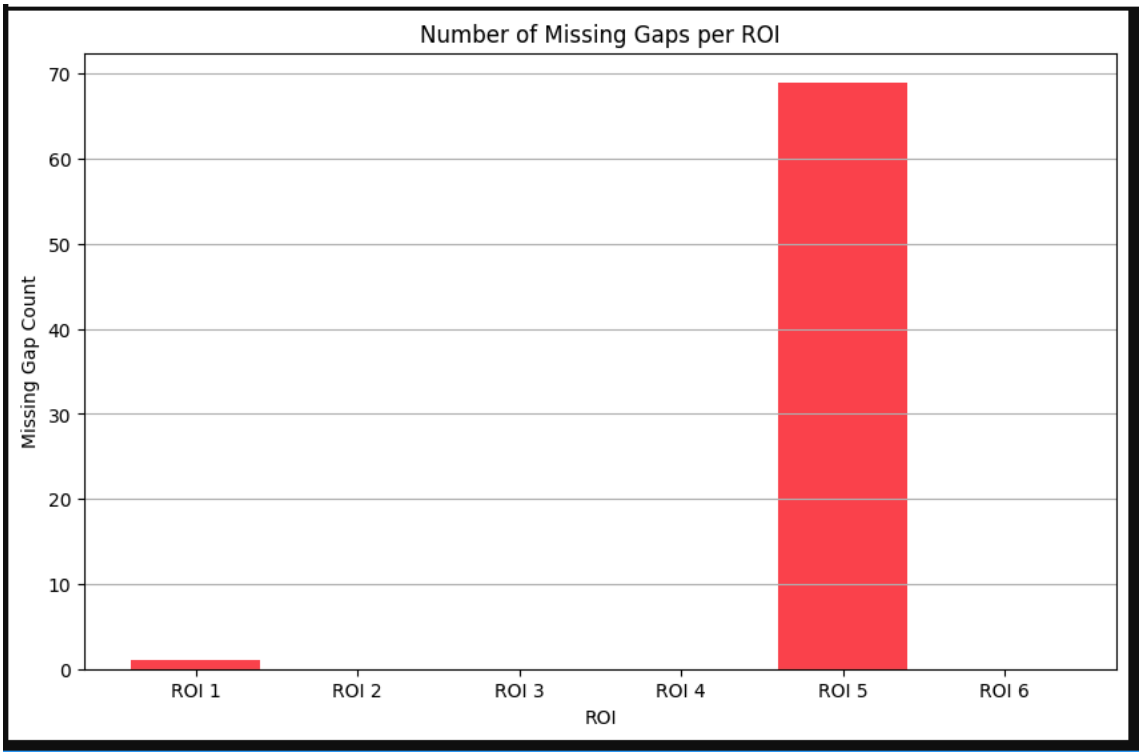


blur

500장 , 필터 평균 전체 처리시간 0.0047sec, 4.7ms

processed_image = cv2.GaussianBlur(image, (15, 15), 0)

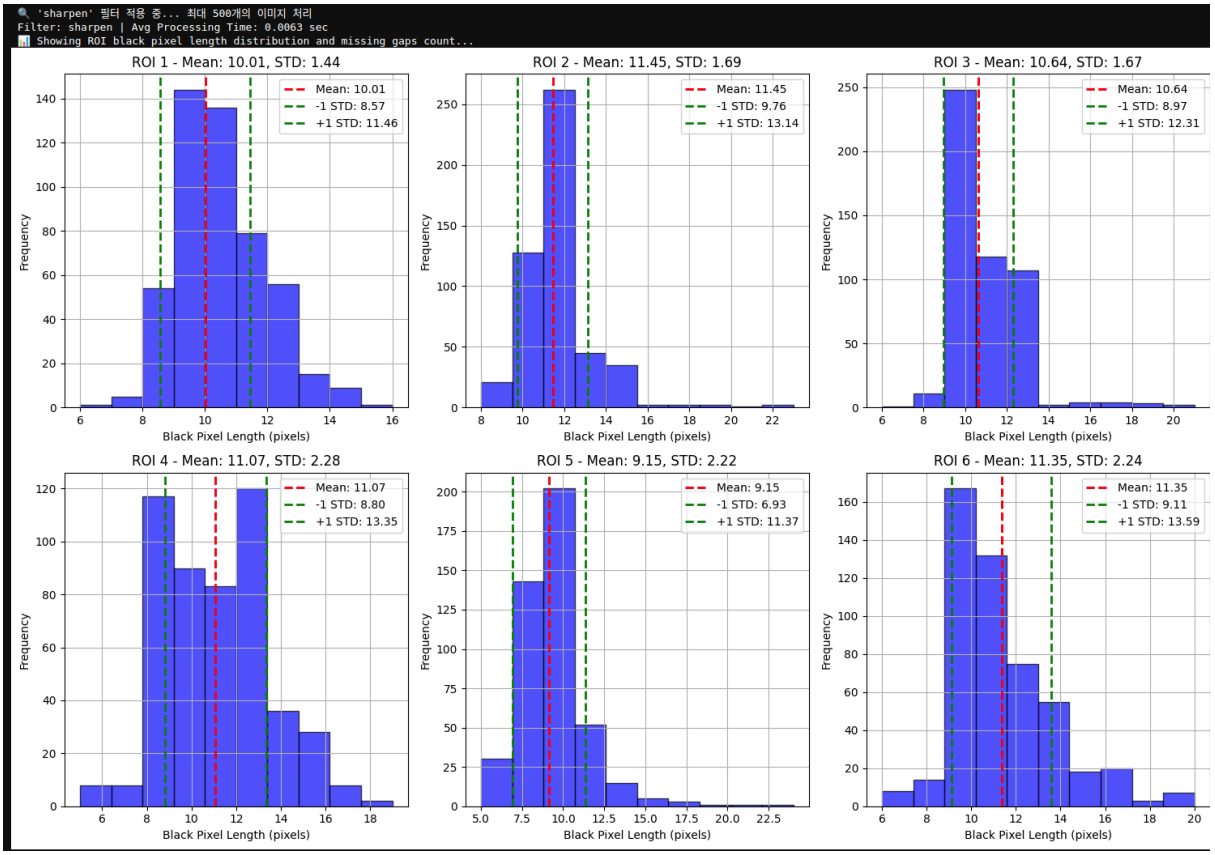


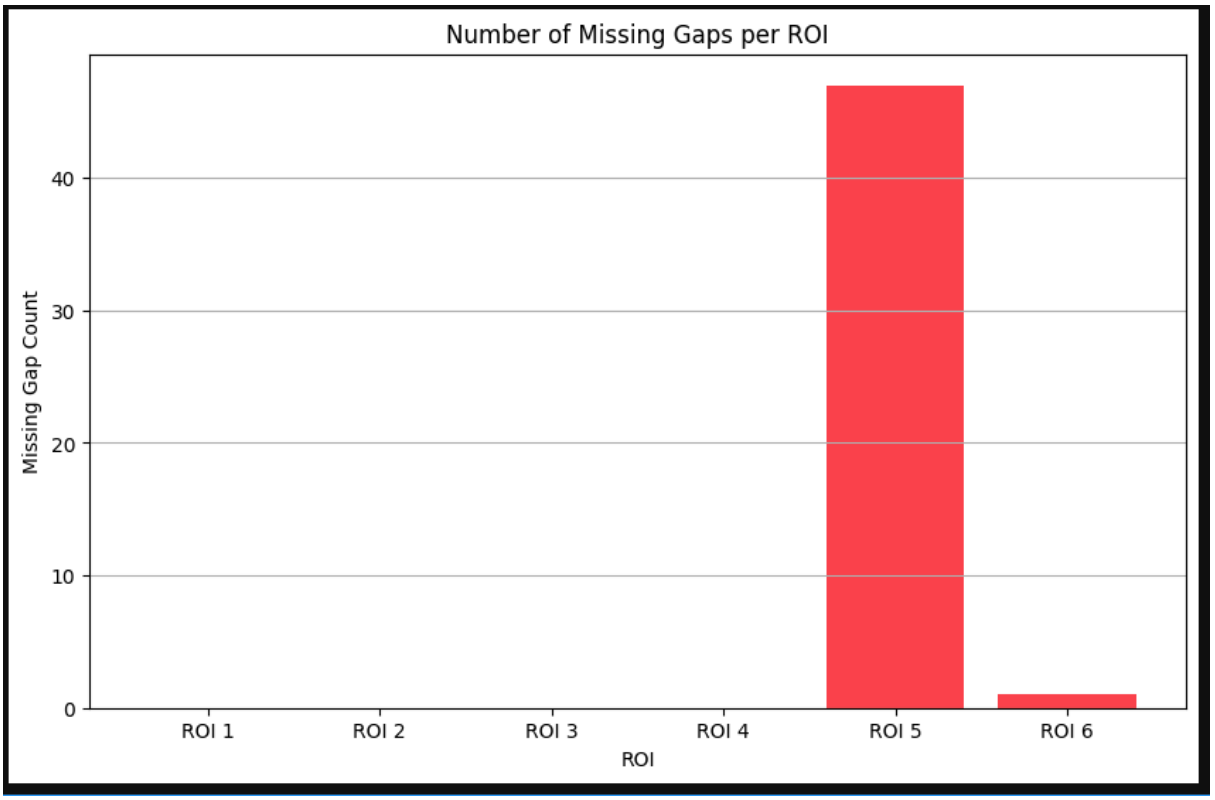


sharpen

500장 , 필터 평균 전체 처리시간 0.0051sec, 5.1ms

kernel = np.array([[0, -1, 0], [-1, 5, -1], [0, -1, 0]])
processed_image = cv2.filter2D(image, -1, kernel)


'sharpen' 필터 적용 중... 최대 500개의 이미지 처리
Filter: sharpen | Avg Processing Time: 0.0063 sec
Showing ROI black pixel length distribution and missing gaps count...
ROI 1 - Mean: 10.01, STD: 1.44
Mean: 10.01
-1 STD: 8.57
+1 STD: 11.46
ROI 2 - Mean: 11.45, STD: 1.69
Mean: 11.45
-1 STD: 9.76
+1 STD: 13.14
ROI 3 - Mean: 10.64, STD: 1.67
Mean: 10.64
-1 STD: 8.97
+1 STD: 12.31
ROI 4 - Mean: 11.07, STD: 2.28
Mean: 11.07
-1 STD: 8.80
+1 STD: 13.35
ROI 5 - Mean: 9.15, STD: 2.22
Mean: 9.15
-1 STD: 6.93
+1 STD: 11.37
ROI 6 - Mean: 11.35, STD: 2.24
Mean: 11.35
-1 STD: 9.11
+1 STD: 13.59
Frequency
Black Pixel Length (pixels)

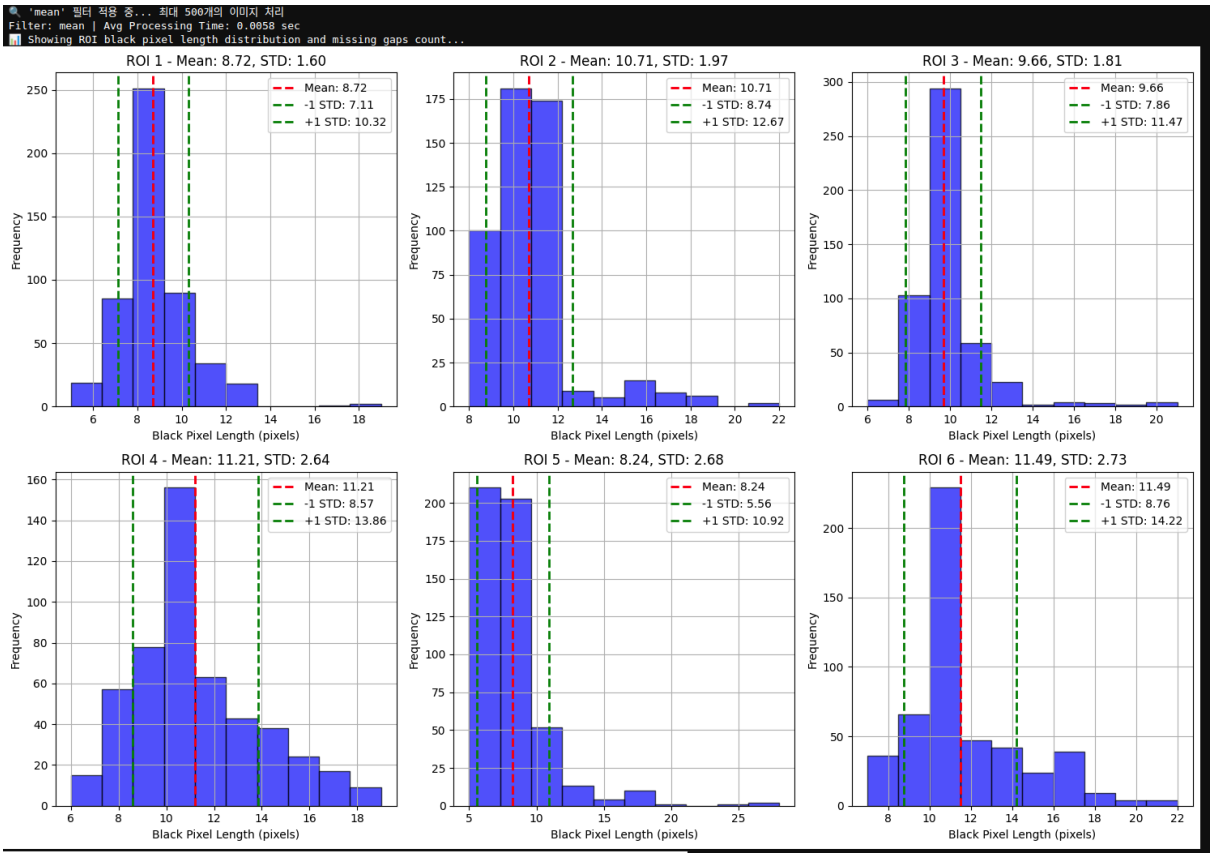

Number of Missing Gaps per ROI
Missing Gap Count
ROI
ROI 1
ROI 2
ROI 3
ROI 4
ROI 5
ROI 6

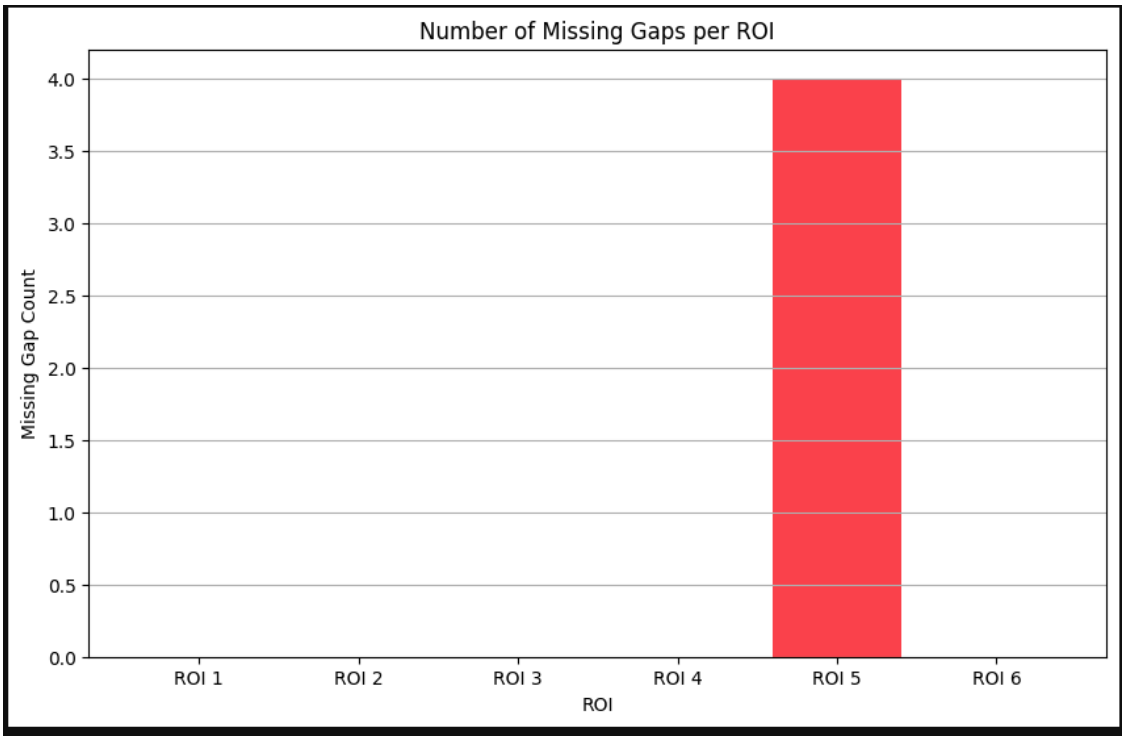
bm3d

mean

Filter: mean | Avg Processing Time: 0.0046 sec

processed_image = cv2.blur(image, (5, 5))

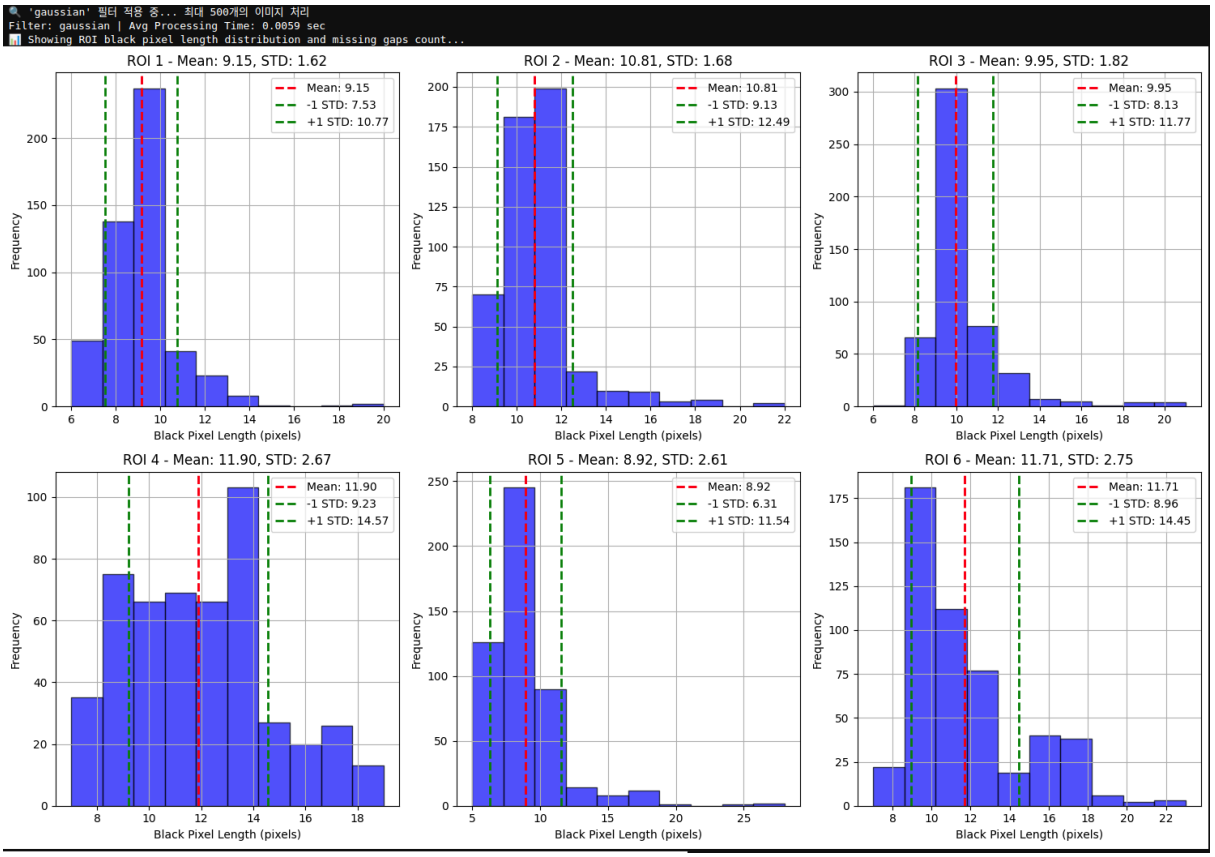


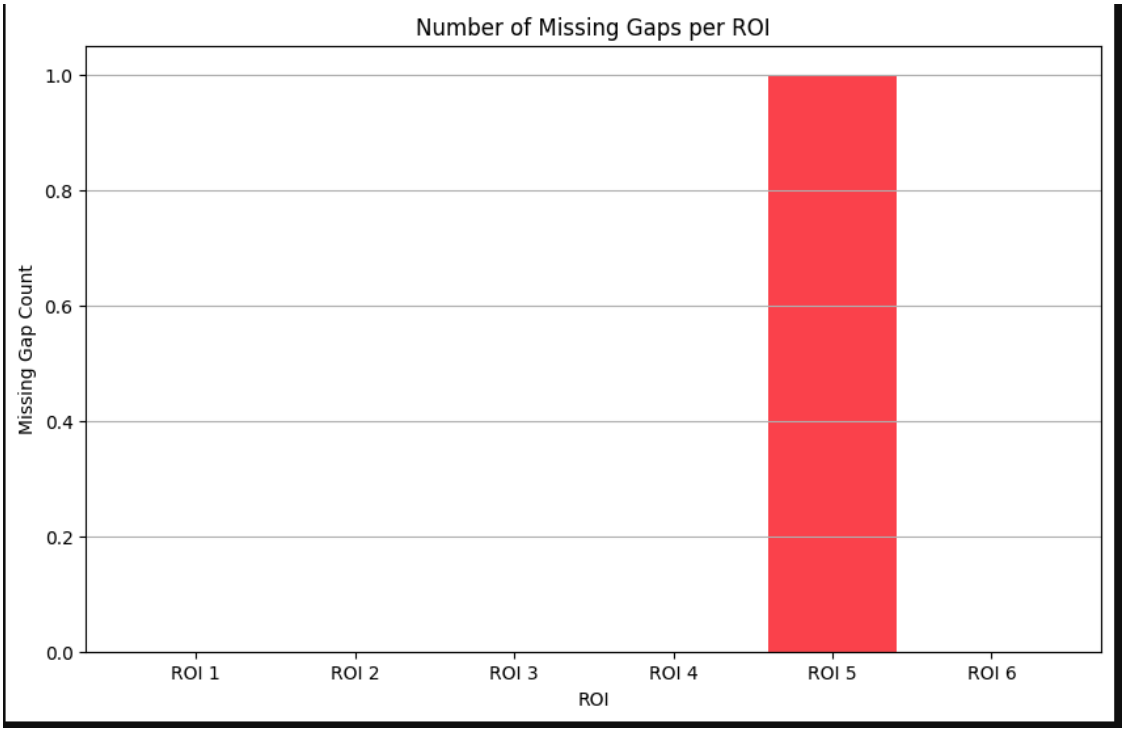


gaussian

Filter: gaussian | Avg Processing Time: 0.0042 sec

processed_image = cv2.GaussianBlur(image, (5, 5), 1.0)

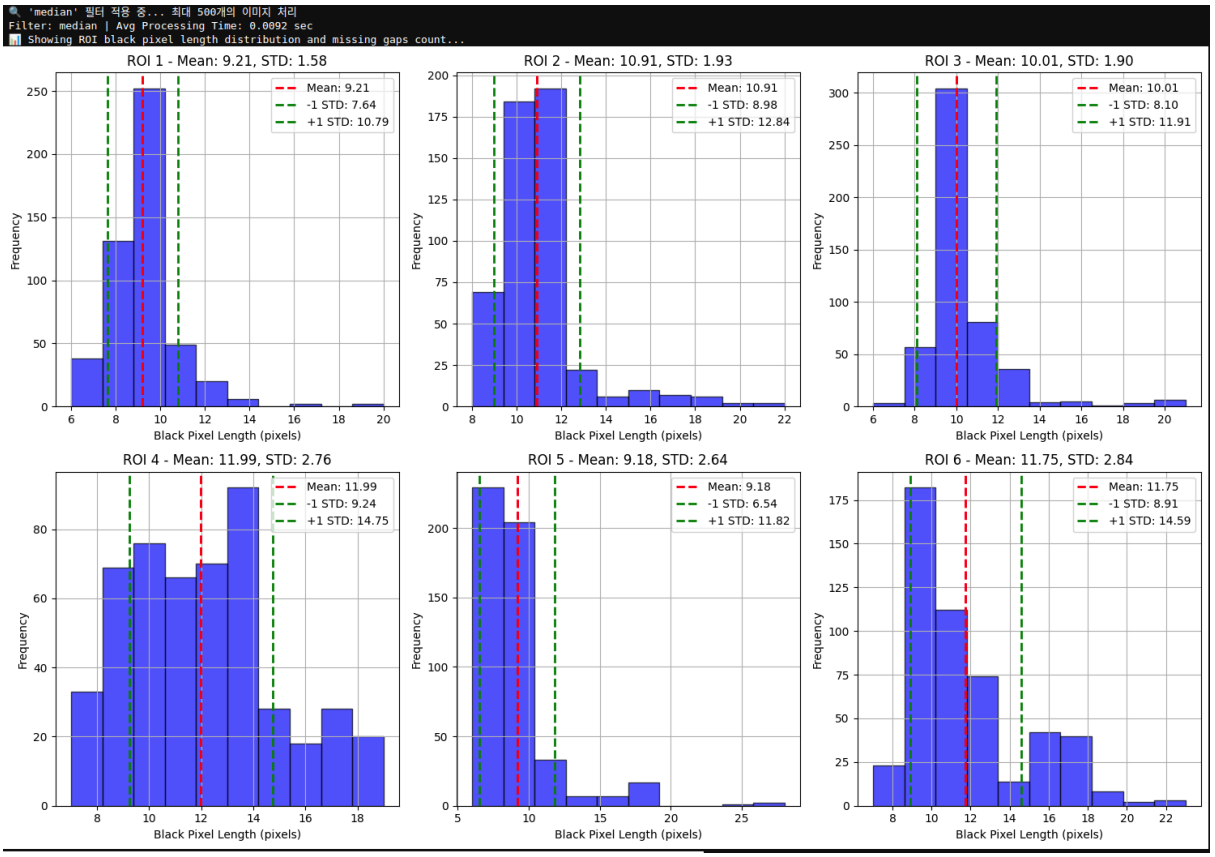


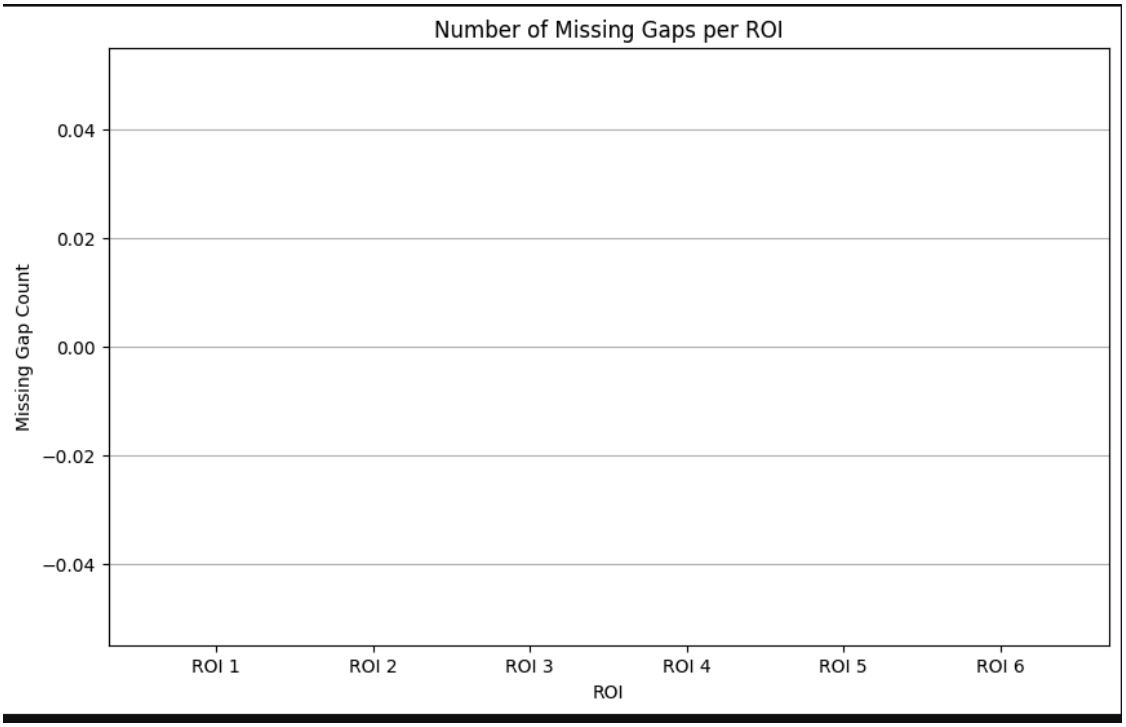


median

Filter: median | Avg Processing Time: 0.0079 sec

processed_image = cv2.medianBlur(image, 5)

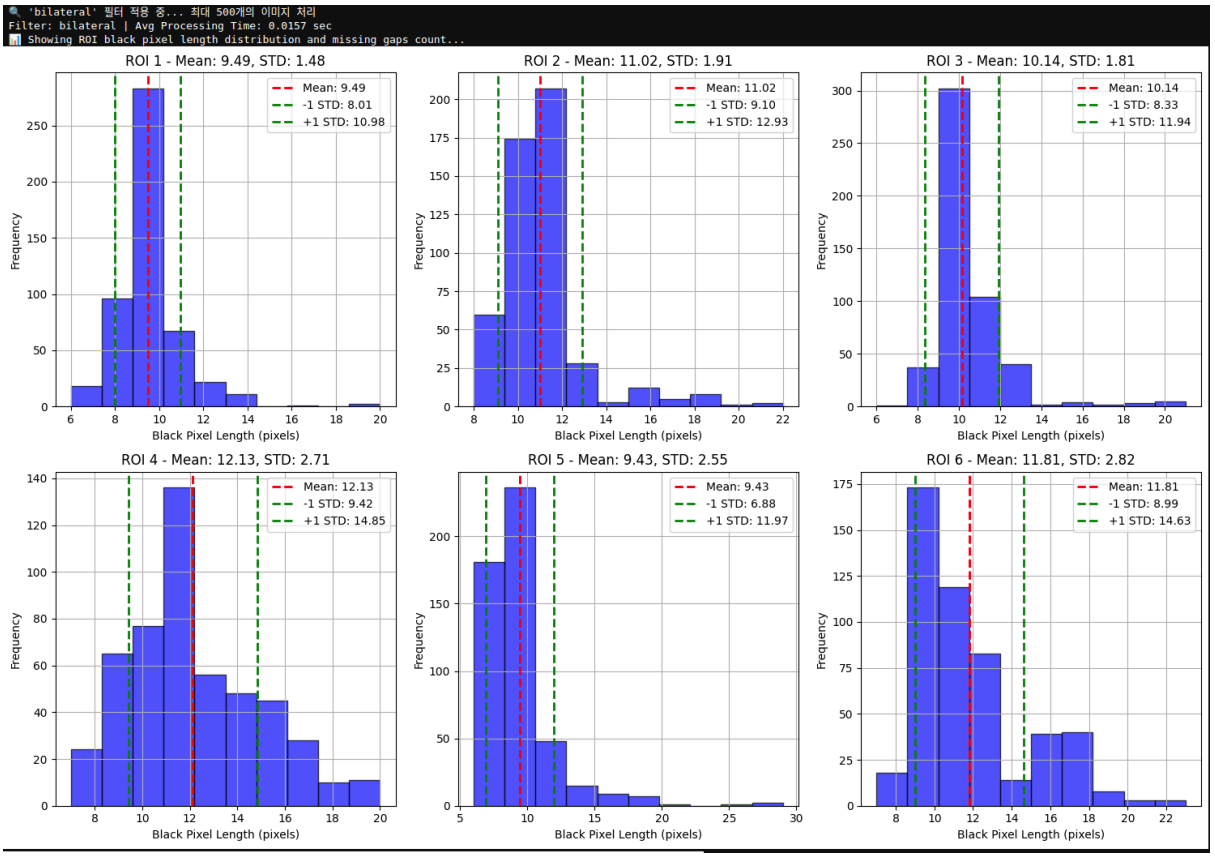


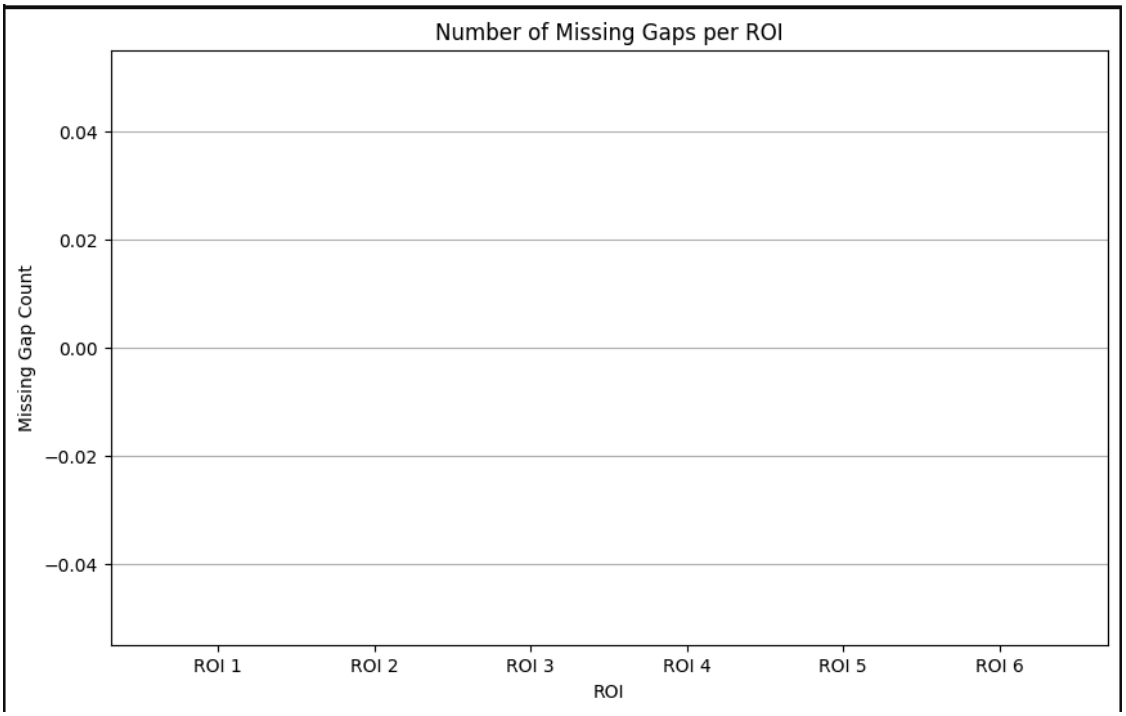


bilateral

Filter: bilateral | Avg Processing Time: 0.0146 sec

processed_image = cv2.bilateralFilter(image, 9, 75, 75)

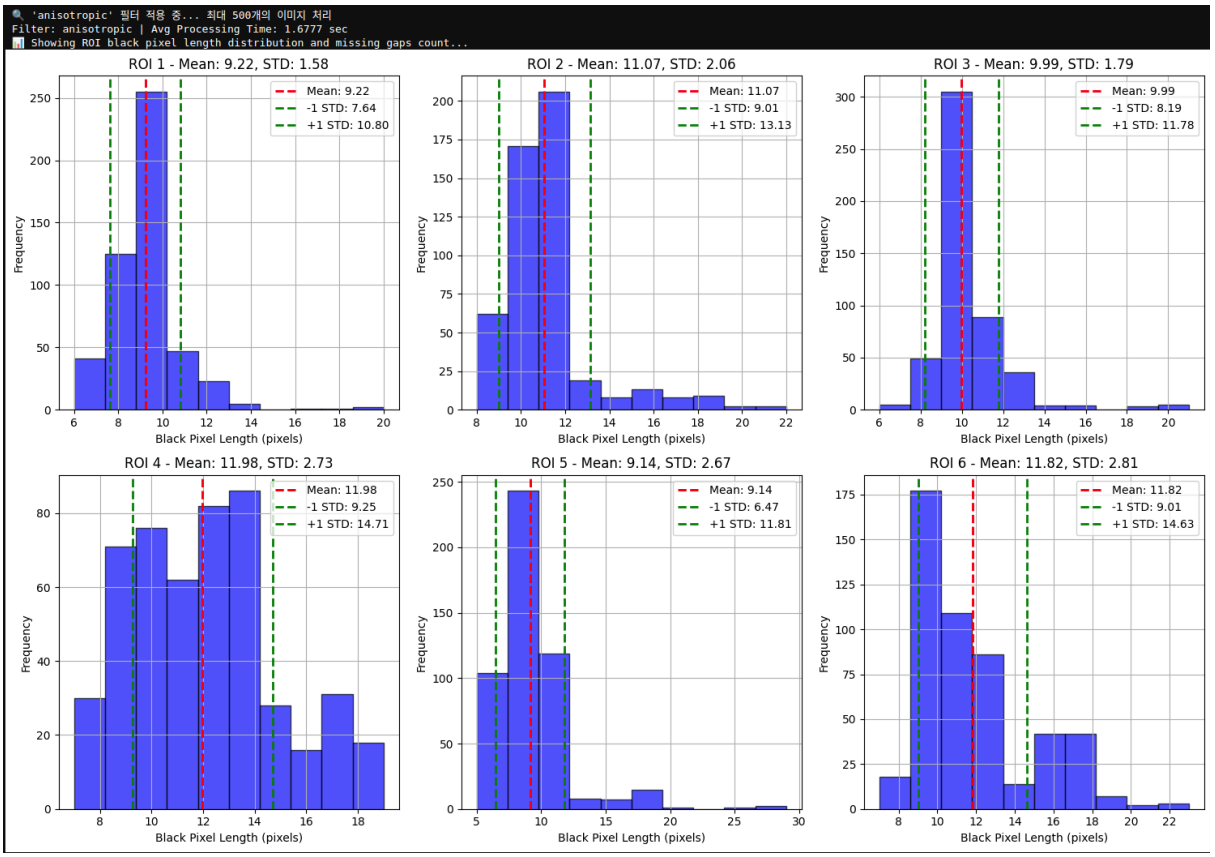


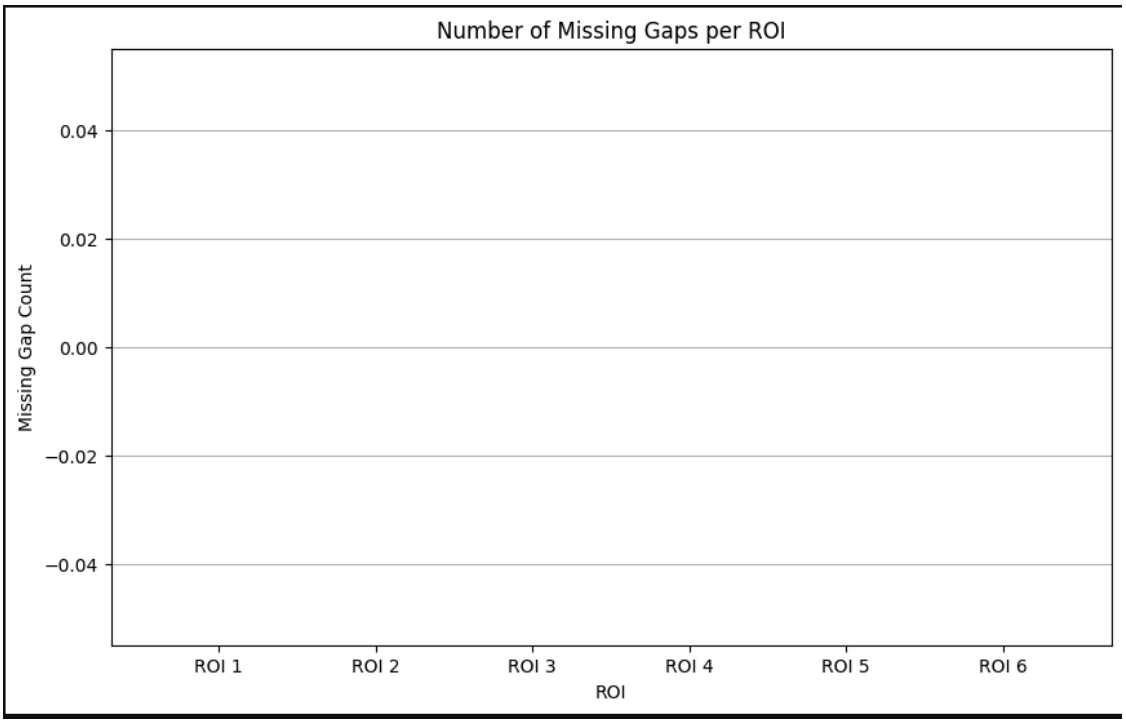


anisotropic

Filter: anisotropic | Avg Processing Time: 1.6805 sec

processed_image = (restoration.denoise_tv_chambolle(image, weight=0.1) * 255).astype(np.uint8)

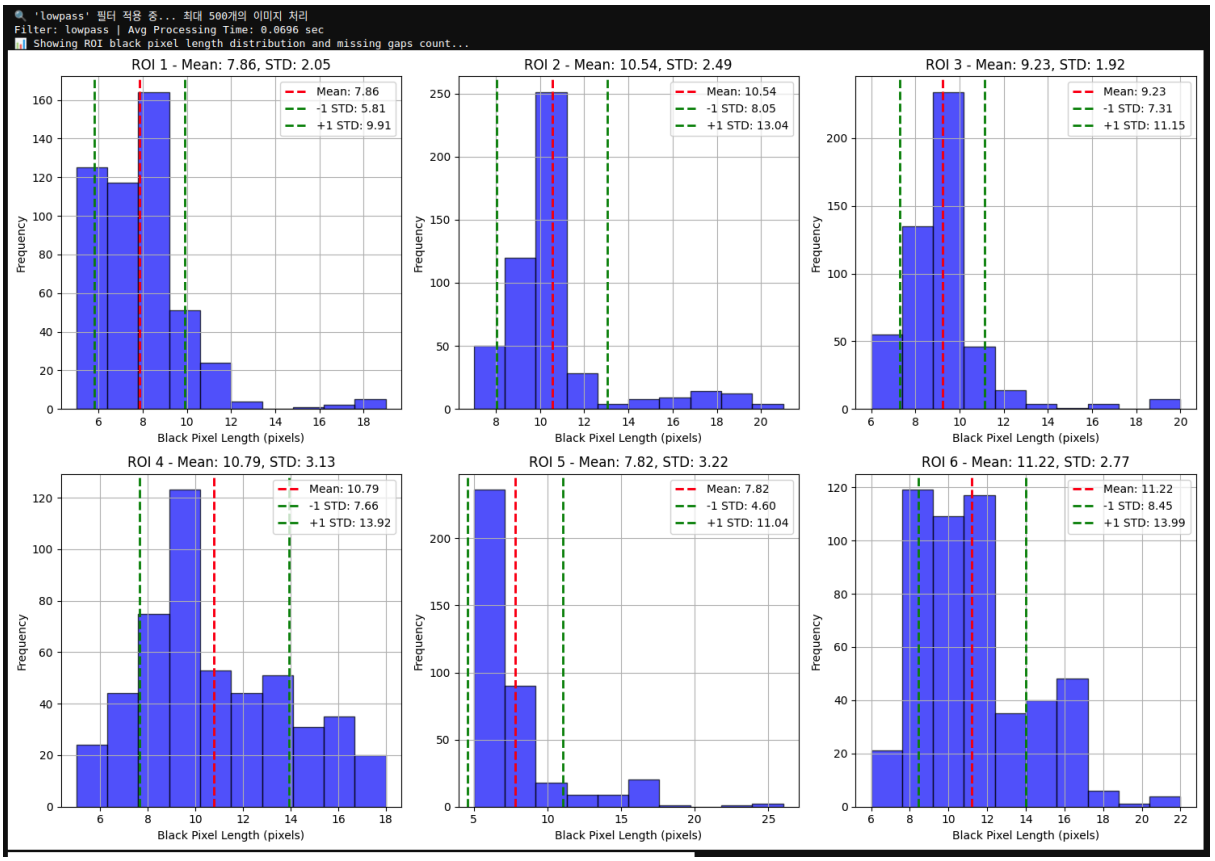


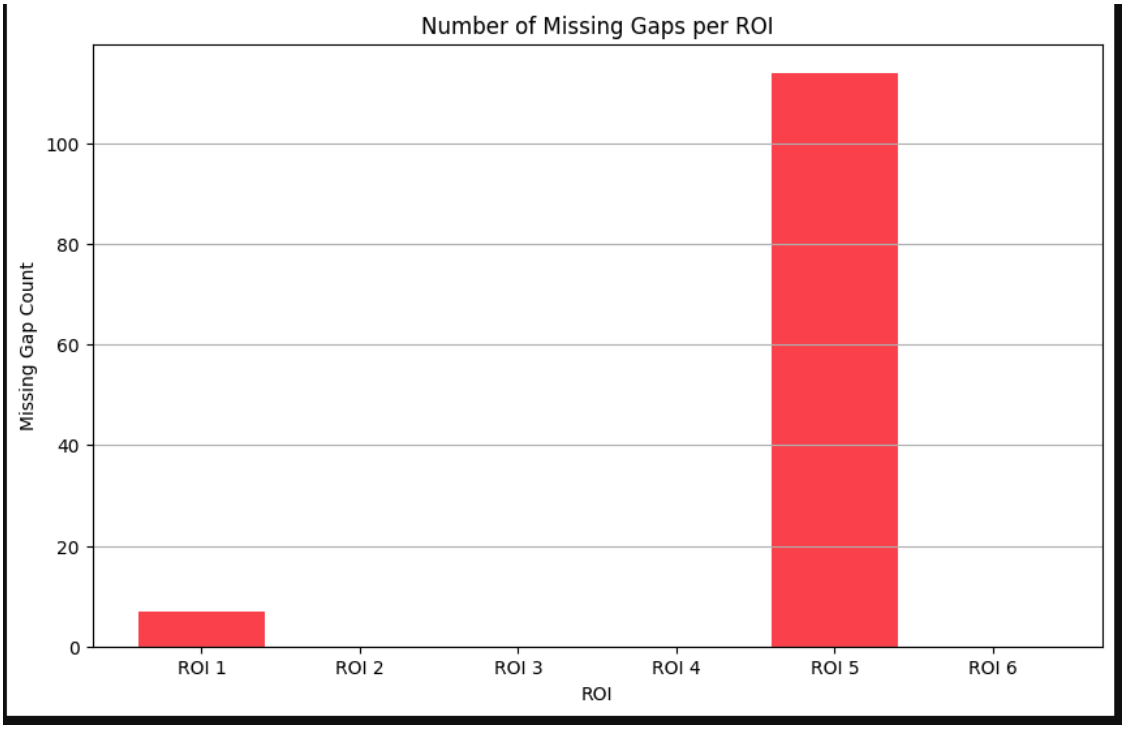


nlm(200장)

lowpass

Filter: lowpass | Avg Processing Time: 0.0696 sec

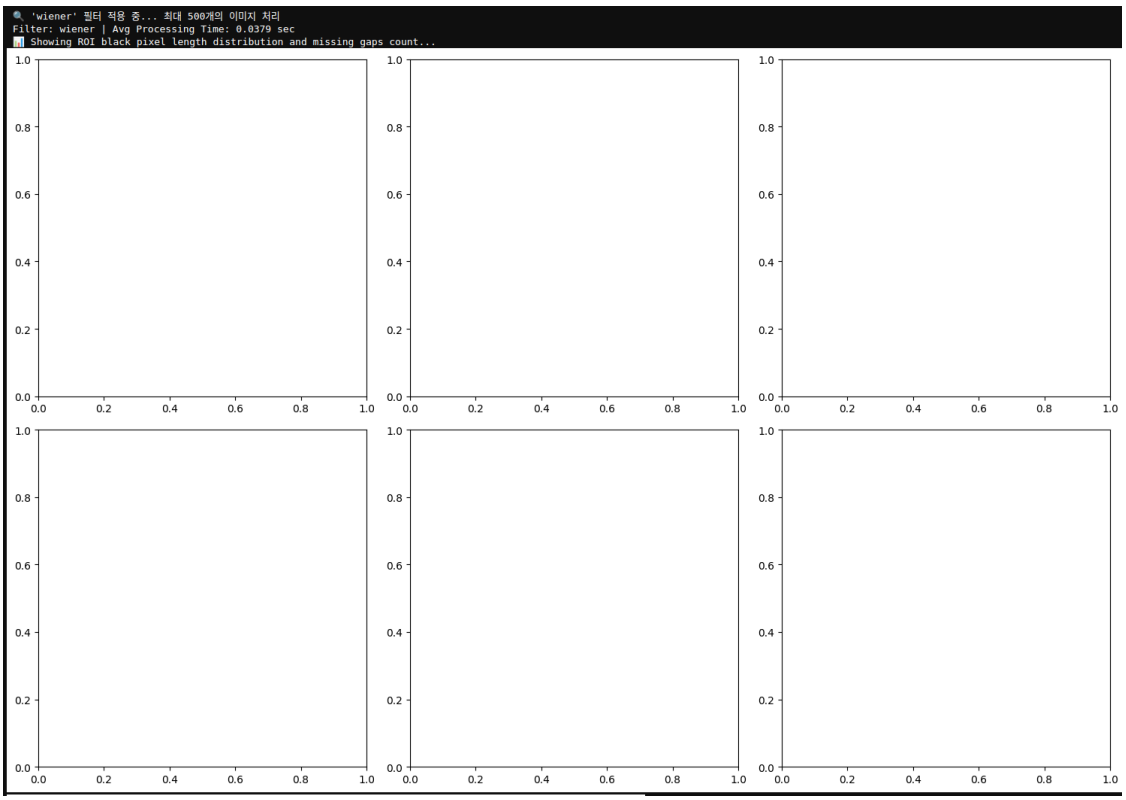


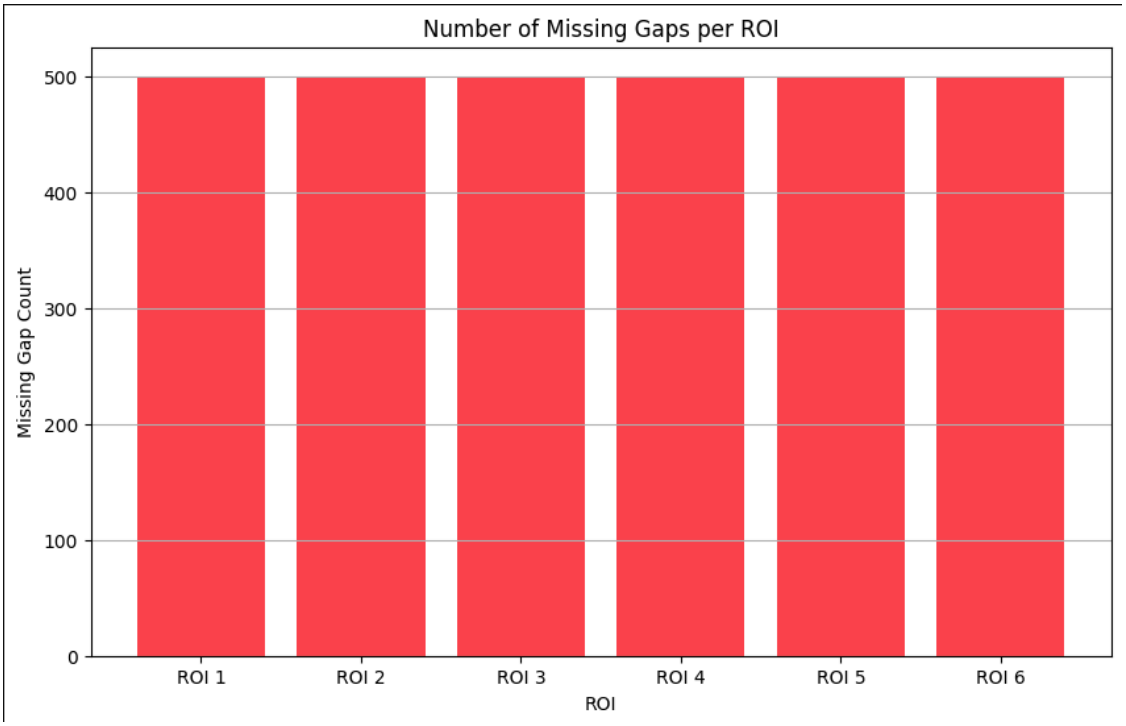


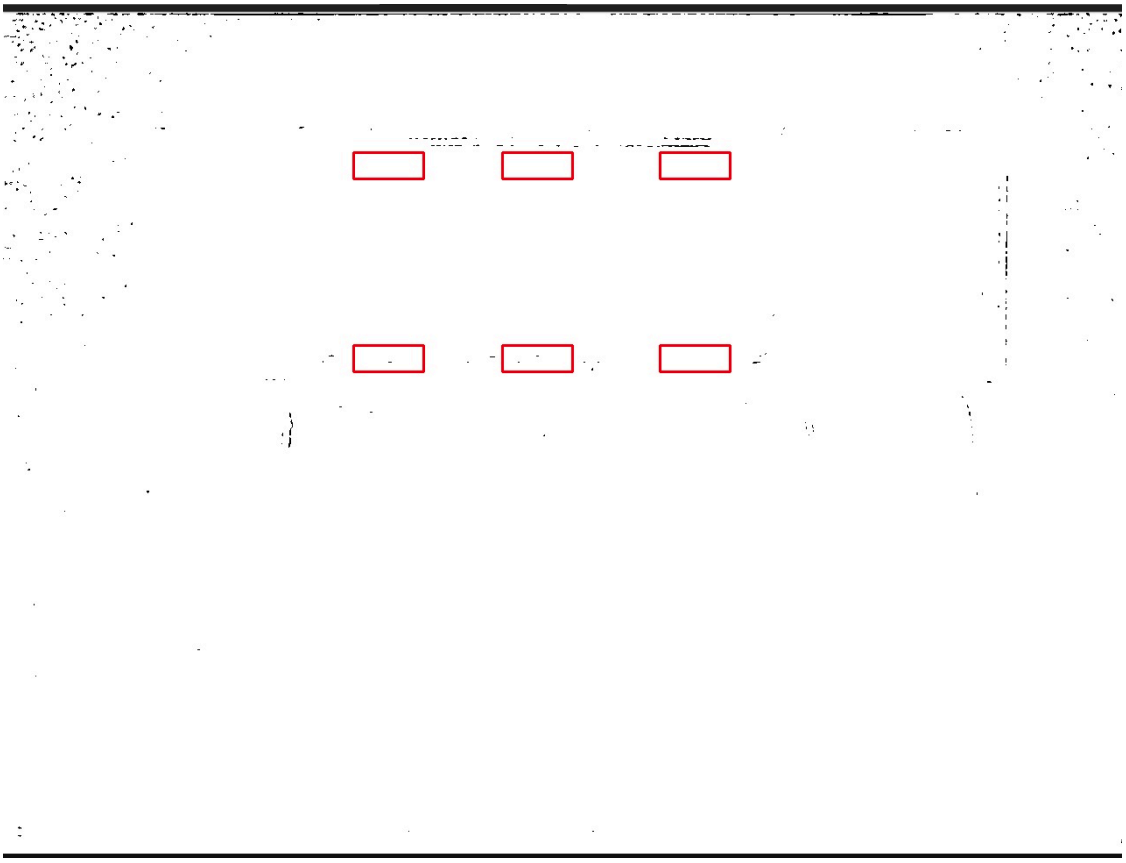
Filter: total_variation_denoising | Avg Processing Time: 0.3650 secFilter: total_variation_denoising | Avg Processing Time: 0.3650 sec

wiener

Filter: wiener | Avg Processing Time: 0.0379 sec

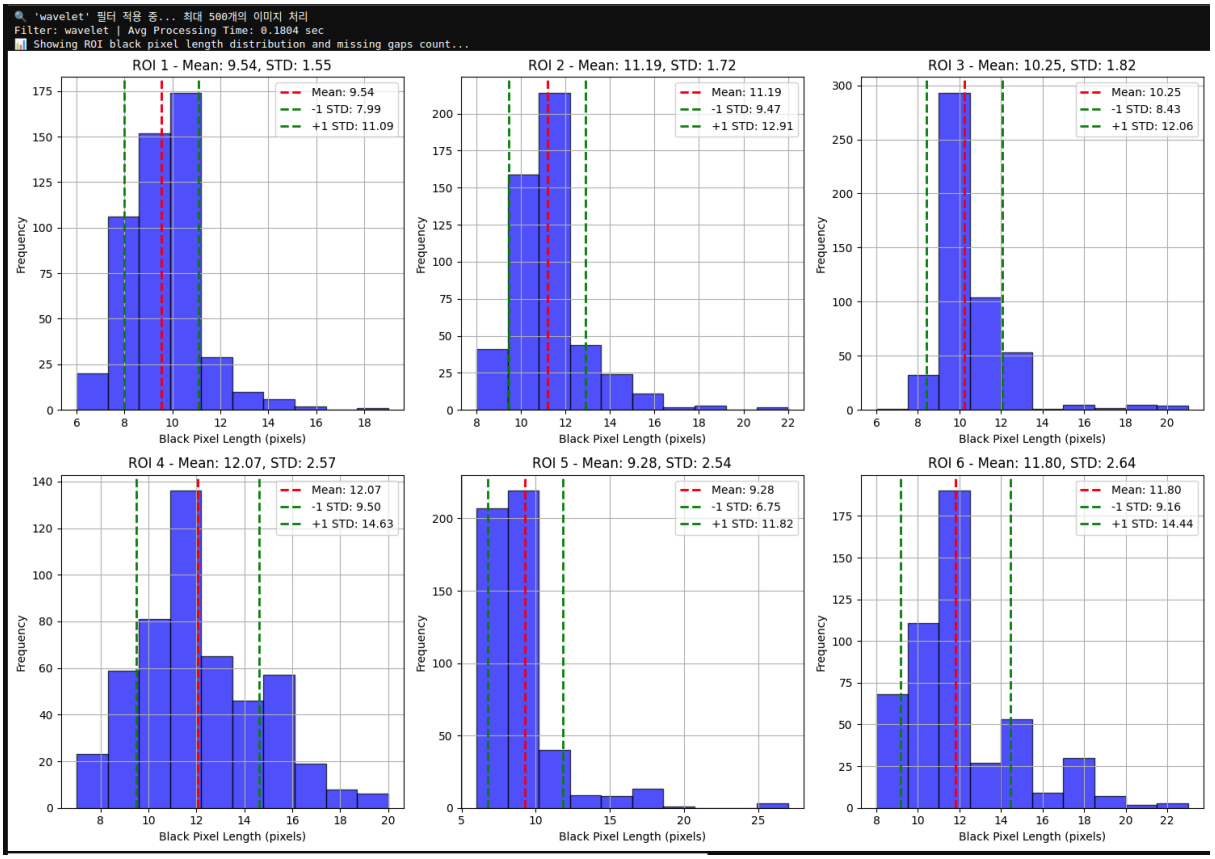


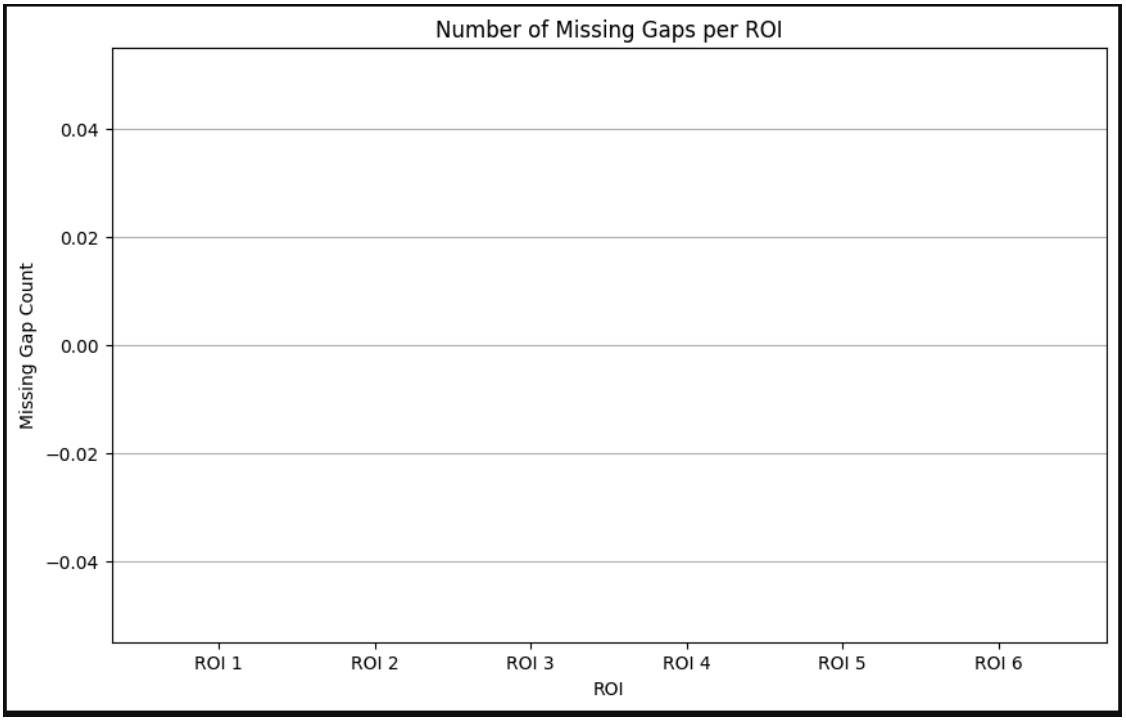




wavelet

Filter: wavelet | Avg Processing Time: 0.1804 sec

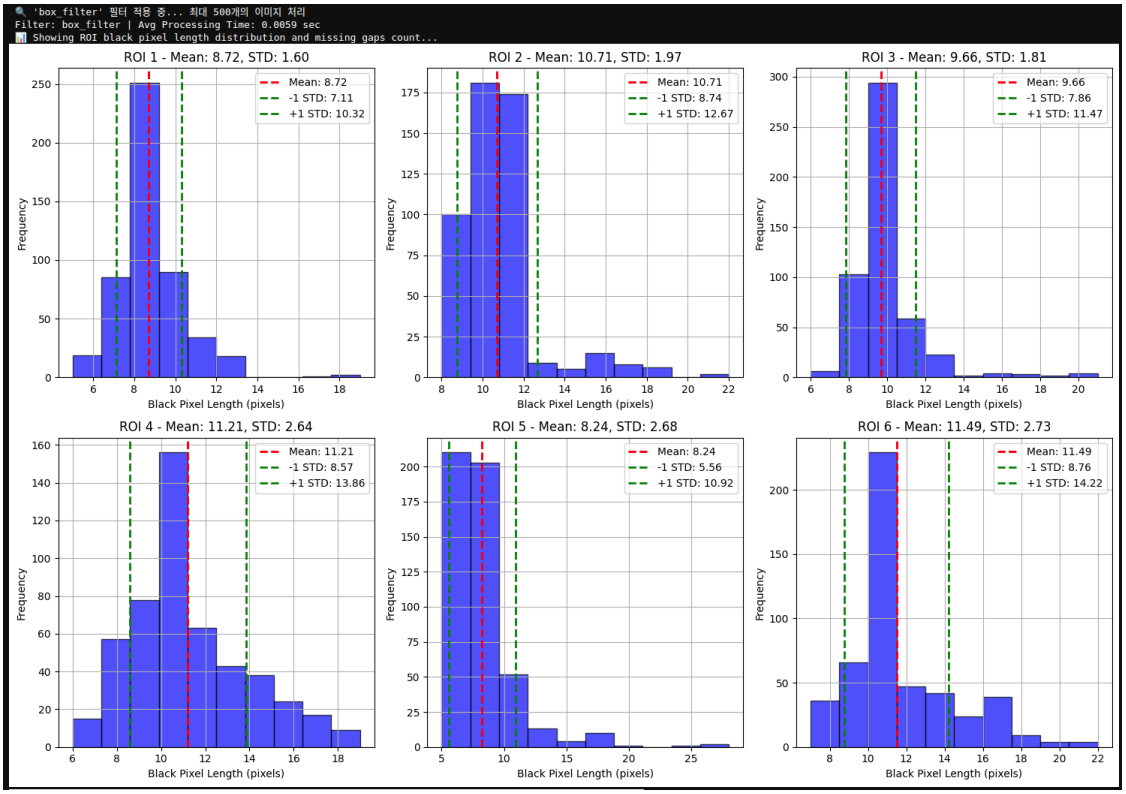

'wavelet' 필터 적용 중... 최대 500개의 이미지 처리
Filter: wavelet | Avg Processing Time: 0.1804 sec
Showing ROI black pixel length distribution and missing gaps count...
ROI 1 - Mean: 9.54, STD: 1.55
Mean: 9.54
-1 STD: 7.99
+1 STD: 11.09
ROI 2 - Mean: 11.19, STD: 1.72
Mean: 11.19
-1 STD: 9.47
+1 STD: 12.91
ROI 3 - Mean: 10.25, STD: 1.82
Mean: 10.25
-1 STD: 8.43
+1 STD: 12.06
ROI 4 - Mean: 12.07, STD: 2.57
Mean: 12.07
-1 STD: 9.50
+1 STD: 14.63
ROI 5 - Mean: 9.28, STD: 2.54
Mean: 9.28
-1 STD: 6.75
+1 STD: 11.82
ROI 6 - Mean: 11.80, STD: 2.64
Mean: 11.80
-1 STD: 9.16
+1 STD: 14.44
Frequency
Black Pixel Length (pixels)

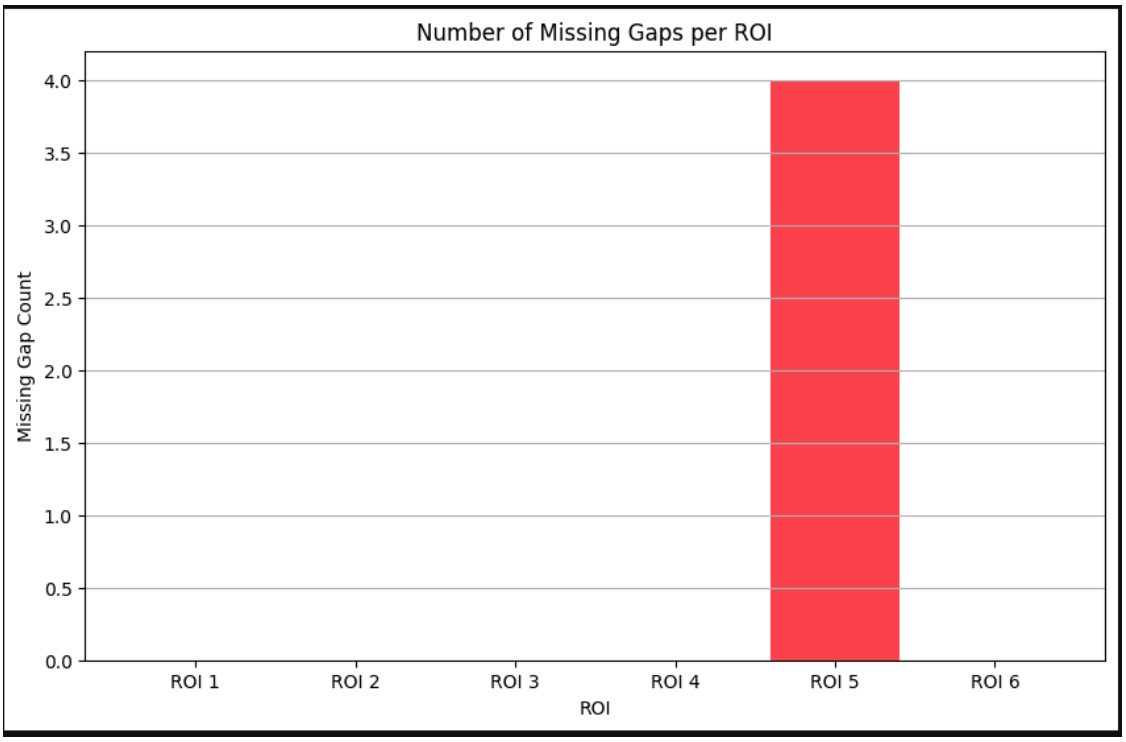

Number of Missing Gaps per ROI
Missing Gap Count
0.04
0.02
0.00
−0.02
−0.04
ROI 1
ROI 2
ROI 3
ROI 4
ROI 5
ROI 6
ROI

box_filter

processed_image = cv2.boxFilter(image, -1, (5, 5))

ilter: box_filter | Avg Processing Time: 0.0059 sec

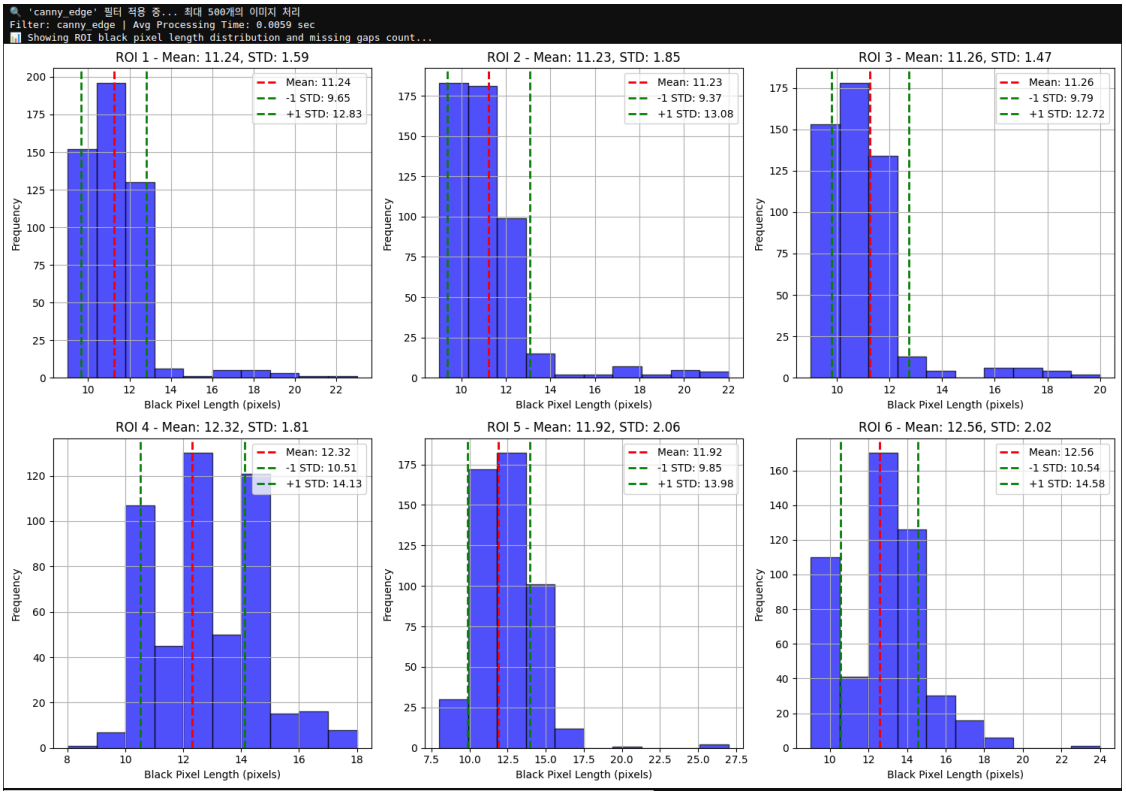


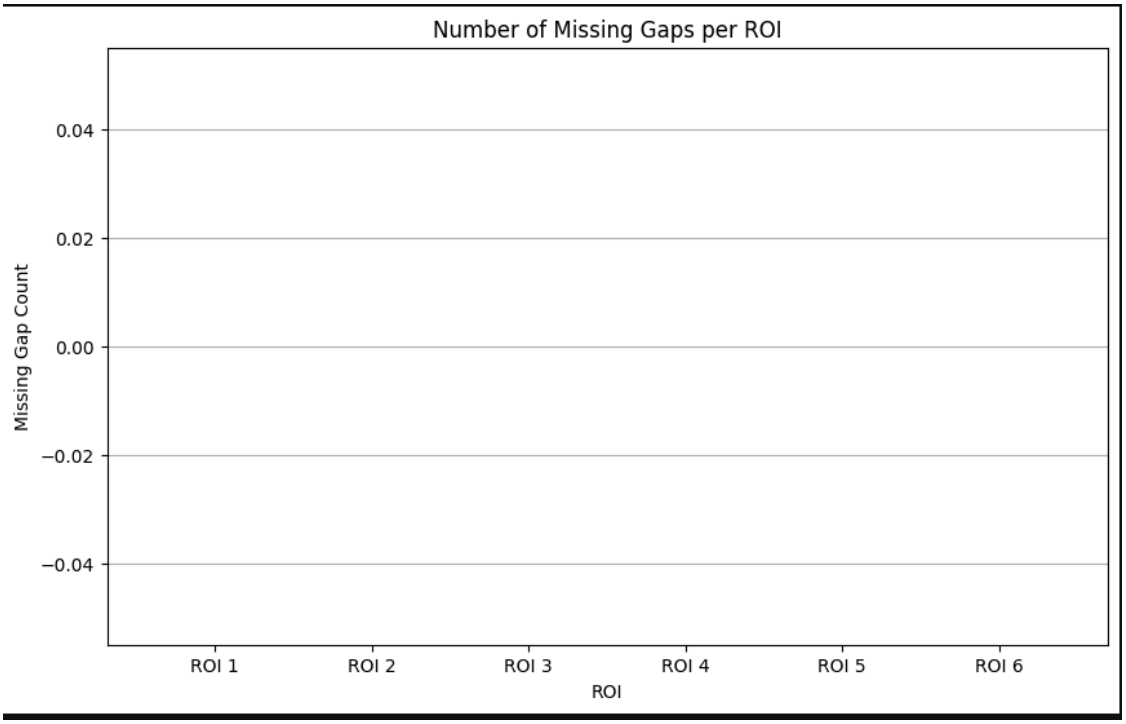


canny_edge

processed_image = cv2.Canny(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY), 100, 200)

Filter: canny_edge | Avg Processing Time: 0.0059 sec



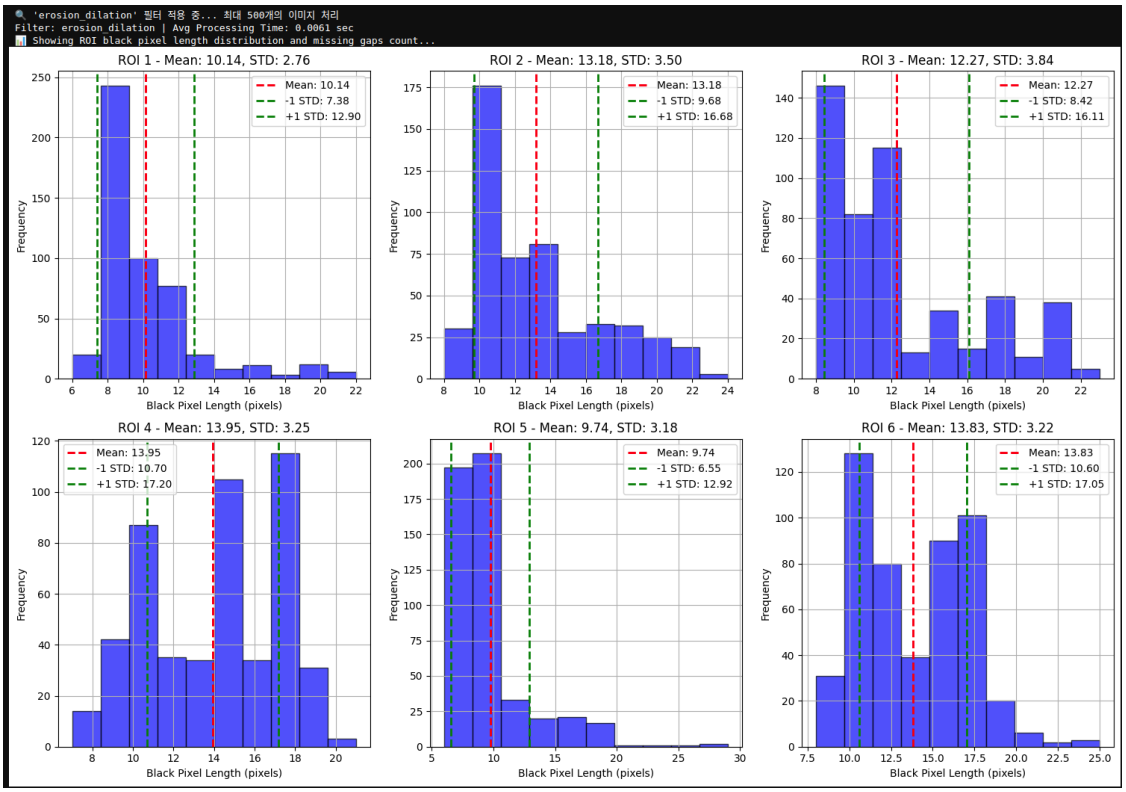


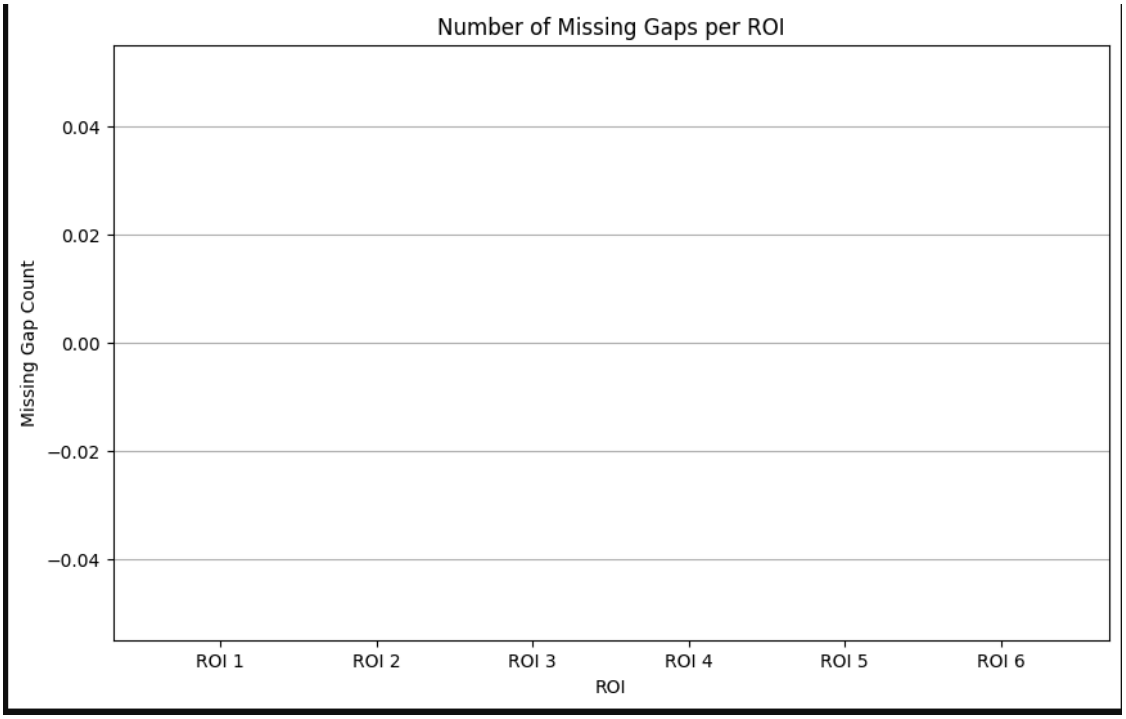
erosion_dilation

kernel = np.ones((5, 5), np.uint8)
eroded = cv2.erode(image, kernel, iterations=1)

processed_image = cv2.dilate(eroded, kernel, iterations=1)

Filter: erosion_dilation | Avg Processing Time: 0.0061 sec

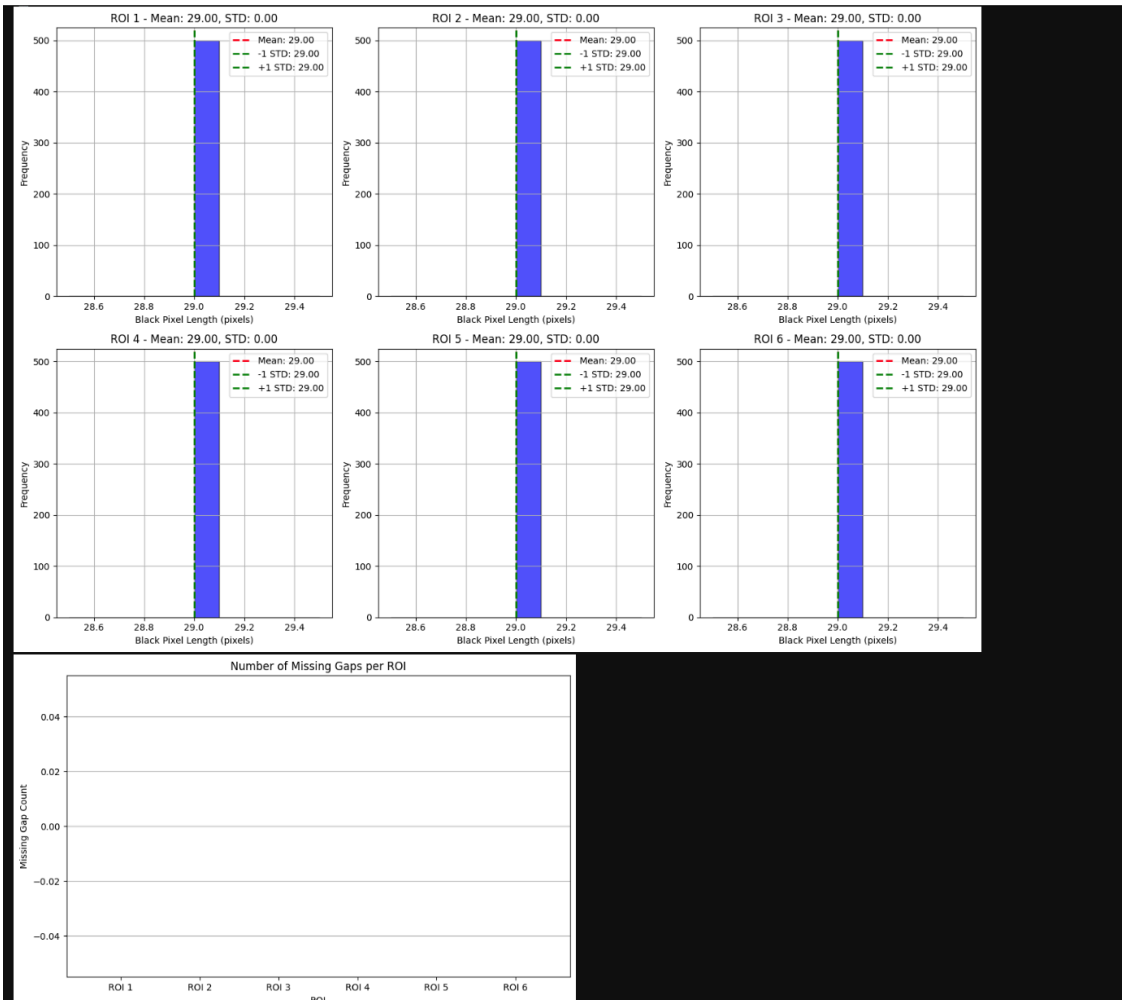




laplacian

processed_image = cv2.Laplacian(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY), cv2.CV_64F)

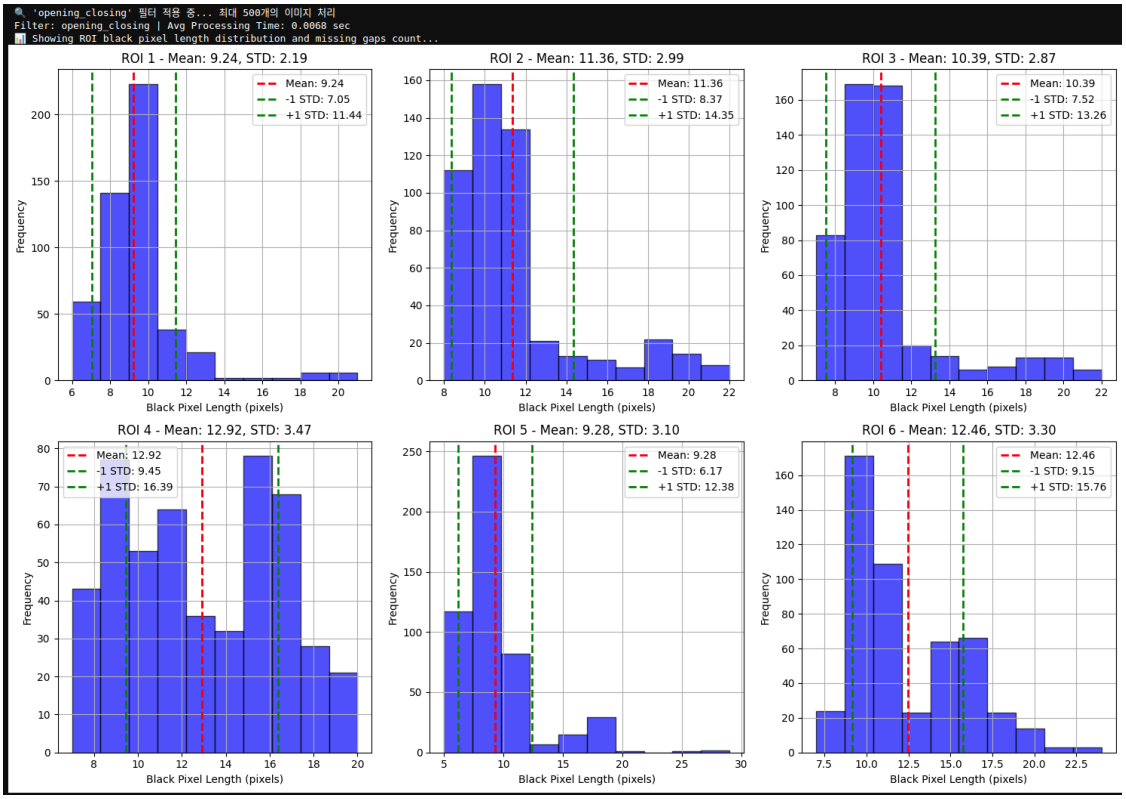
Filter: laplacian | Avg Processing Time: 0.0082 sec

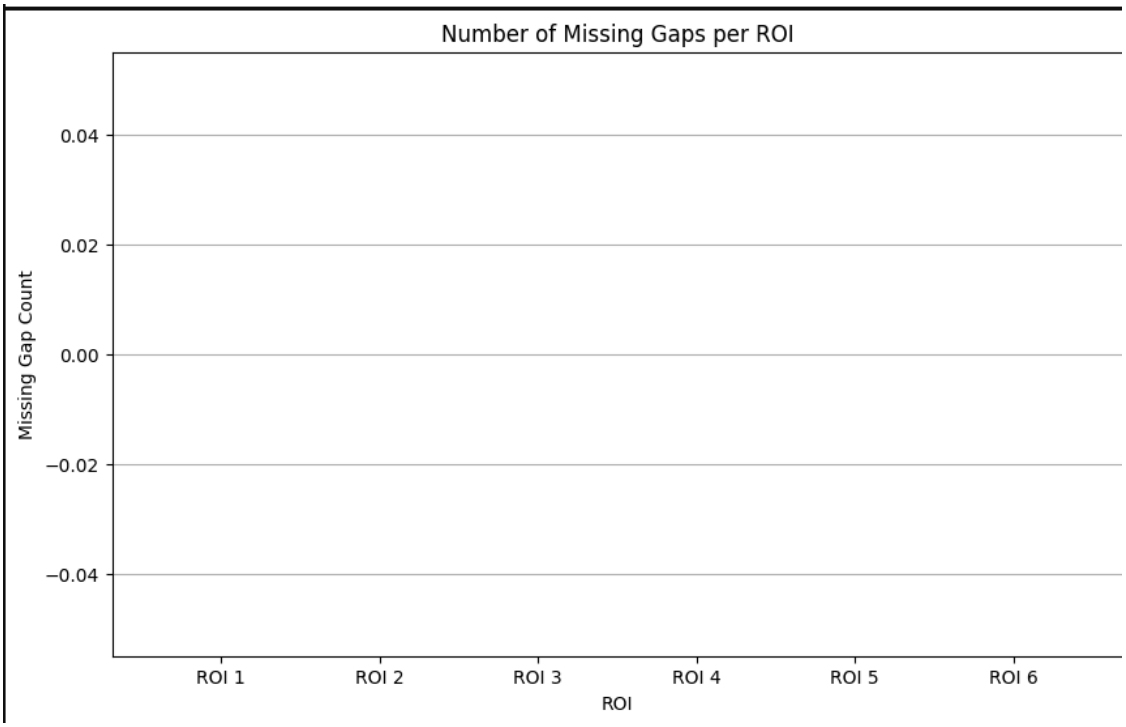


opening_closing

```
kernel = np.ones((5, 5), np.uint8)
opened = cv2.morphologyEx(image, cv2.MORPH_OPEN, kernel)
processed_image = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)
```

Filter: opening_closing | Avg Processing Time: 0.0071 sec

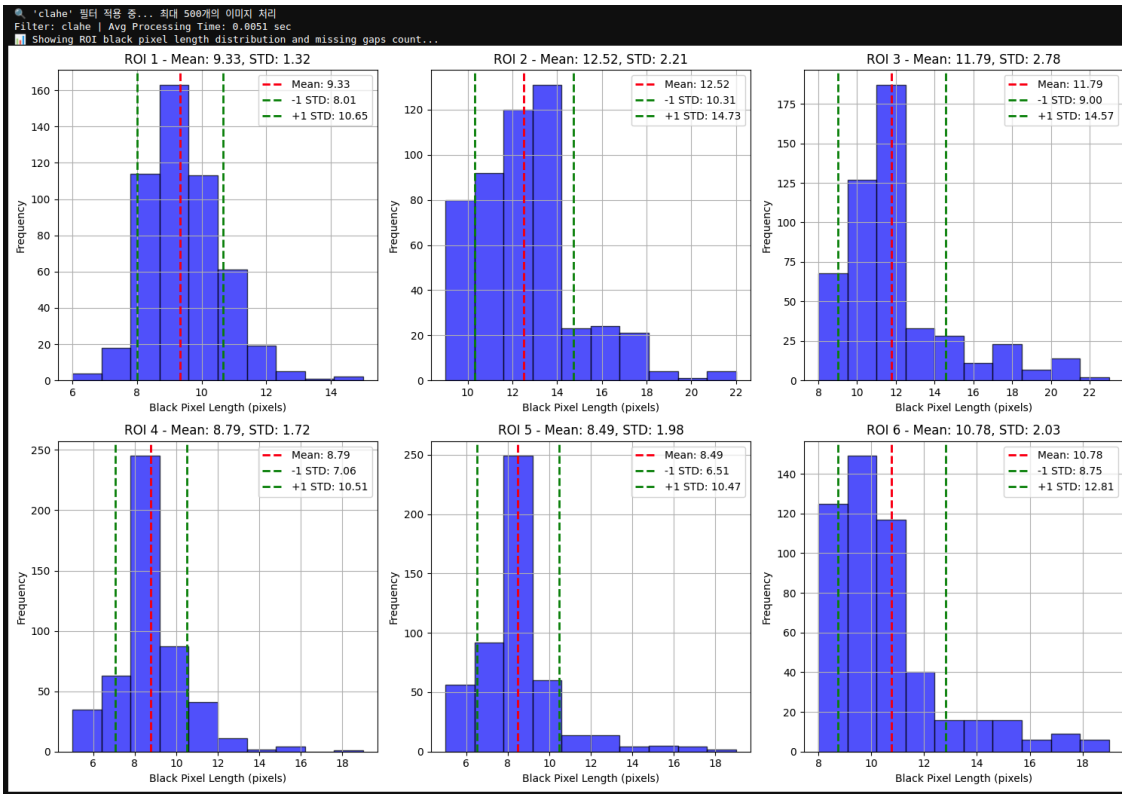


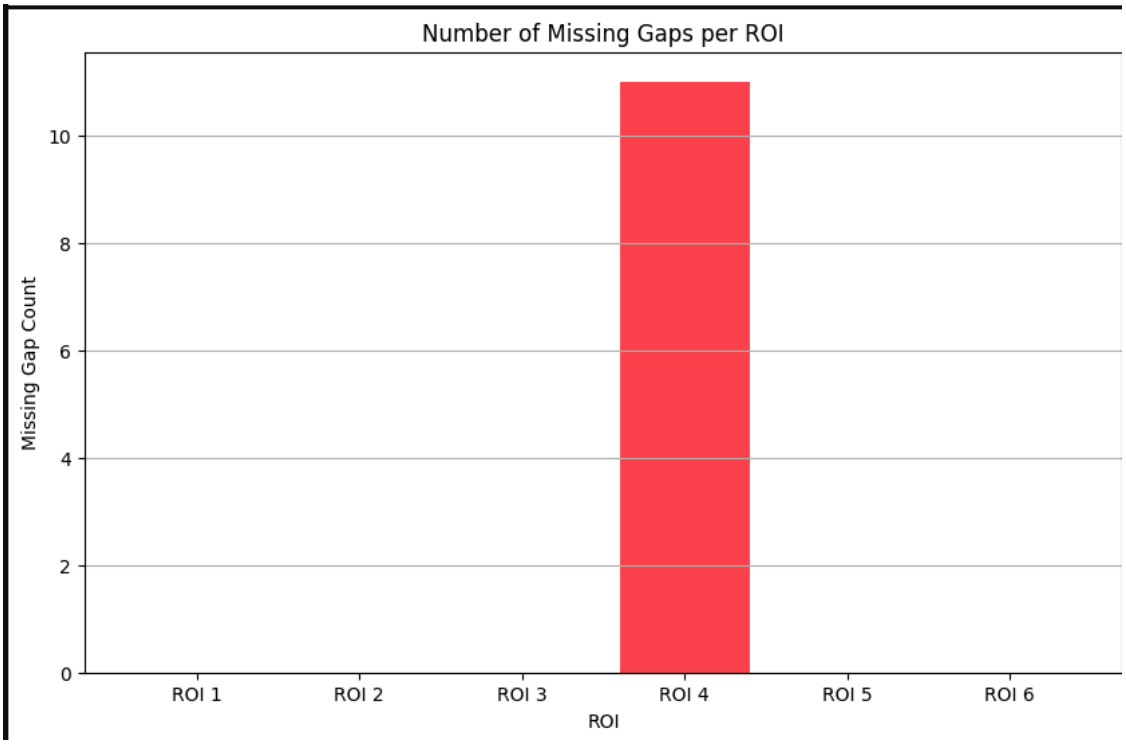


clahe

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
processed_image = clahe.apply(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY))

Filter: clahe | Avg Processing Time: 0.0051 sec

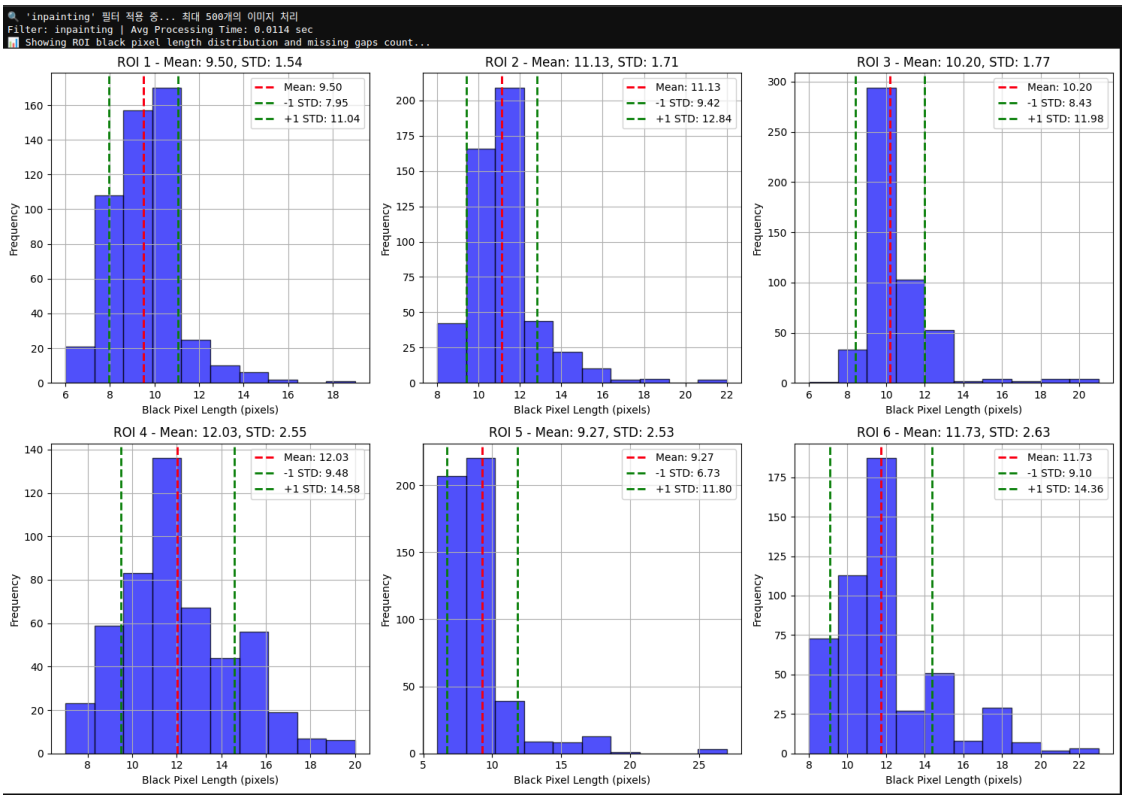


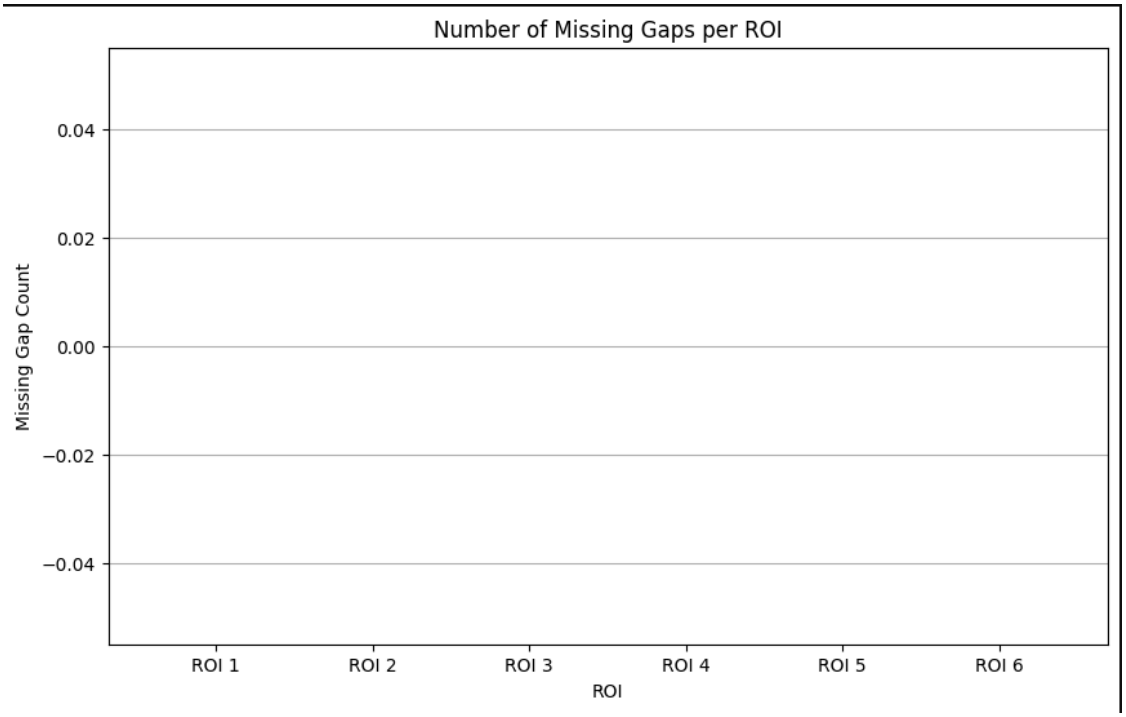


inpainting

```
mask = np.zeros(image.shape[:2], np.uint8)
mask[100:200, 100:200] = 255  # 예제: 임의의 영역 마스크 생성
processed_image = cv2.inpaint(image, mask, 3, cv2.INPAINT_TELEA)
```

```
Filter: inpainting | Avg Processing Time: 0.0114 sec
```

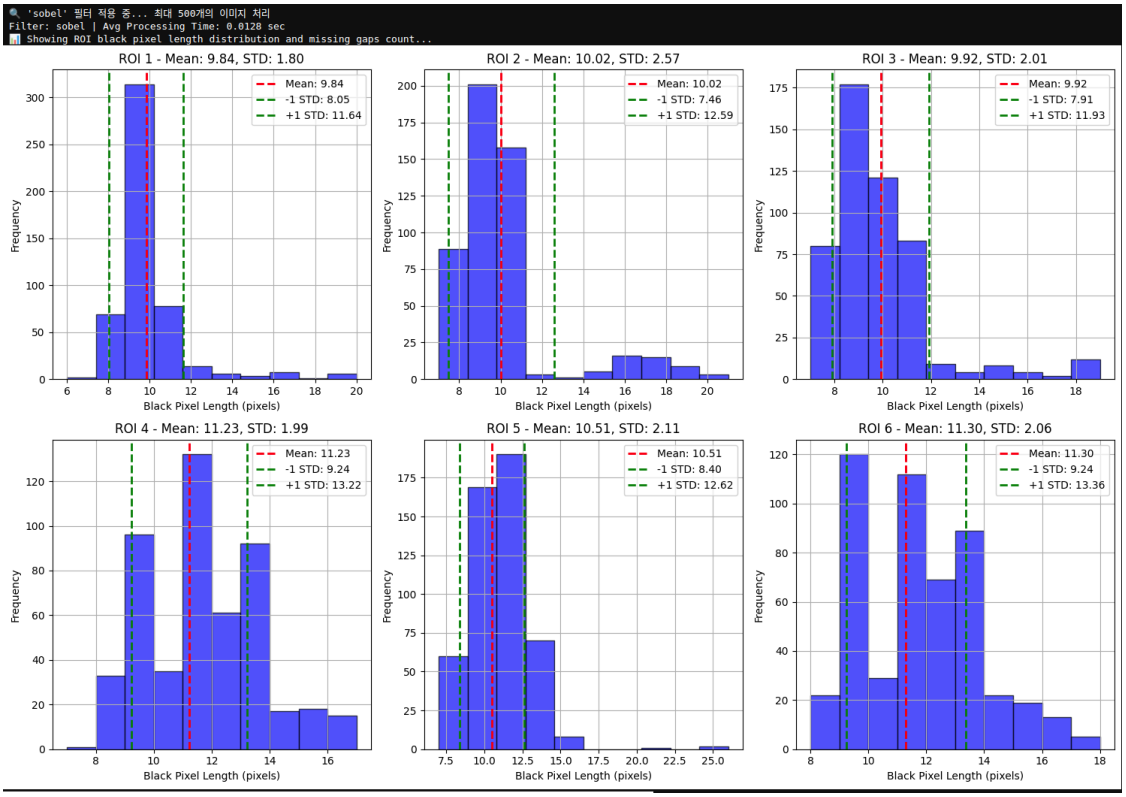


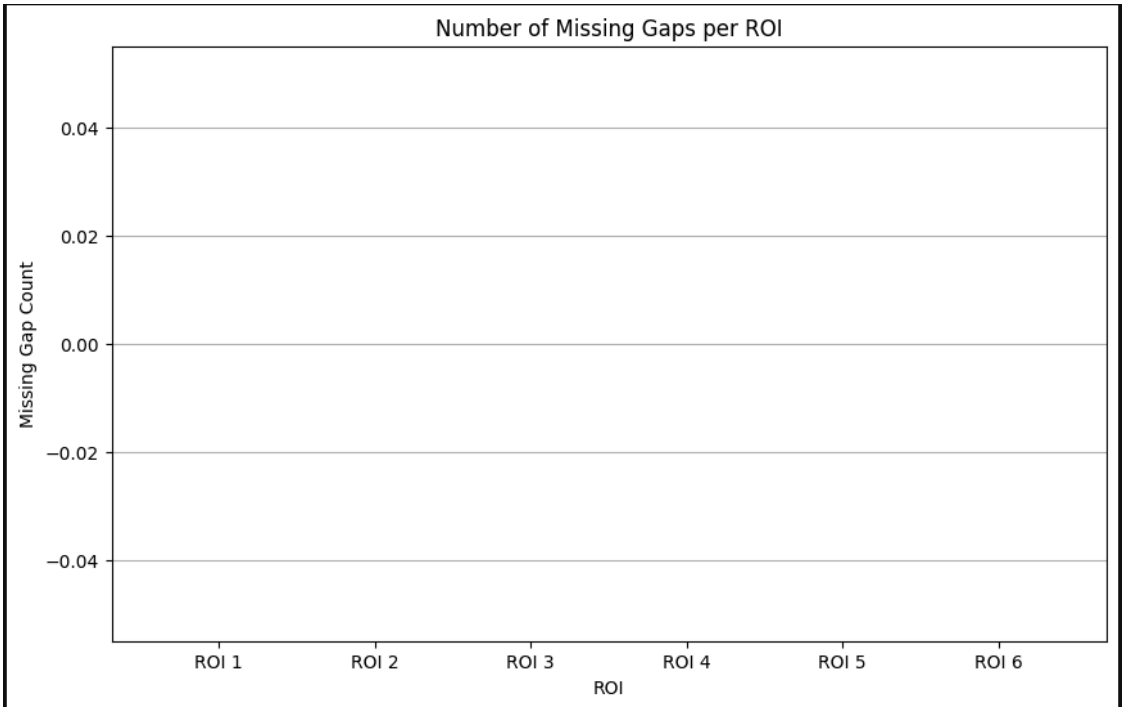


sobel

```
grad_x = cv2.Sobel(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY), cv2.CV_64F, 1, 0, ksize=3)
grad_y = cv2.Sobel(cv2.cvtColor(image, cv2.COLOR_BGR2GRAY), cv2.CV_64F, 0, 1, ksize=3)
processed_image = np.sqrt(grad_x ** 2 + grad_y ** 2)
```

Filter: sobel | Avg Processing Time: 0.0170 sec



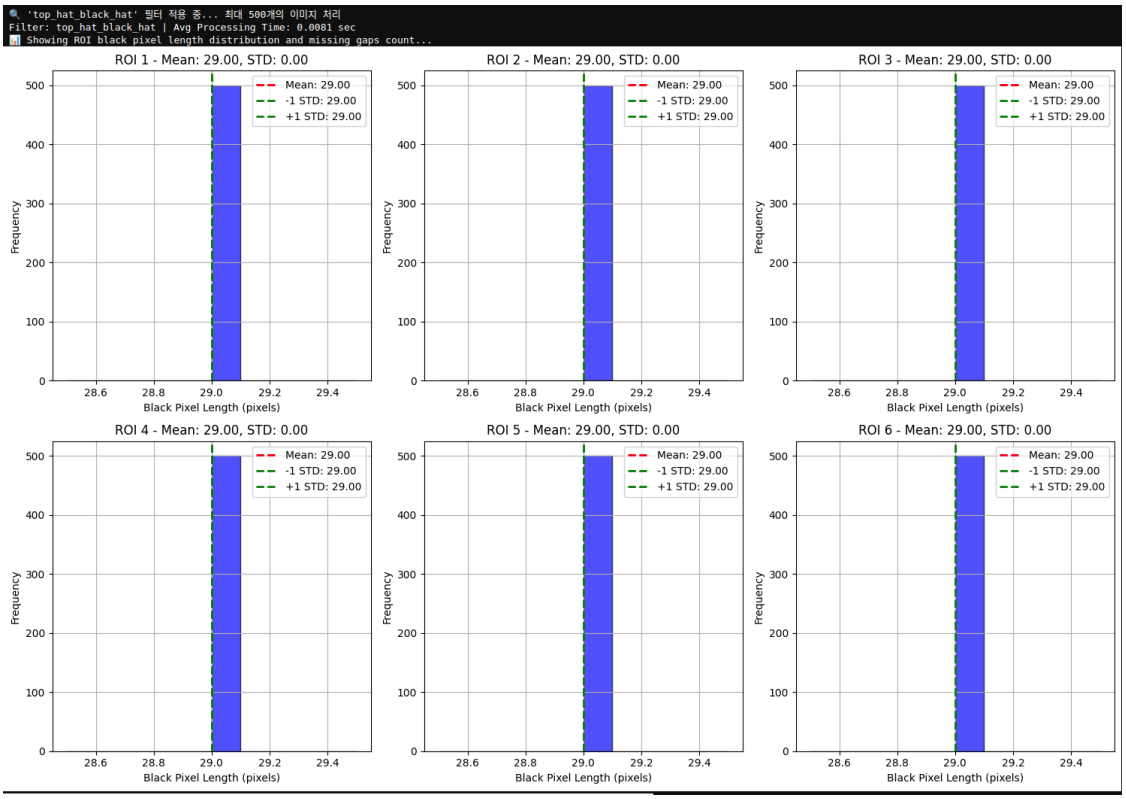


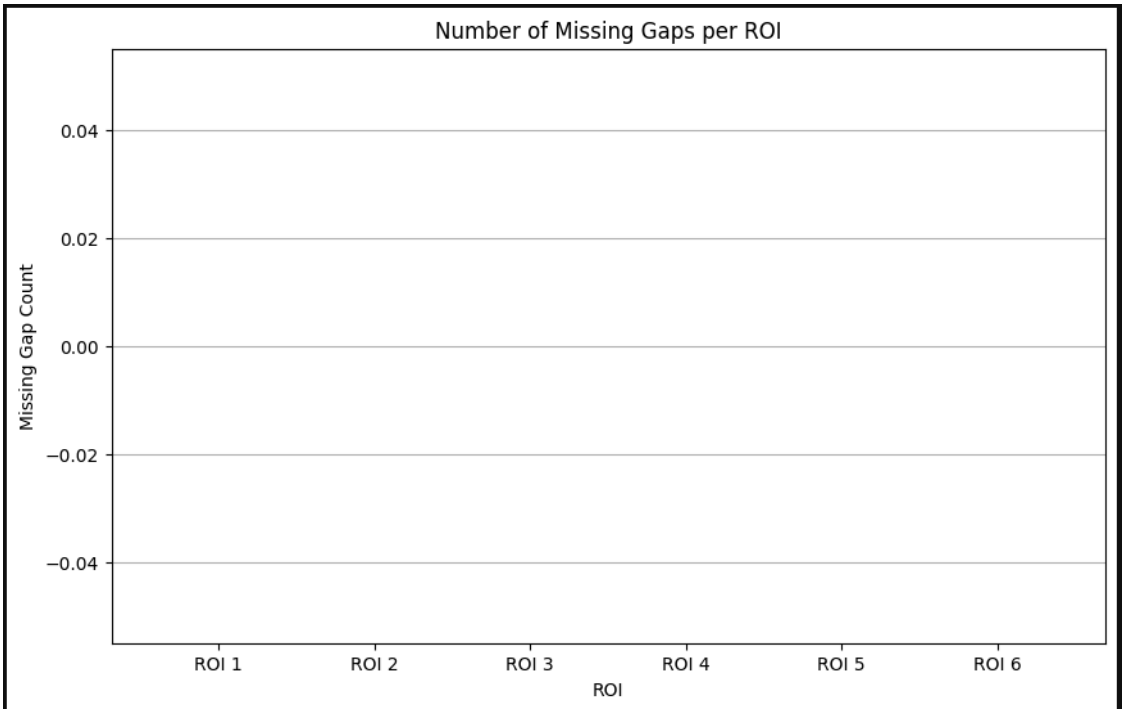
top_hat_black_hat

kernel = np.ones((5, 5), np.uint8)Filter: total_variation_denoising | Avg Processing Time: 0.3650 secFilter: total_variation_denoising | Avg Processing Time: 0.3650 sec

```
top_hat = cv2.morphologyEx(image, cv2.MORPH_TOPHAT, kernel)
black_hat = cv2.morphologyEx(image, cv2.MORPH_BLACKHAT, kernel)
processed_image = cv2.addWeighted(top_hat, 0.5, black_hat, 0.5, 0)
```

Filter: top_hat_black_hat | Avg Processing Time: 0.0077 sec

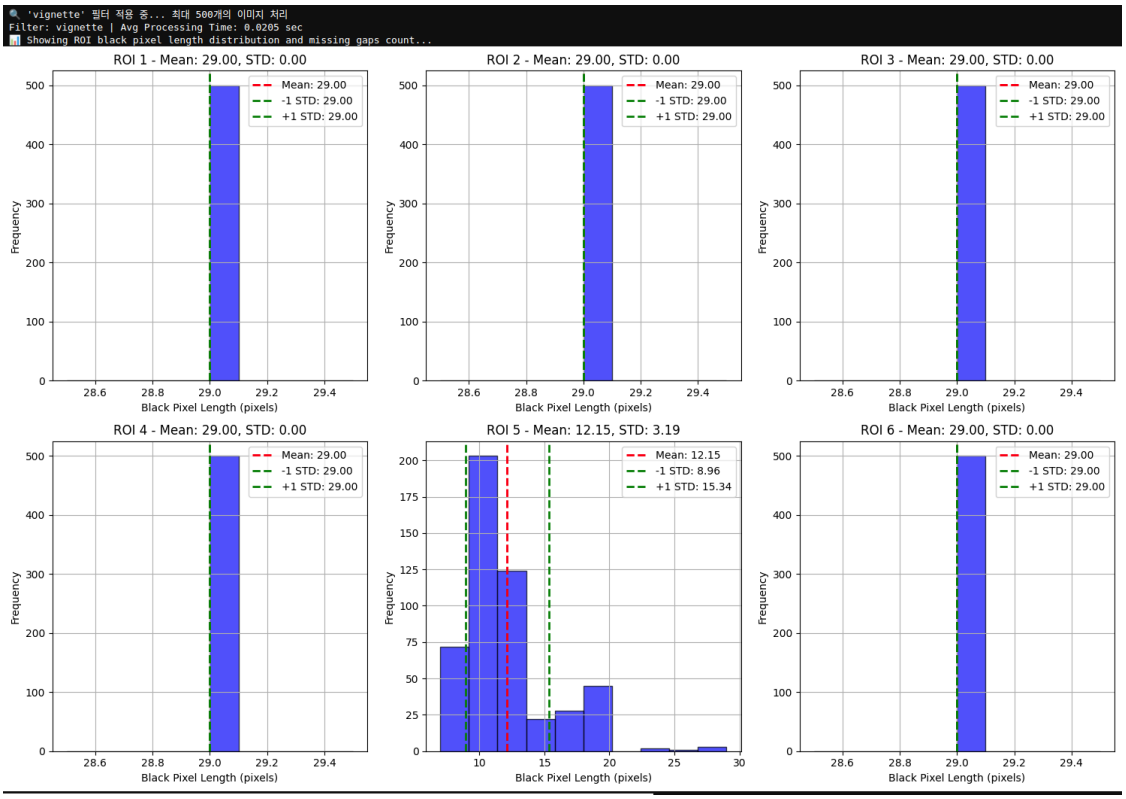


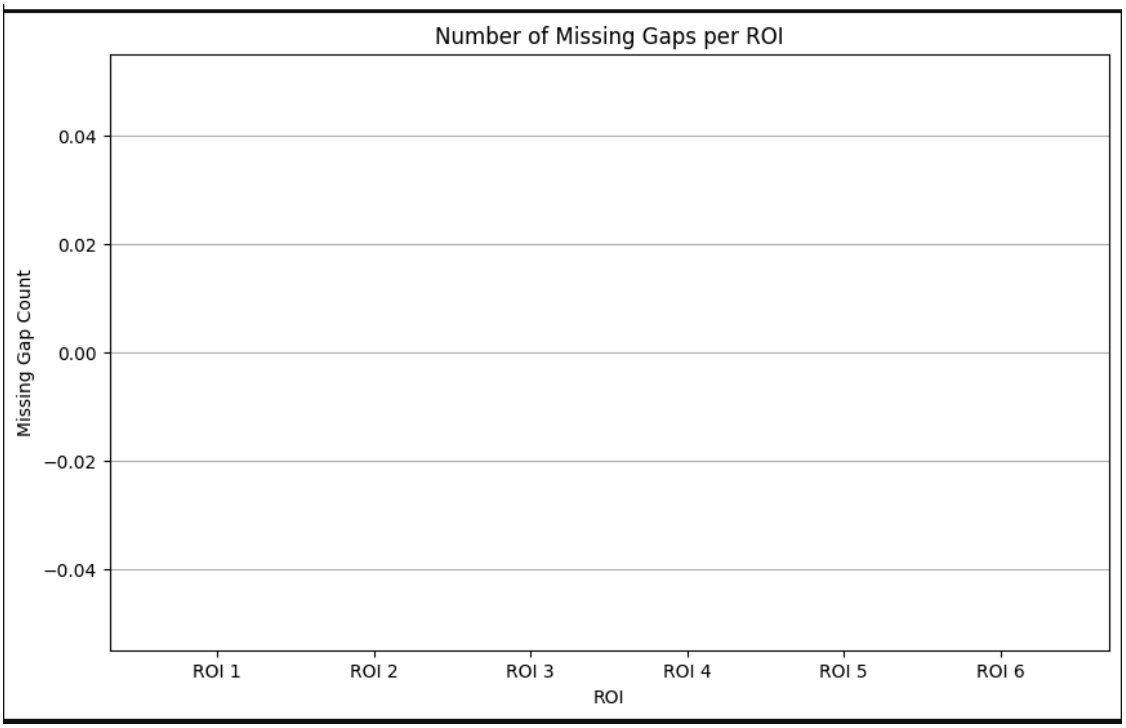


vignette

```
rows, cols = image.shape[:2]
kernel_x = cv2.getGaussianKernel(cols, 200)
kernel_y = cv2.getGaussianKernel(rows, 200)
kernel = kernel_y * kernel_x.T
mask = 255 * kernel / np.linalg.norm(kernel)
processed_image = (image * mask[:, :, np.newaxis]).astype(np.uint8)
```

Filter: vignette | Avg Processing Time: 0.0197 sec

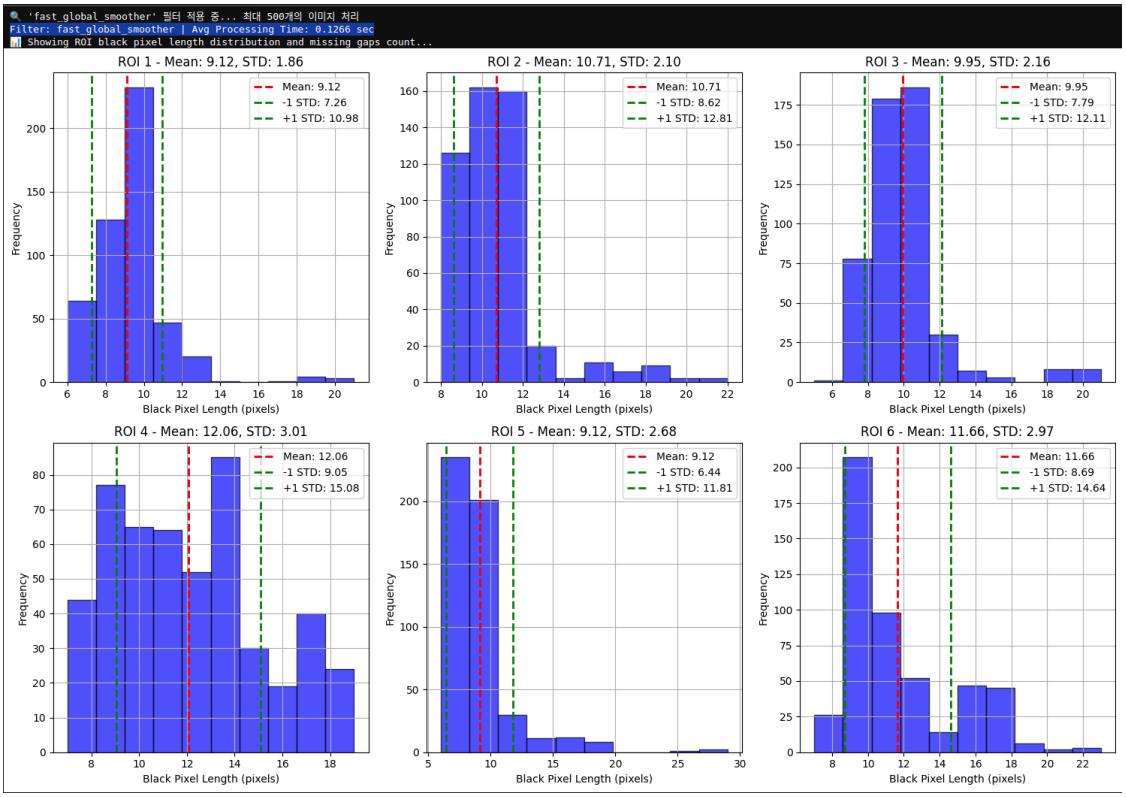


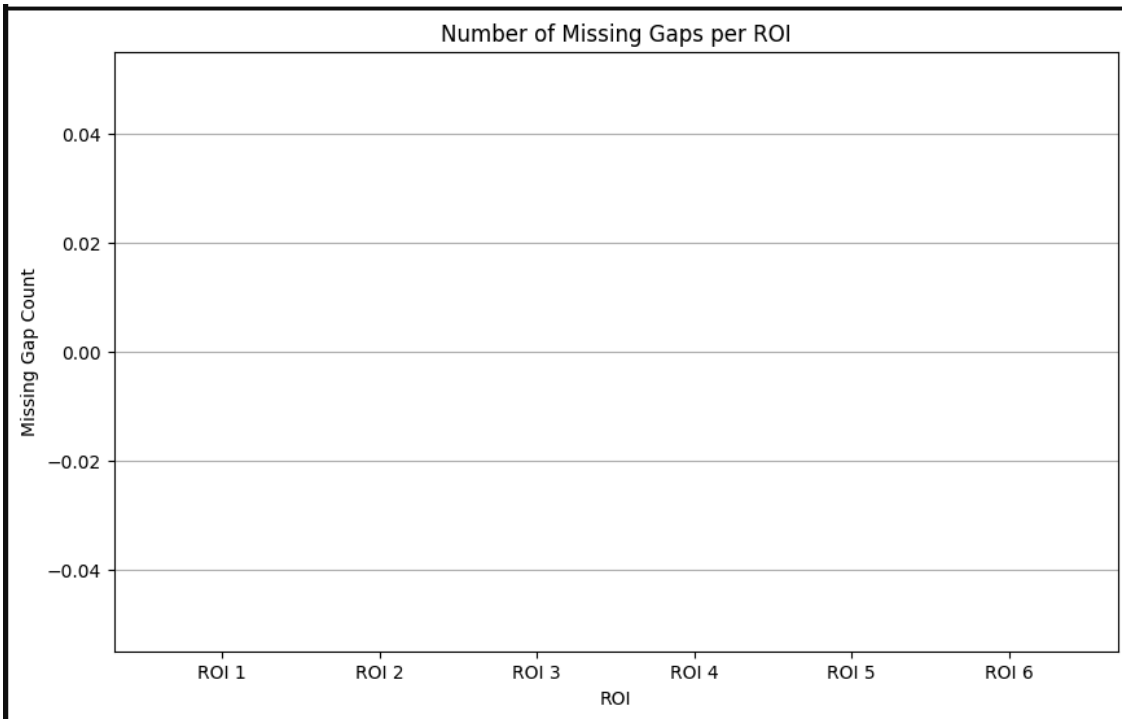


fast_global_smoother

processed_image = cv2.edgePreservingFilter(image, flags=1, sigma_s=60, sigma_r=0.4)

Filter: fast_global_smoother | Avg Processing Time: 0.1266 sec

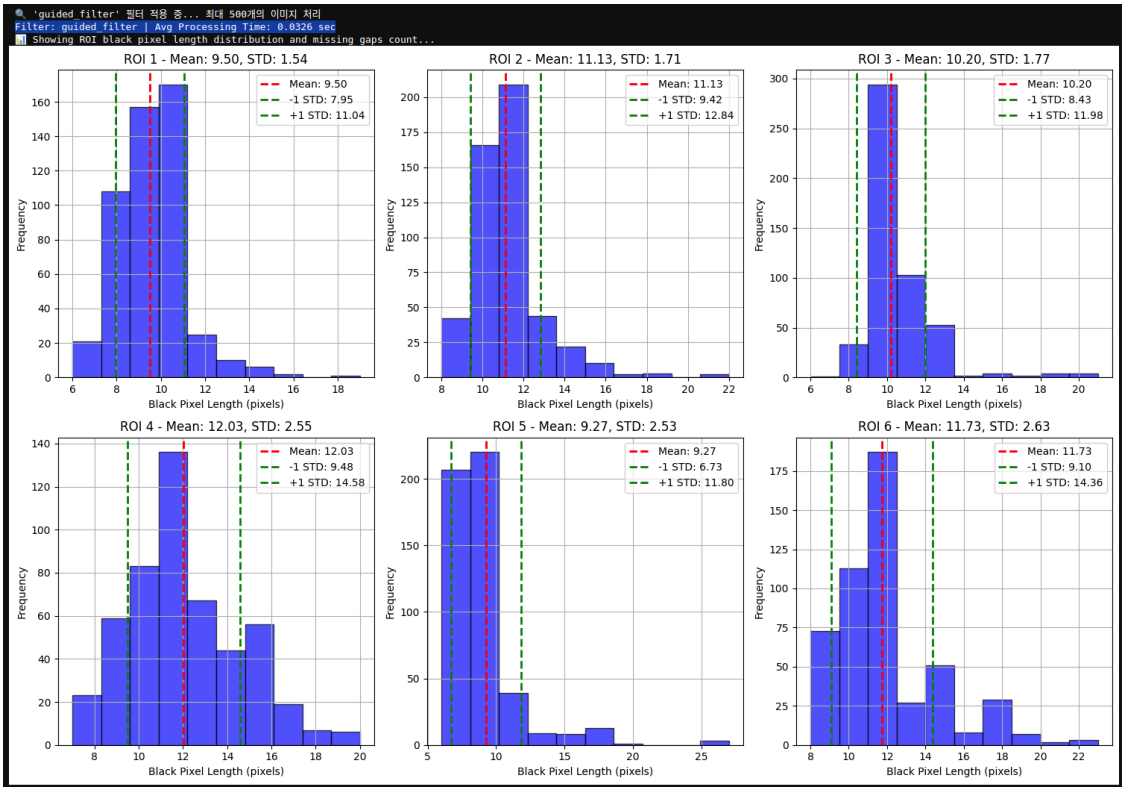


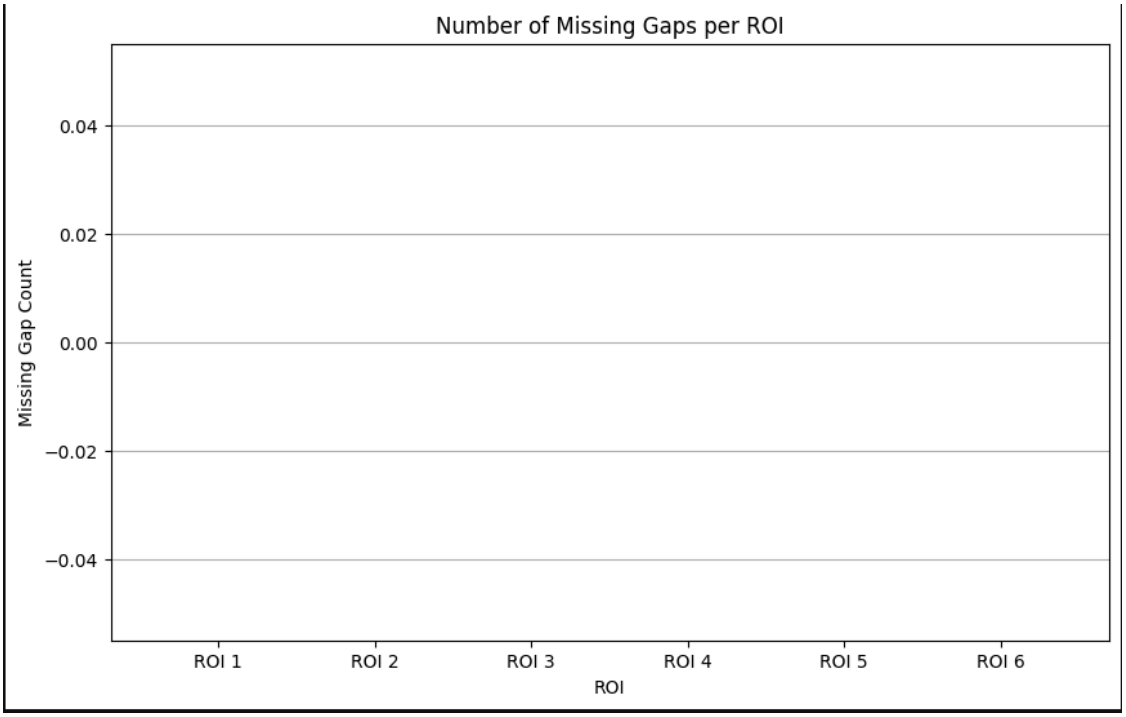


guided_filter

processed_image = cv2.ximgproc.guidedFilter(image, image, 8, 0.01)

Filter: guided_filter | Avg Processing Time: 0.0326 sec



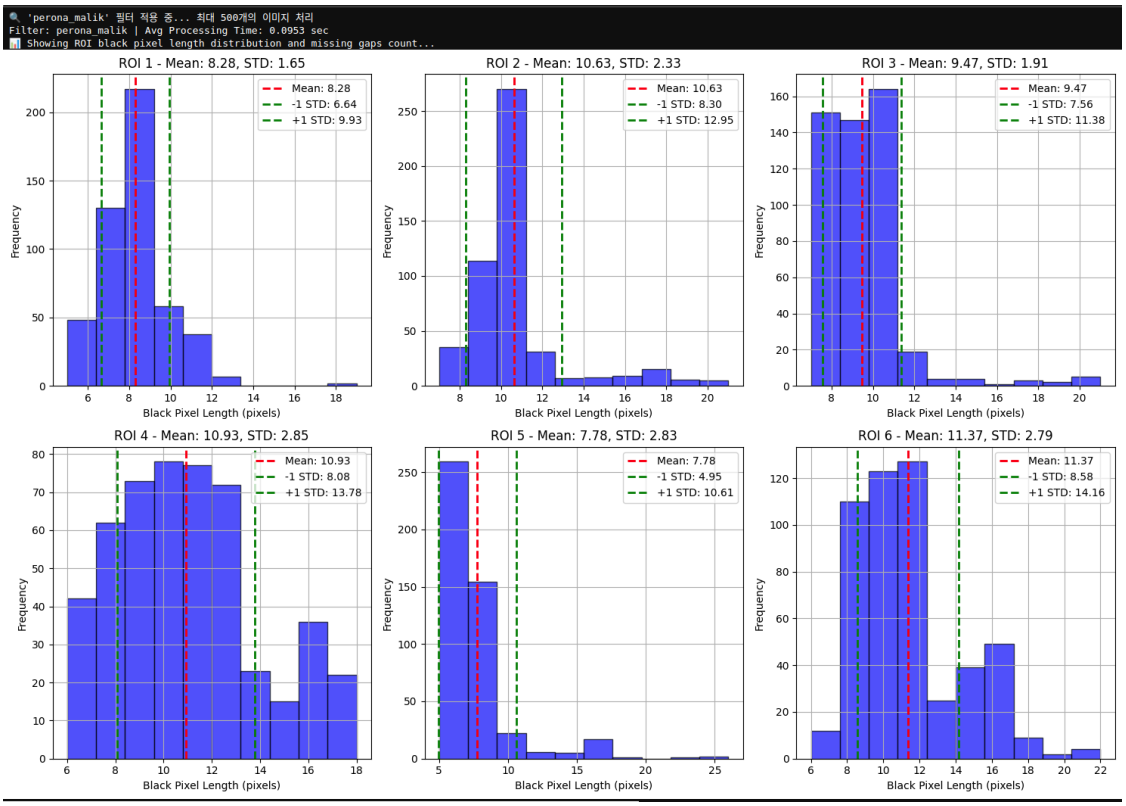


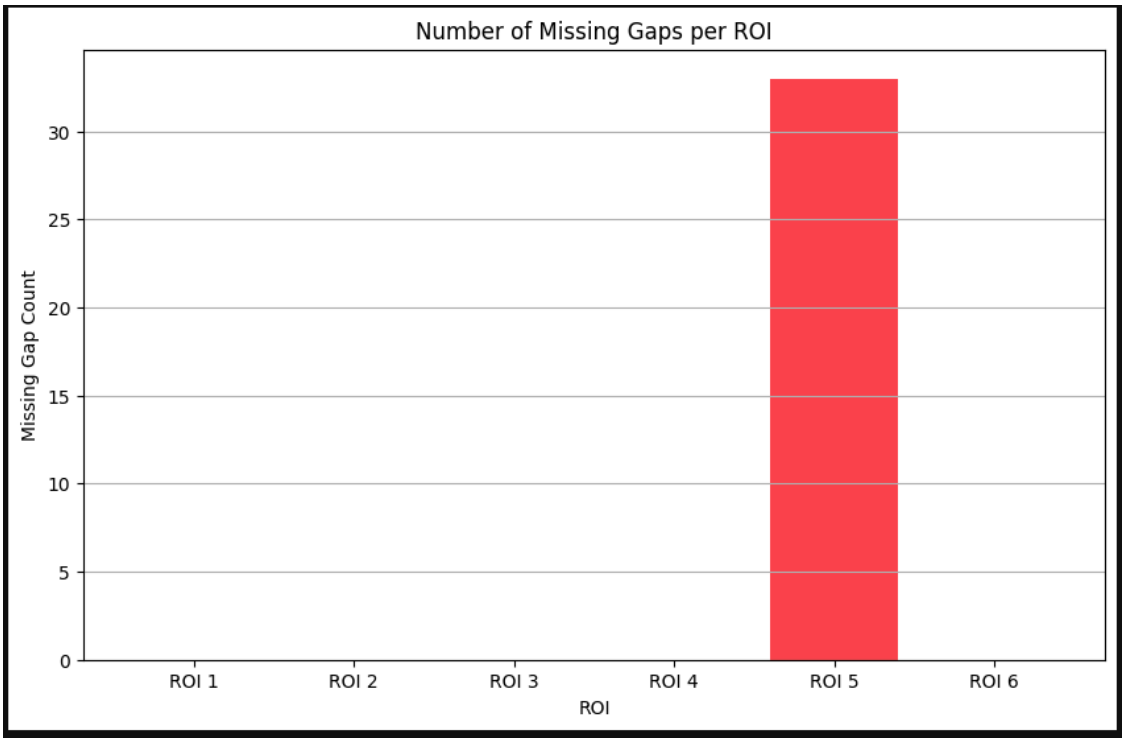
perona_malik

for _ in range(10):
laplacian = cv2.Laplacian(image, cv2.CV_64F)
image = image + 0.25 * laplacian

processed_image = np.clip(image, 0, 255).astype(np.uint8)

Filter: perona_malik | Avg Processing Time: 0.0953 sec

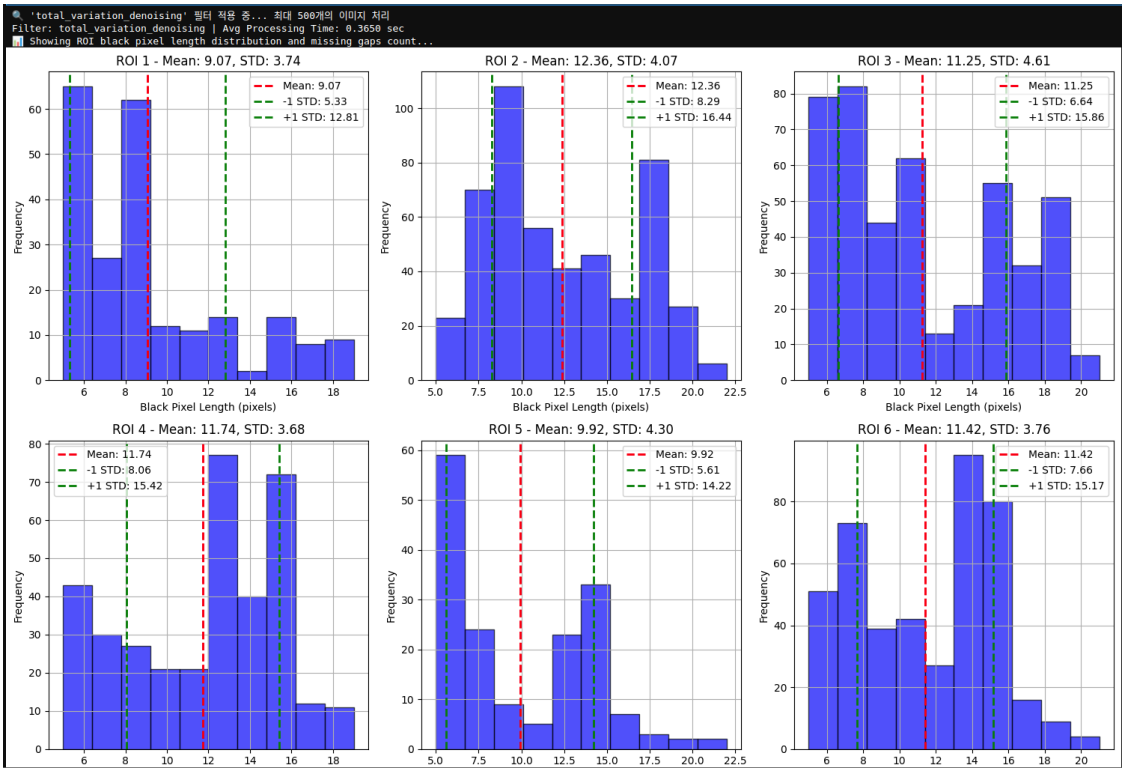


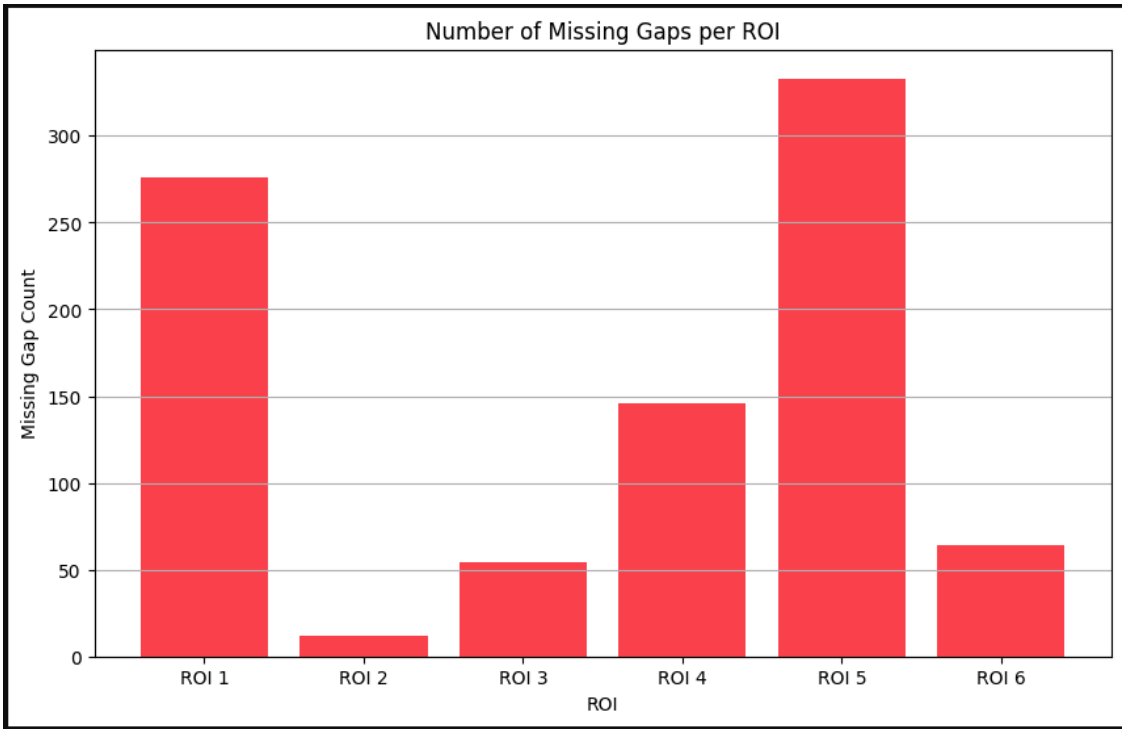


total_variation_denoising

Filter: total_variation_denoising | Avg Processing Time: 0.3650 sec

processed_image = (restoration.denoise_tv_bregman(image, weight=0.1) * 255).astype(np.uint8)





2025-03-05

단일 필터 결론 :

✅ 노이즈 제거 필터 TOP 10

순위	필터	Mean Gap	Std Gap
1	blur	9.20	2.27
2	median	9.87	2.29
3	gaussian	9.92	2.31
4	guided_filter	9.71	2.31
5	bilateral	10.11	2.38
6	box_filter	9.62	2.42
7	anisotropic	10.02	2.44
8	fast_global_smoother	9.81	2.46
9	wiener	9.79	2.47
10	wavelet	9.74	2.47

🔍 필터 성능 비교 중...
Filter: bilateral + sharpen | Avg Processing Time: 0.0164 sec
Filter: bilateral + sobel_filter | Avg Processing Time: 0.0223 sec
Filter: bilateral + canny_edge | Avg Processing Time: 0.0172 sec
Filter: bilateral + laplacian_filter | Avg Processing Time: 0.0185 sec
Filter: bilateral + opening_closing | Avg Processing Time: 0.0167 sec
Filter: bilateral + clahe | Avg Processing Time: 0.0157 sec
Filter: bilateral + top_hat_black_hat | Avg Processing Time: 0.0176 sec
Filter: wavelet + sharpen | Avg Processing Time: 0.0323 sec
Filter: wavelet + sobel_filter | Avg Processing Time: 0.0390 sec
Filter: wavelet + canny_edge | Avg Processing Time: 0.0324 sec
Filter: wavelet + laplacian_filter | Avg Processing Time: 0.0343 sec
Filter: wavelet + opening_closing | Avg Processing Time: 0.0325 sec
Filter: wavelet + clahe | Avg Processing Time: 0.0307 sec
Filter: wavelet + top_hat_black_hat | Avg Processing Time: 0.0348 sec
Filter: fast_global_smoother + sharpen | Avg Processing Time: 0.1250 sec
Filter: fast_global_smoother + sobel_filter | Avg Processing Time: 0.1296 sec
Filter: fast_global_smoother + canny_edge | Avg Processing Time: 0.1256 sec
Filter: fast_global_smoother + laplacian_filter | Avg Processing Time: 0.1279 sec
Filter: fast_global_smoother + opening_closing | Avg Processing Time: 0.1266 sec
Filter: fast_global_smoother + clahe | Avg Processing Time: 0.1245 sec
Filter: fast_global_smoother + top_hat_black_hat | Avg Processing Time: 0.1258 sec
Filter: guided_filter + sharpen | Avg Processing Time: 0.0327 sec
Filter: guided_filter + sobel_filter | Avg Processing Time: 0.0404 sec
Filter: guided_filter + canny_edge | Avg Processing Time: 0.0324 sec
Filter: guided_filter + laplacian_filter | Avg Processing Time: 0.0348 sec
Filter: guided_filter + opening_closing | Avg Processing Time: 0.0344 sec
Filter: guided_filter + clahe | Avg Processing Time: 0.0335 sec
Filter: guided_filter + top_hat_black_hat | Avg Processing Time: 0.0357 sec
Filter: blur + sharpen | Avg Processing Time: 0.0069 sec
Filter: blur + sobel_filter | Avg Processing Time: 0.0129 sec
Filter: blur + canny_edge | Avg Processing Time: 0.0062 sec
Filter: blur + laplacian_filter | Avg Processing Time: 0.0087 sec
Filter: blur + opening_closing | Avg Processing Time: 0.0069 sec
Filter: blur + clahe | Avg Processing Time: 0.0055 sec
Filter: blur + top_hat_black_hat | Avg Processing Time: 0.0099 sec
Filter: gaussian + sharpen | Avg Processing Time: 0.0064 sec
Filter: gaussian + sobel_filter | Avg Processing Time: 0.0124 sec

Filter: gaussian + canny_edge | Avg Processing Time: 0.0060 sec
Filter: gaussian + laplacian_filter | Avg Processing Time: 0.0083 sec
Filter: gaussian + opening_closing | Avg Processing Time: 0.0067 sec
Filter: gaussian + clahe | Avg Processing Time: 0.0050 sec
Filter: gaussian + top_hat_black_hat | Avg Processing Time: 0.0080 sec
Filter: median + sharpen | Avg Processing Time: 0.0100 sec
Filter: median + sobel_filter | Avg Processing Time: 0.0160 sec
Filter: median + canny_edge | Avg Processing Time: 0.0113 sec
Filter: median + laplacian_filter | Avg Processing Time: 0.0119 sec
Filter: median + opening_closing | Avg Processing Time: 0.0104 sec
Filter: median + clahe | Avg Processing Time: 0.0091 sec
Filter: median + top_hat_black_hat | Avg Processing Time: 0.0114 sec
Filter: box_filter + sharpen | Avg Processing Time: 0.0067 sec
Filter: box_filter + sobel_filter | Avg Processing Time: 0.0126 sec
Filter: box_filter + canny_edge | Avg Processing Time: 0.0065 sec
Filter: box_filter + laplacian_filter | Avg Processing Time: 0.0086 sec
Filter: box_filter + opening_closing | Avg Processing Time: 0.0068 sec
Filter: box_filter + clahe | Avg Processing Time: 0.0055 sec
Filter: box_filter + top_hat_black_hat | Avg Processing Time: 0.0081 sec
Filter: anisotropic + sharpen | Avg Processing Time: 0.0327 sec
Filter: anisotropic + sobel_filter | Avg Processing Time: 0.0400 sec
Filter: anisotropic + canny_edge | Avg Processing Time: 0.0332 sec
Filter: anisotropic + laplacian_filter | Avg Processing Time: 0.0355 sec
Filter: anisotropic + opening_closing | Avg Processing Time: 0.0330 sec
Filter: anisotropic + clahe | Avg Processing Time: 0.0315 sec
Filter: anisotropic + top_hat_black_hat | Avg Processing Time: 0.0351 sec
Filter: wiener + sharpen | Avg Processing Time: 0.1853 sec
Filter: wiener + sobel_filter | Avg Processing Time: 0.1908 sec
Filter: wiener + canny_edge | Avg Processing Time: 0.1859 sec
Filter: wiener + laplacian_filter | Avg Processing Time: 0.1859 sec
Filter: wiener + opening_closing | Avg Processing Time: 0.1835 sec
Filter: wiener + clahe | Avg Processing Time: 0.1820 sec
Filter: wiener + top_hat_black_hat | Avg Processing Time: 0.1845 sec

2025-03-07

전체 필터의 성능 비교(1000장)

filter_performance_roi_results2.csv

확인해볼 필터

Filtered_and_Sorted_Filters__Updated_with_Mean_Similarity_.csv

정렬 기준

missing gap이 0인 애들 중에서

ROI 1-3끼리, ROI 4-6끼리 비슷한 값이어야 하므로, 각 그룹 내부에서 **최댓값과 최솟값의 차이**를 계산했어.

(1) ROI 1-3 평균 차이 (ROI 1-3 Mean Diff)

ROI 1-3 Mean Diff=max(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)
−min(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)\text{ROI 1-3 Mean Diff} = \max(\text{ROI 1 Mean Gap}, \text{ROI 2 Mean Gap}, \text{ROI 3 Mean Gap}) - \min(\text{ROI 1 Mean Gap}, \text{ROI 2 Mean Gap}, \text{ROI 3 Mean Gap})

ROI 1-3 Mean Diff=max(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)
−min(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)

- ROI 1, 2, 3의 평균값 중에서 **가장 큰 값과 가장 작은 값의 차이**를 계산
- 차이가 작을수록 ROI 1-3이 균일하다는 의미

(2) ROI 4-6 평균 차이 (ROI 4-6 Mean Diff)

ROI 4-6 Mean Diff=max(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)
−min(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)\text{ROI 4-6 Mean Diff} = \max(\text{ROI 4 Mean Gap}, \text{ROI 5 Mean Gap}, \text{ROI 6 Mean Gap}) - \min(\text{ROI 4 Mean Gap}, \text{ROI 5 Mean Gap}, \text{ROI 6 Mean Gap})

ROI 4-6 Mean Diff=max(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)
−min(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)

- ROI 4, 5, 6의 평균값 중에서 **가장 큰 값과 가장 작은 값의 차이**를 계산
- 차이가 작을수록 ROI 4-6이 균일하다는 의미

(3) 두 그룹의 평균 차이 합 (Group Mean Difference Sum)

Group Mean Difference Sum=ROI 1-3 Mean Diff+ROI 4-6 Mean Diff\text{Group Mean Difference Sum} = \text{ROI 1-3 Mean Diff} + \text{ROI 4-6 Mean Diff}

Group Mean Difference Sum=ROI 1-3 Mean Diff+ROI 4-6 Mean Diff

- 이 값이 작을수록, ROI 1-3과 ROI 4-6 각각 내부에서 값들이 비슷하다는 의미

3. ROI 1-3과 ROI 4-6의 표준편차 계산

ROI 1-3과 ROI 4-6이 내부적으로 균일한 값들을 가지는지도 고려하기 위해, 각 그룹의 **표준편차**를 계산했어.

(4) ROI 1-3 표준편차 (ROI 1-3 Std Dev)

ROI 1-3 Std Dev=std(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)\text{ROI 1-3 Std Dev} = \text{std}(\text{ROI 1 Mean Gap}, \text{ROI 2 Mean Gap}, \text{ROI 3 Mean Gap})

ROI 1-3 Std Dev=std(ROI 1 Mean Gap,ROI 2 Mean Gap,ROI 3 Mean Gap)

- ROI 1, 2, 3의 표준편차를 계산
- 값이 작을수록 ROI 1-3 내에서 변동성이 적다는 의미

(5) ROI 4-6 표준편차 (ROI 4-6 Std Dev)

ROI 4-6 Std Dev=std(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)\text{ROI 4-6 Std Dev} = \text{std}(\text{ROI 4 Mean Gap}, \text{ROI 5 Mean Gap}, \text{ROI 6 Mean Gap})

ROI 4-6 Std Dev=std(ROI 4 Mean Gap,ROI 5 Mean Gap,ROI 6 Mean Gap)

- ROI 4, 5, 6의 표준편차를 계산
- 값이 작을수록 ROI 4-6 내에서 변동성이 적다는 의미

(6) 두 그룹의 표준편차 합 (Group Std Deviation Sum)

Group Std Deviation Sum=ROI 1-3 Std Dev+ROI 4-6 Std Dev\text{Group Std Deviation Sum} = \text{ROI 1-3 Std Dev} + \text{ROI 4-6 Std Dev}

Group Std Deviation Sum=ROI 1-3 Std Dev+ROI 4-6 Std Dev

- 값이 작을수록 **ROI 1-3과 ROI 4-6 각각 내부에서 변동성이 적다**는 의미

추린 필터 종류 6개

fast_global_smoother + sharpen

bilateral + sharpen

median + sharpen

gaussian + sharpen

wiener + clahe

gaussian + opening_closing

2025-03-11

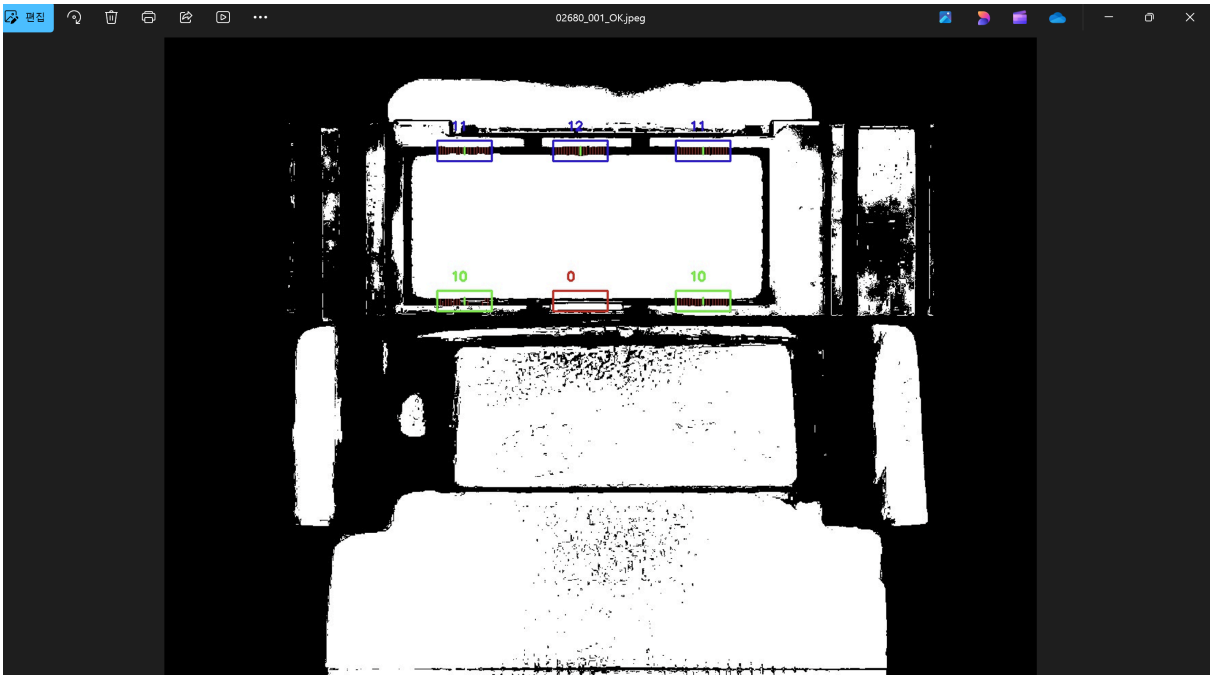
필터 하나 정해서

하위 10프로 상위 10프로 확인하고 진짜 문제 있는 것들 확인

fast_global_smoother + sharpen

1000장 확인





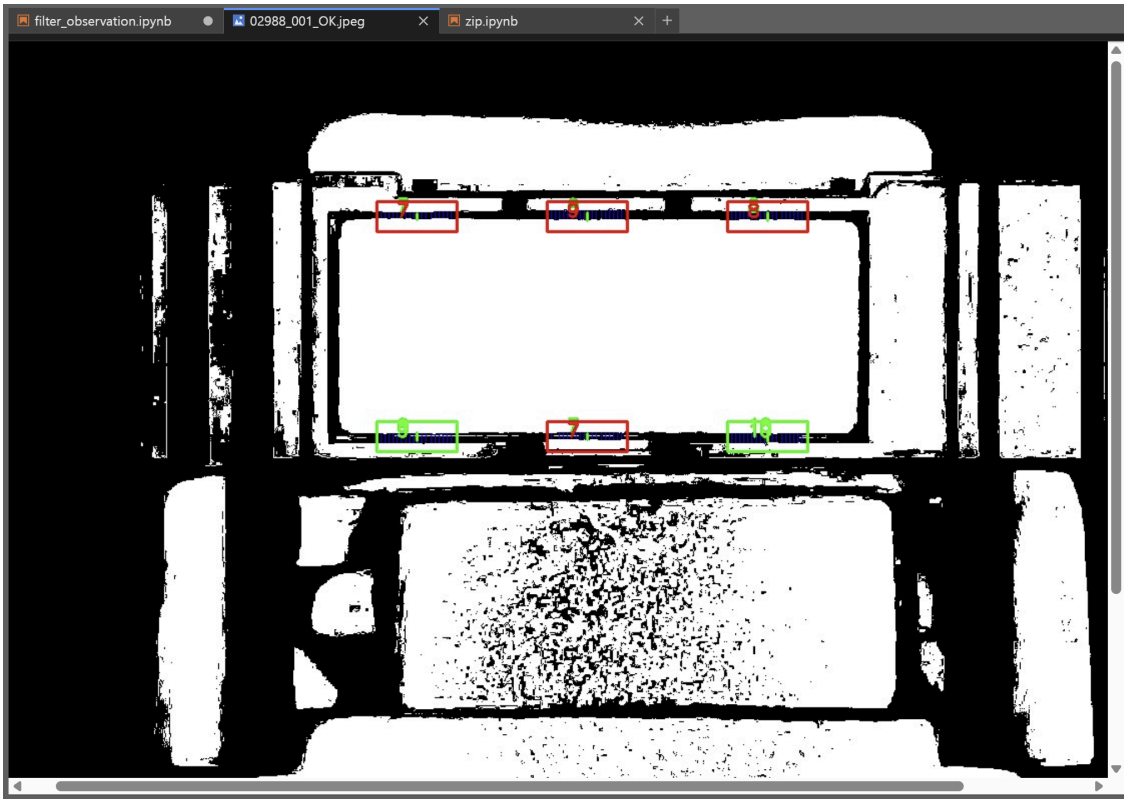
기존 갭 탐지 방식

연속된 검정갭들 중 (5픽셀 이하는 제외하고) 가장 긴 갭을 표시.

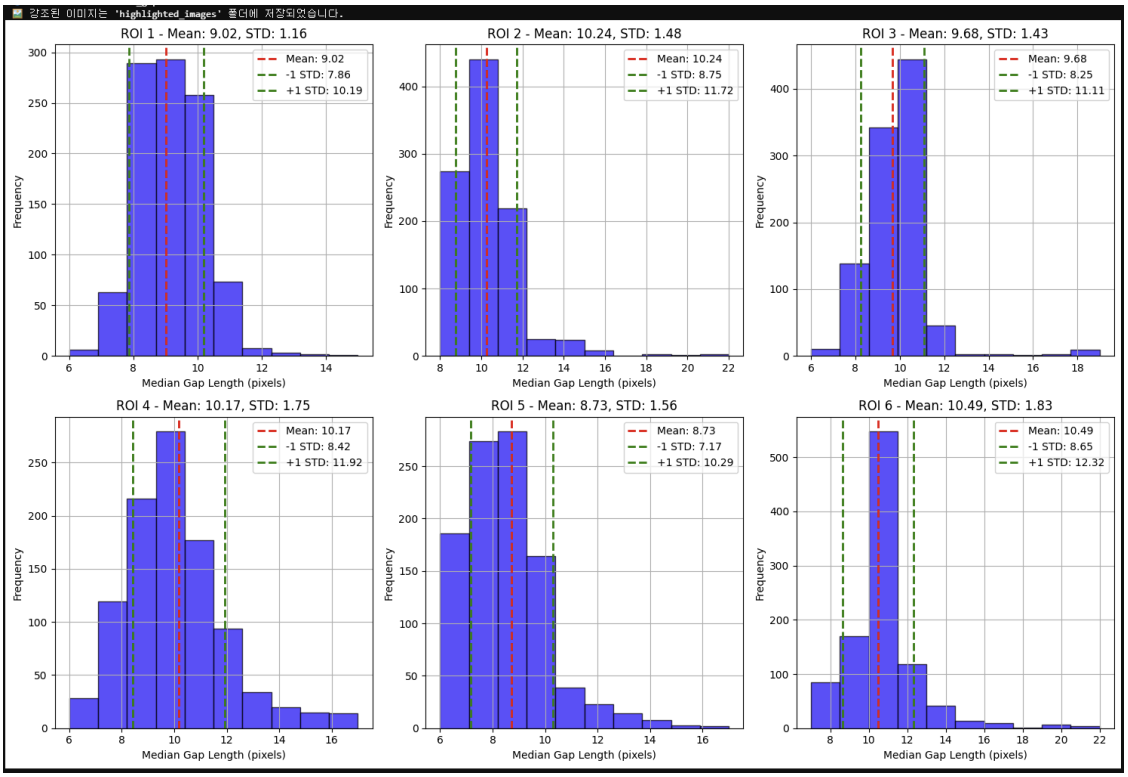
문제점 : 중간에 흰색 노이즈가 있는 경우 잘못된 길이 측정. / 중간에 갭이 측정 안된경우 9장 (0.009)

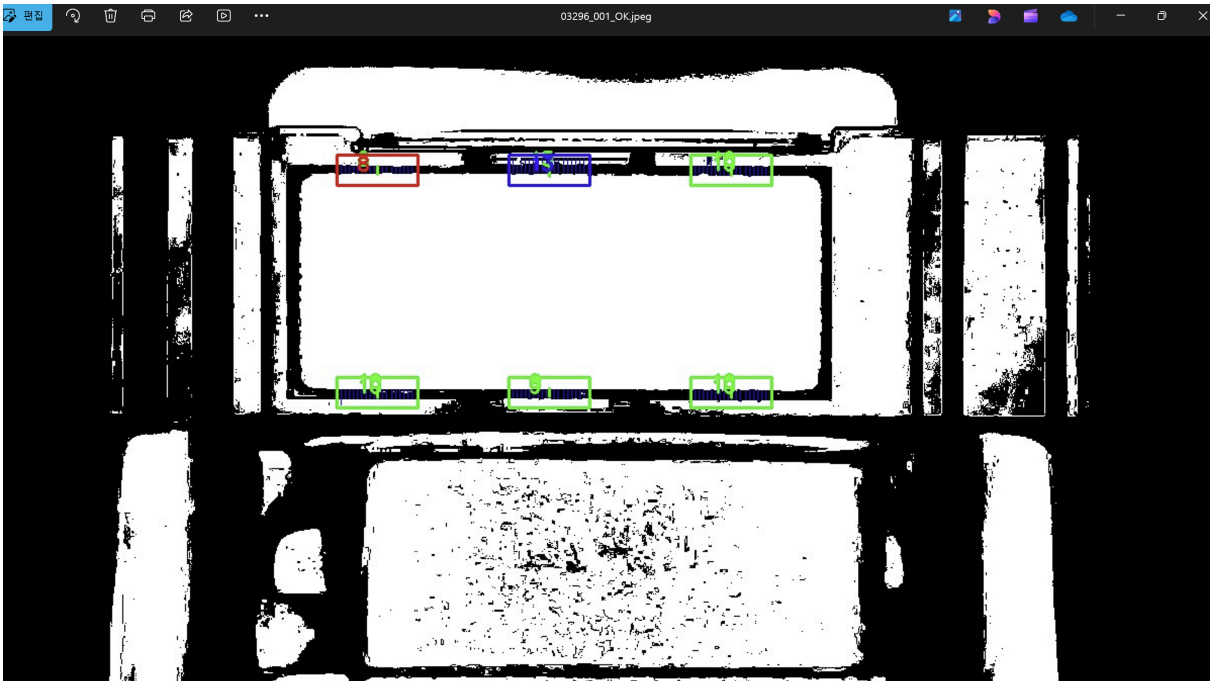
해결 : 연속된 검정갭들 중 길이가 5픽셀 이하로 떨어져 있으면 합치고 가장 긴 갭을 표시

중간에 갭이 없는 경우 0장

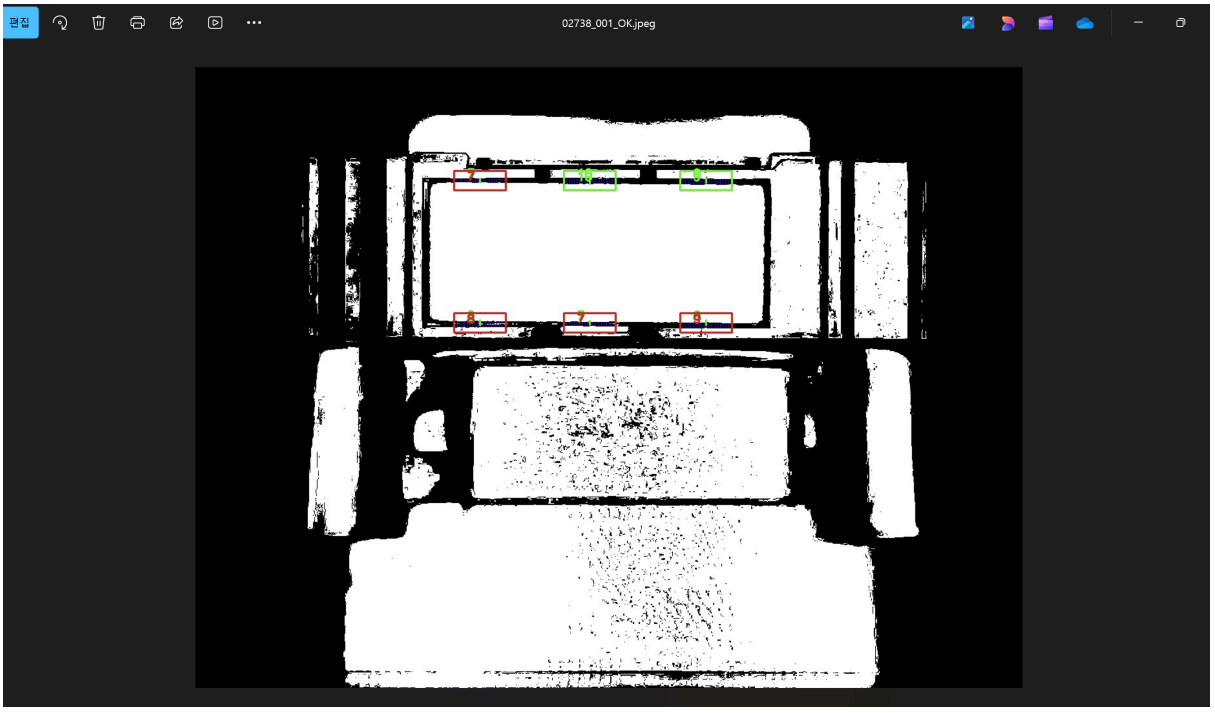


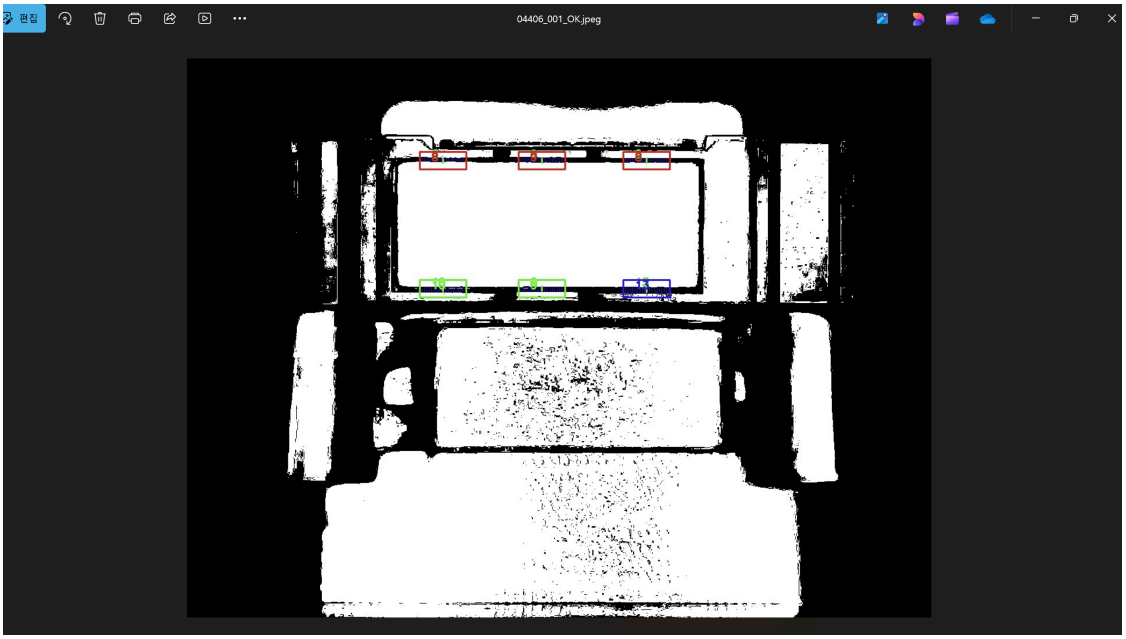
median_gap 분포표



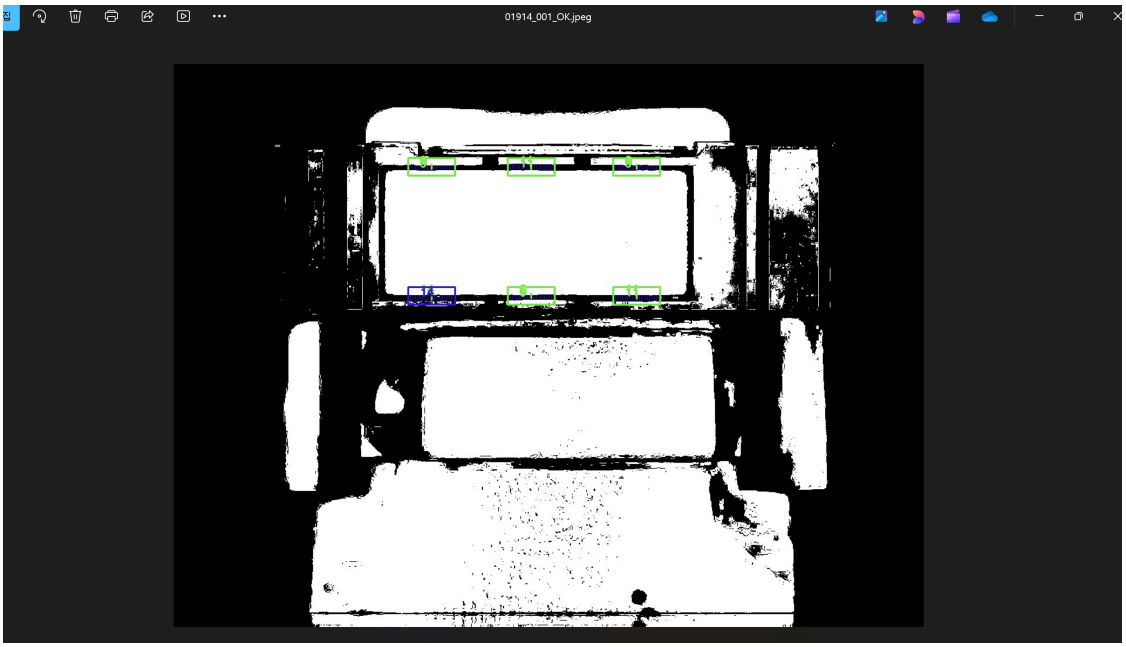


5픽셀 너무 길다. 조정 필요





이 사진 정상인데 하단 10 , 9 , 13 나옴



좌측 하단 이진화 잘못된 경우

2025-03-13

필터 옵션을 변경해보거나

2025-03-24

마그넷 전부 안해도 됨

1C2G GAPCHECK만

일단 객체 고정 먼저 해야함. 안하니 갭체크 데이터 신뢰 X

객체 고정 완료(rotate_and_parallel translation.ipynb 에서 최종코드 돌림)

마그넷부분(?)의 좌상단점 (400,420)으로 고정

2025-03-25

이동된 객체 roi 위치 설정하고

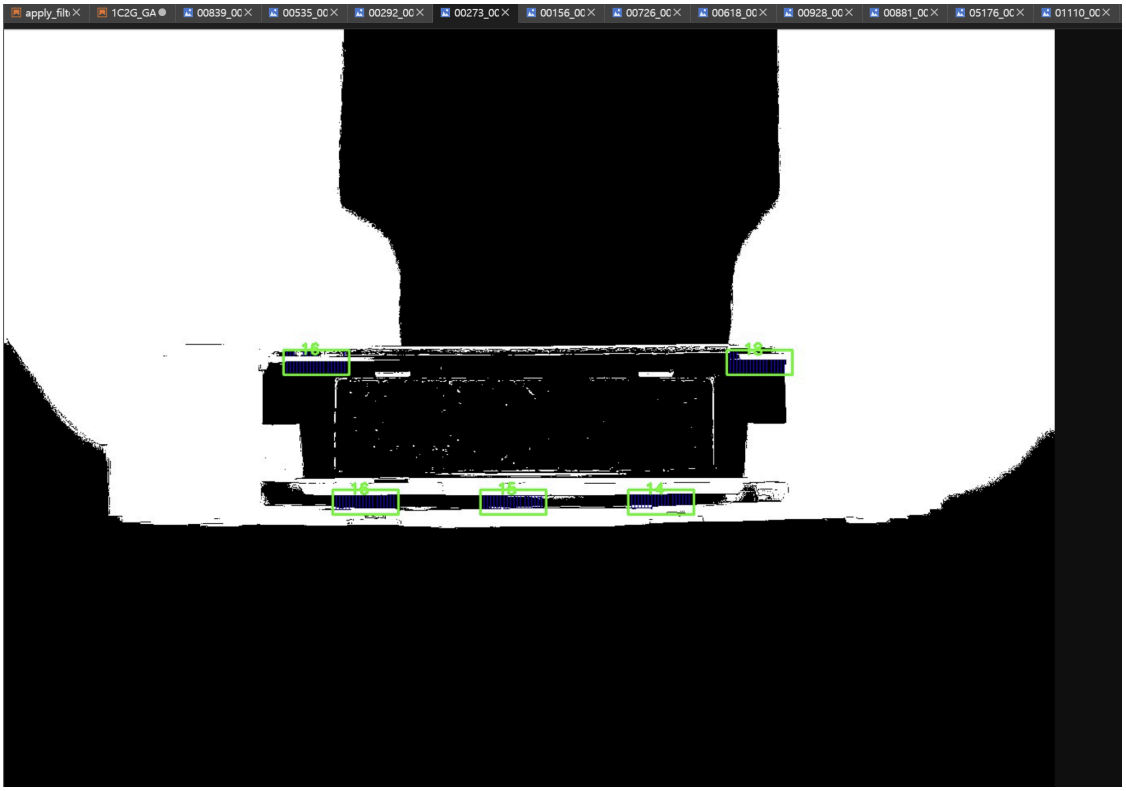
필터 비교해보기

일단 먼저 하나 돌렸을 때 상단갭은 빛이 세서 흰색 갭으로 나오고

하단은 검정 갭으로 측정 됨

roi 위치

```
rois = [
      {"x": 340, "y": 390, "w": 80, "h": 30},
      {"x": 880, "y": 390, "w": 80, "h": 30},
      {"x": 400, "y": 560, "w": 80, "h": 30},
      {"x": 580, "y": 560, "w": 80, "h": 30},
      {"x": 760, "y": 560, "w": 80, "h": 30},
   ]
```



필터 후보

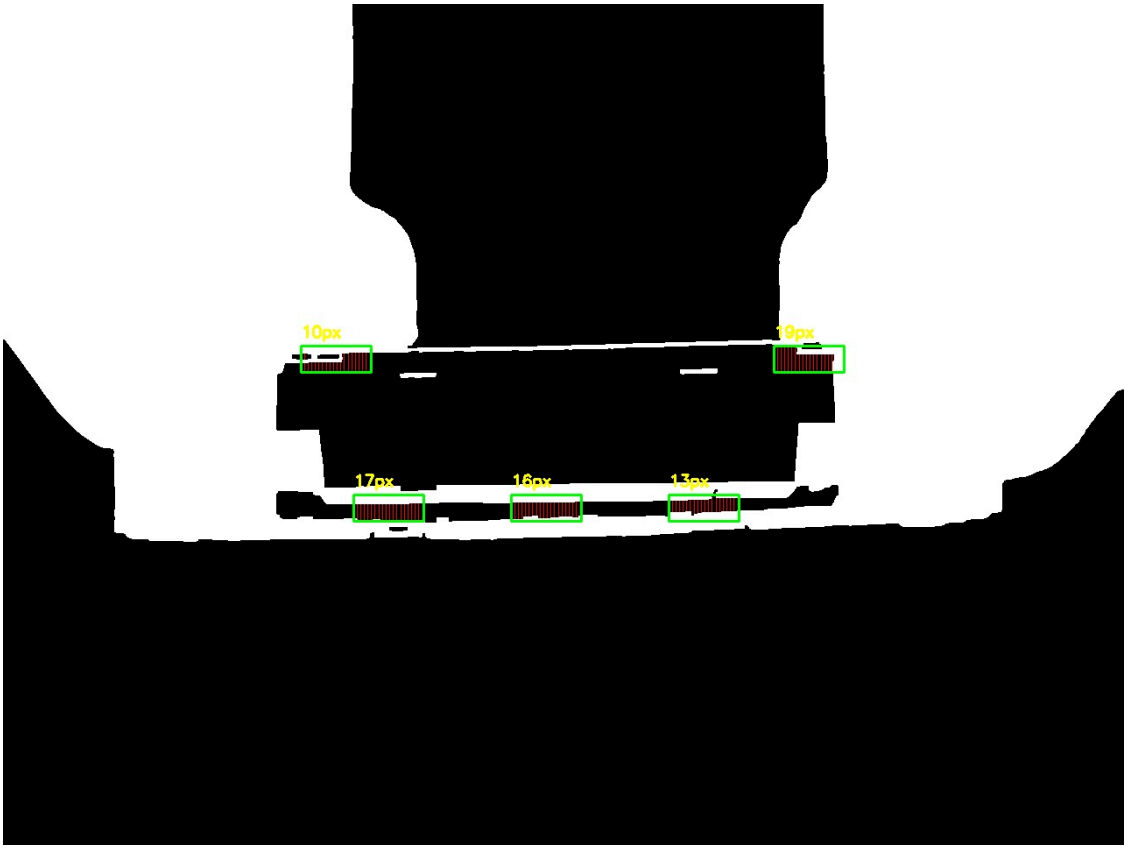
1_2

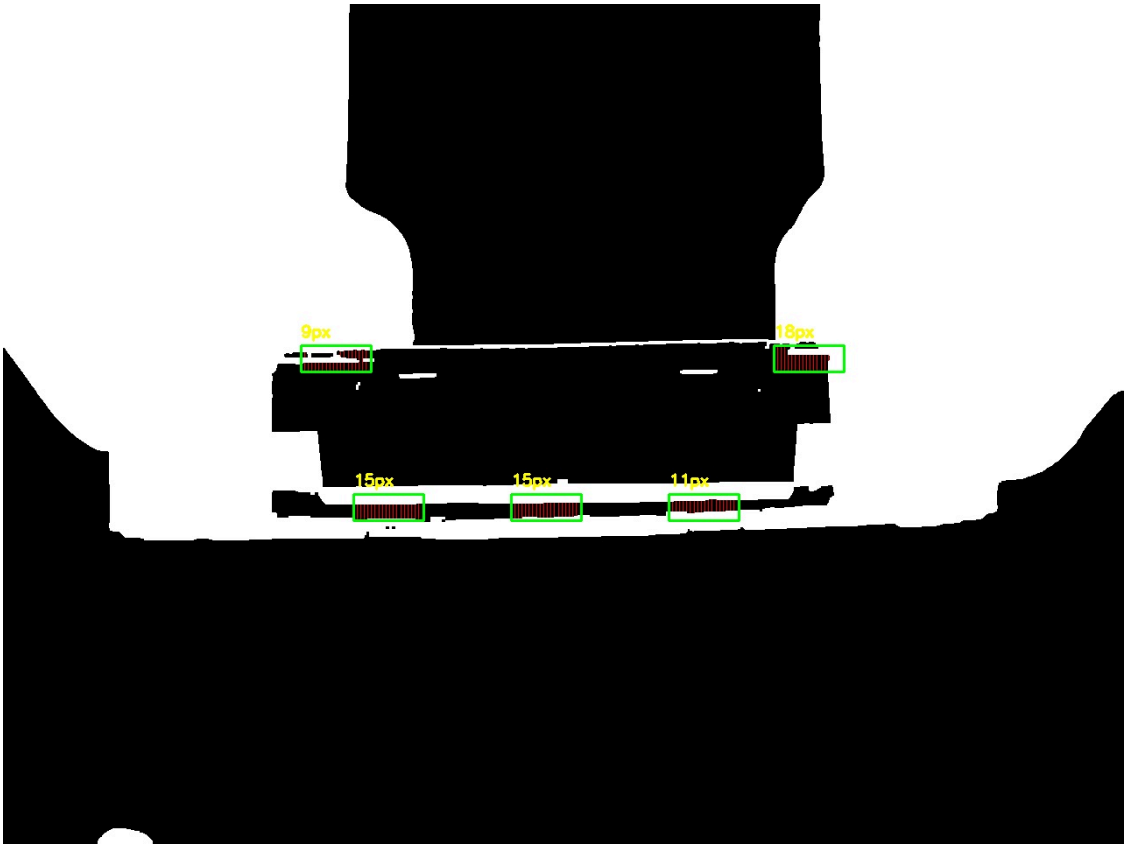
1
anisotropic
bilateral
fast_global_smoother
gaussian
guided_filter
wavelet
wiener

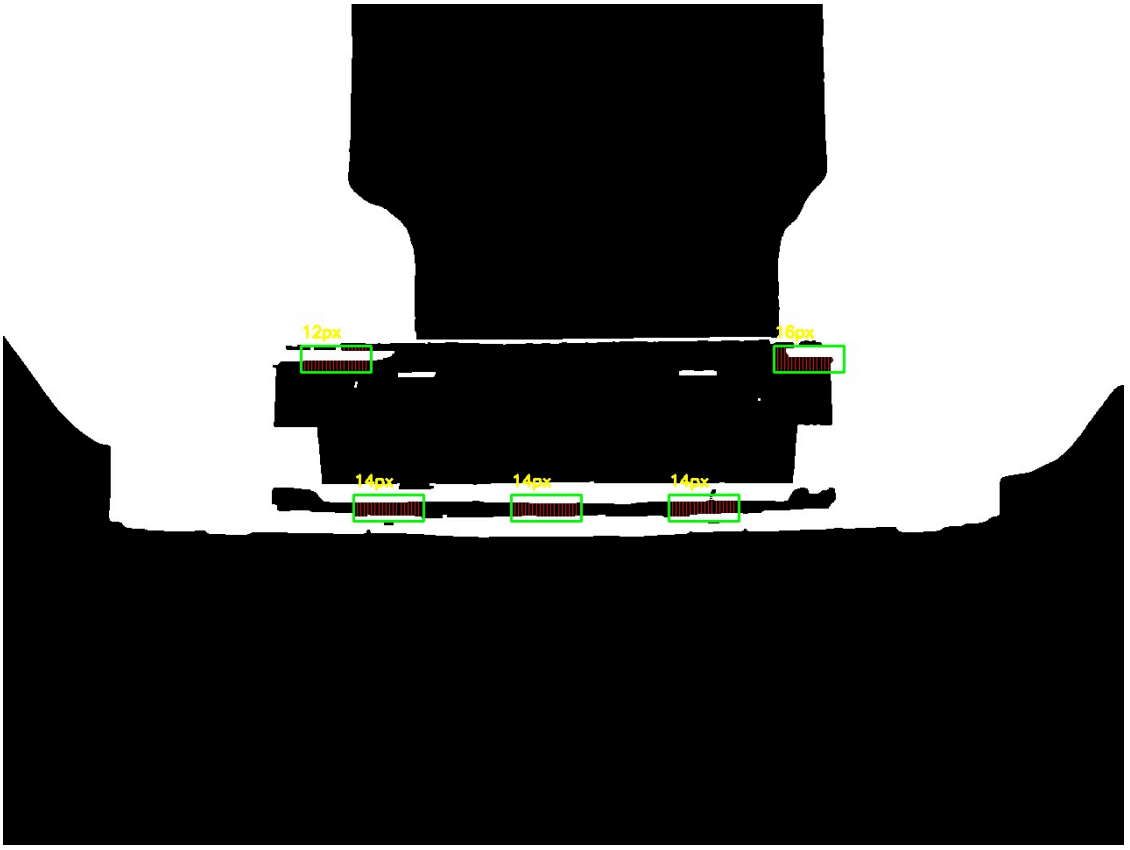
2
opening_closing
clahe

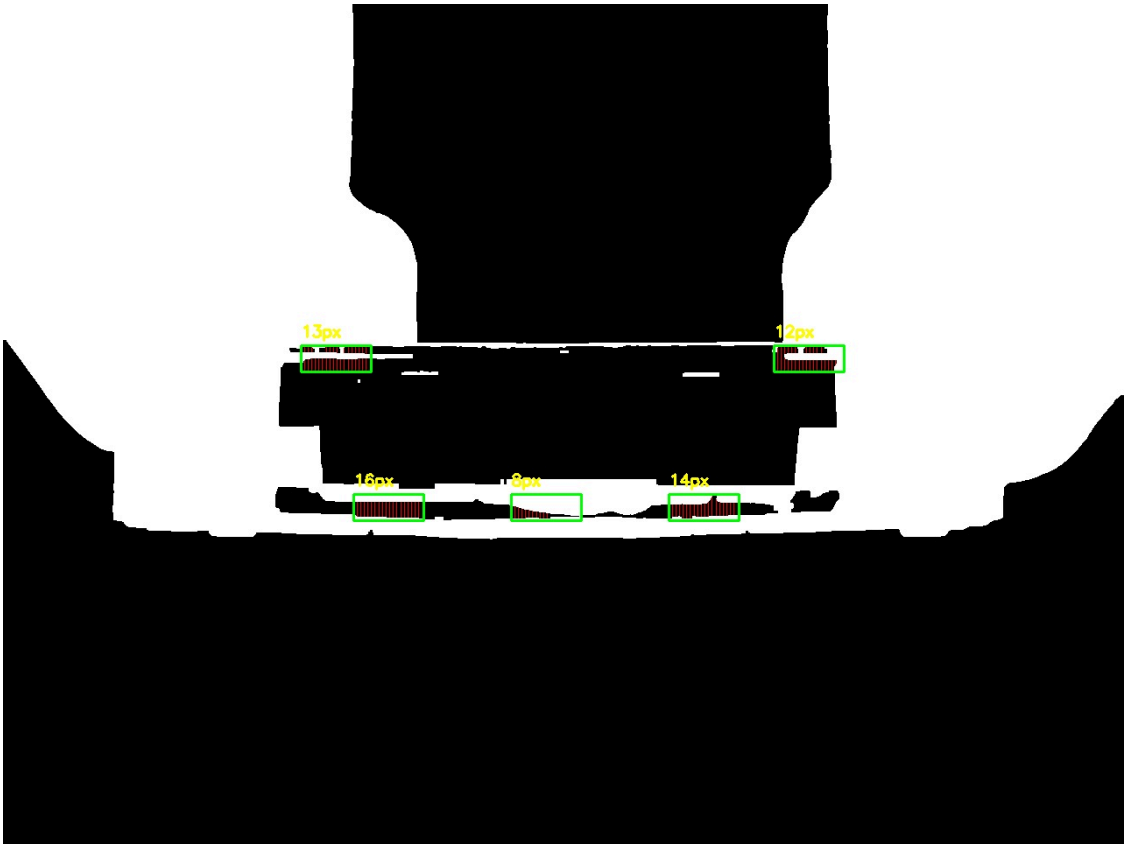
wiener_opening_closing_{'h'10_'hColor'10_'templateWindowSize'7_'searchWindowSize'21}_{'kernel'3}

이게 제일 나아보이는데

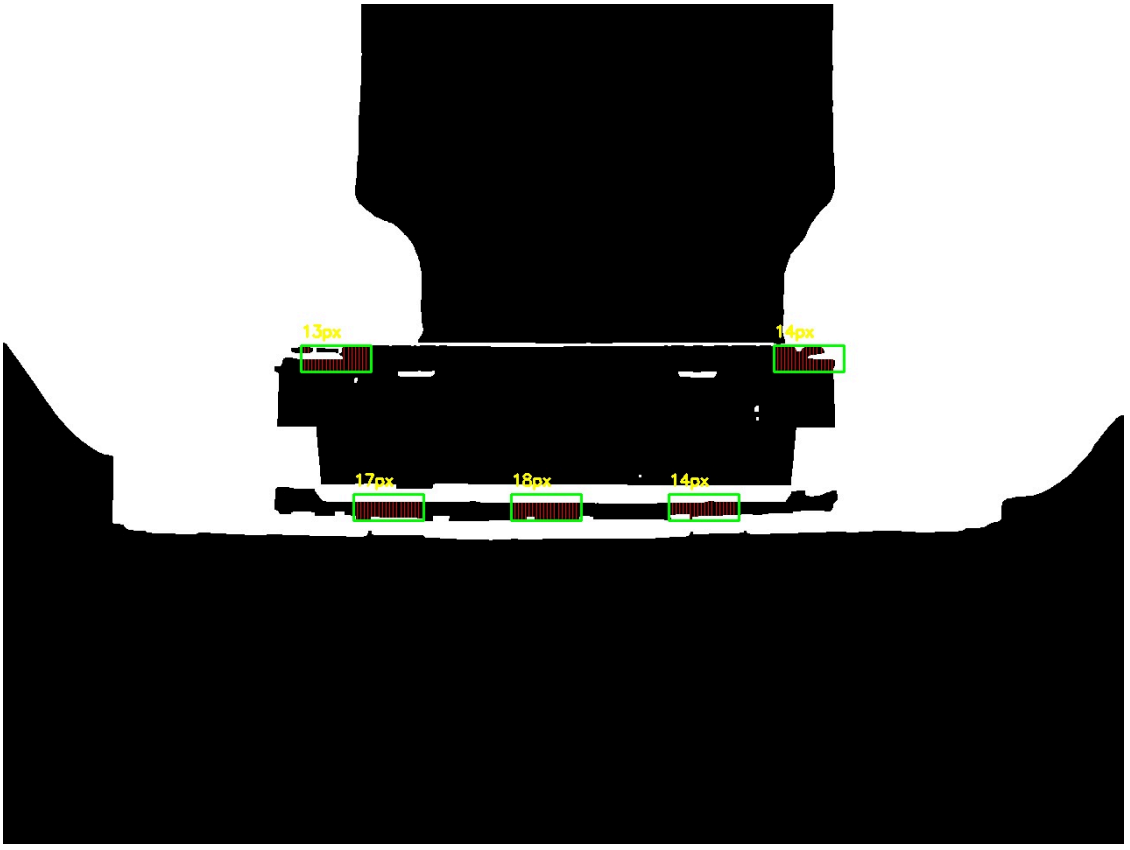

10px
19px
17px
16px
13px

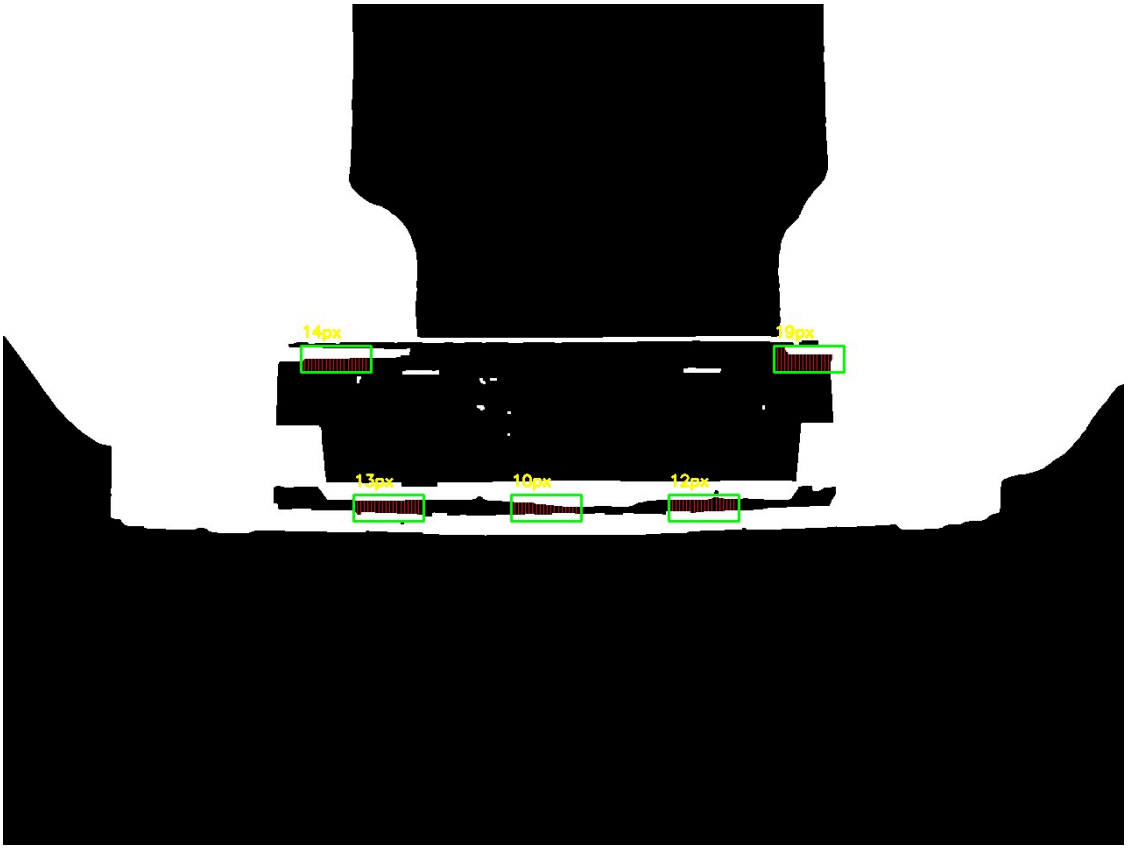

9px
18px
15px
15px
11px

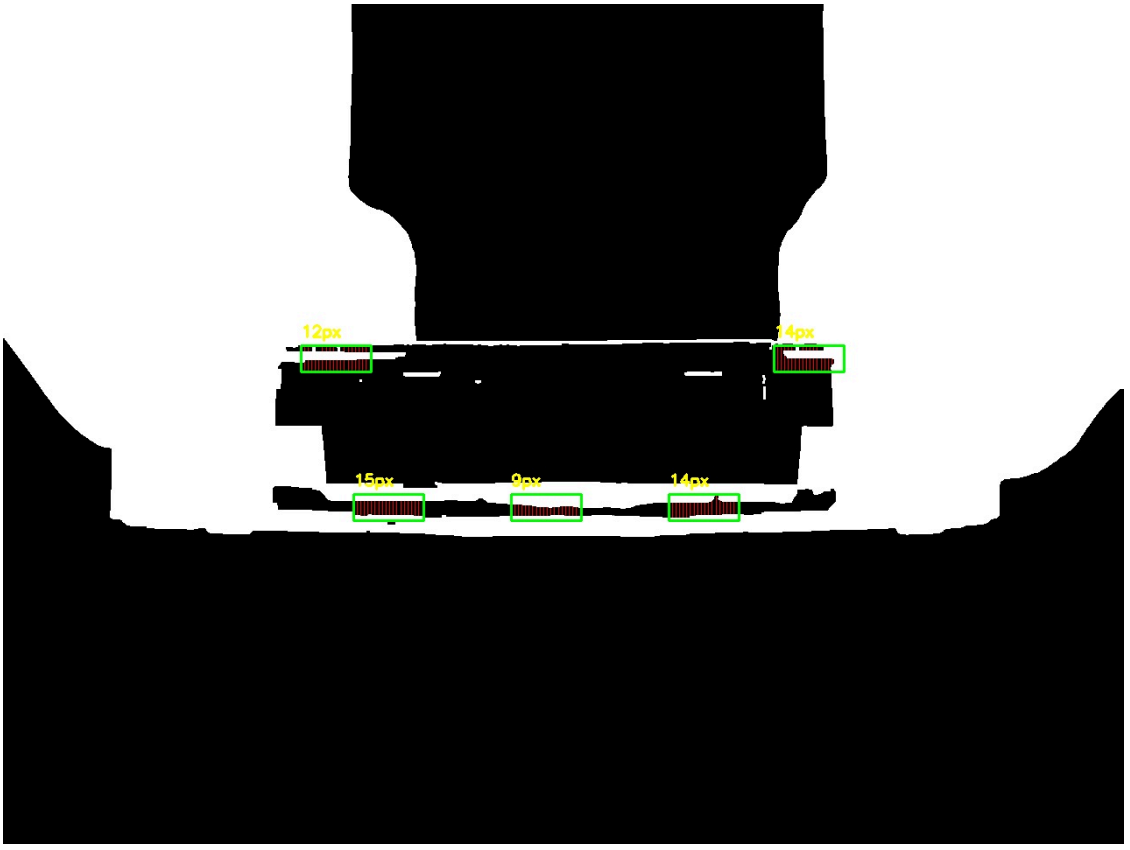

12px
16px
14px
14px
14px

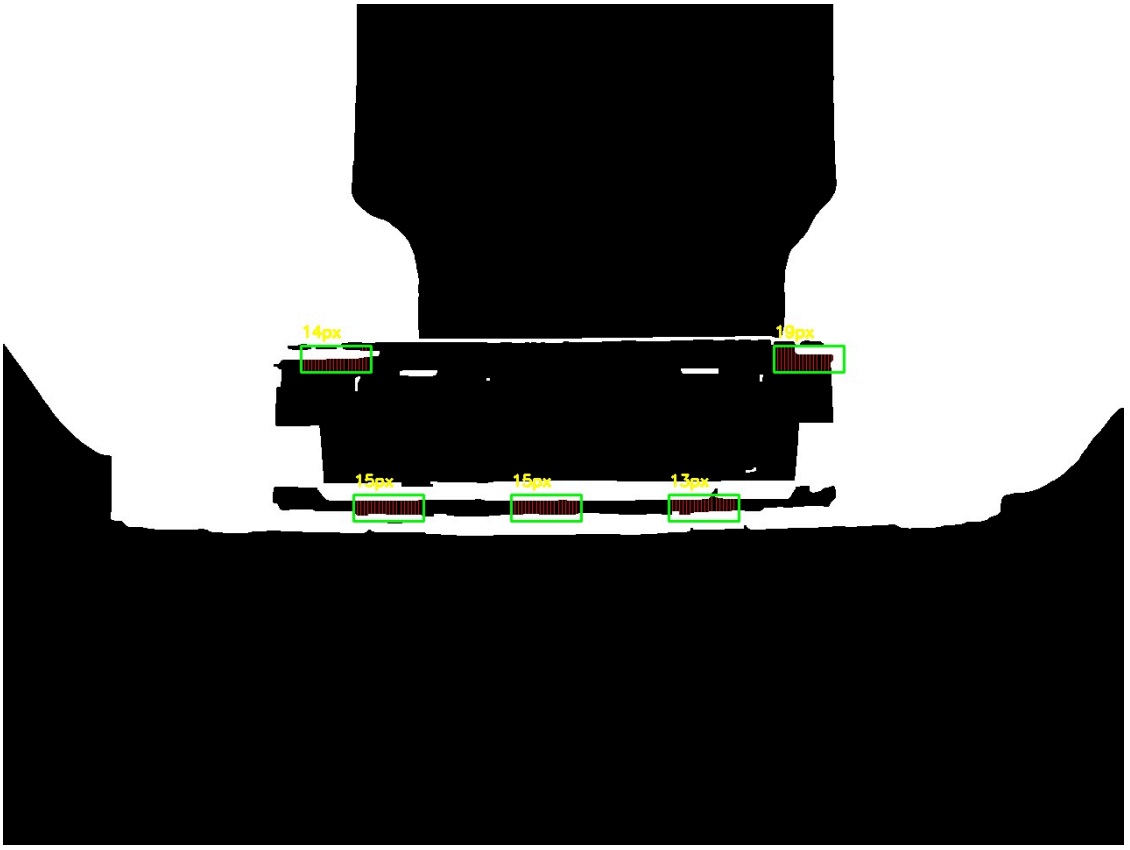

13px
12px
16px
14px

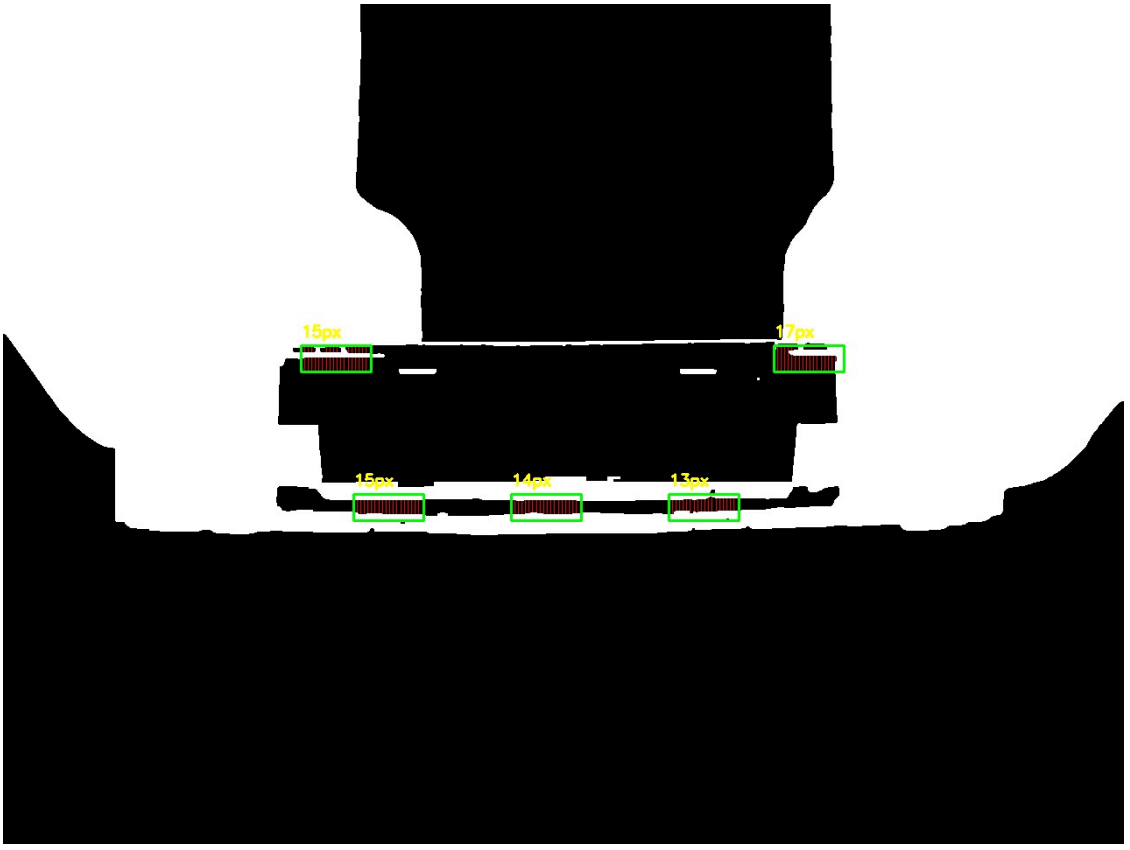

16px
14px
14px
10px


13px
14px
17px
18px
14px


14px
10px
13px
10px
12px

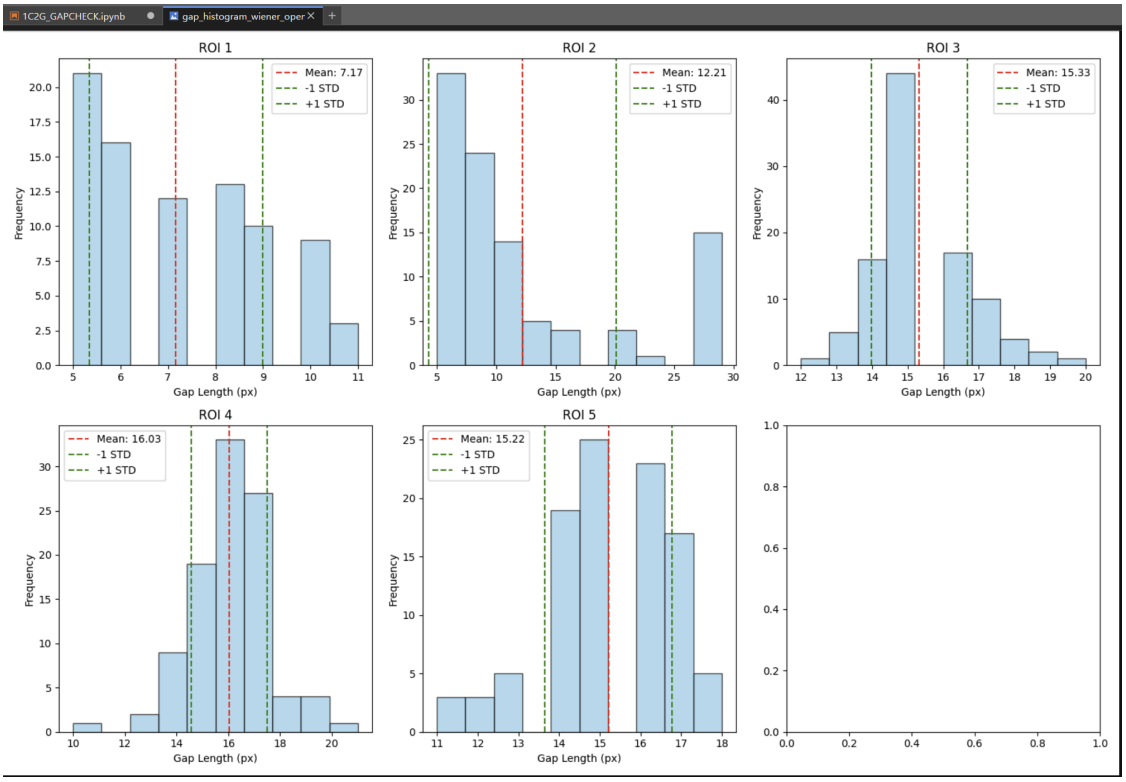

12px
14px
15px
9px
14px


14px
19px
15px
15px
13px


15px
17px
15px
14px
13px

기존 필터에 이진화 경계값 변경

이진화 임계값 130, 100장



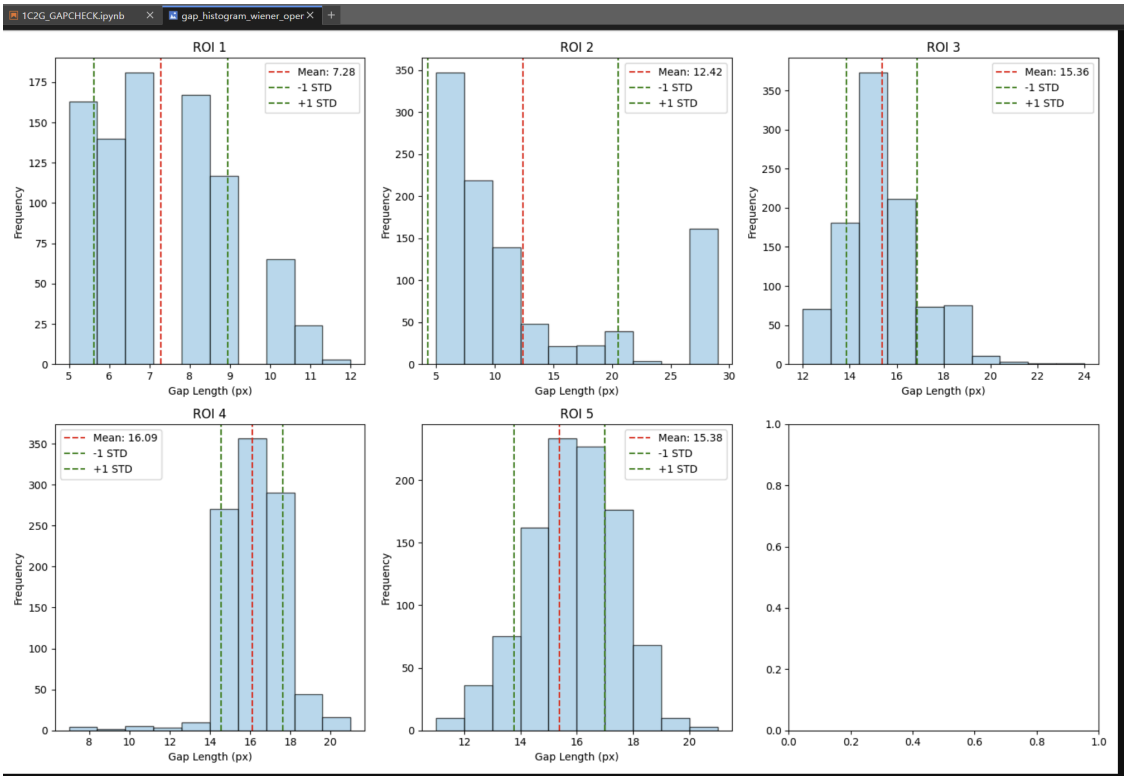
이진화 임계값 130, 1000장

옵션

```
def apply_threshold(image, threshold_value=130)

    noise_filter = "wiener"
edge_filter = "opening_closing"

noise_params = {"h": 10, "hColor": 10, "templateWindowSize": 7, "searchWindowSize": 21}
edge_params = {"kernel": 3}
```



상단은 매우 별로임

아예 필터부터 다시 찾고 임계값도 다시 확인해보기

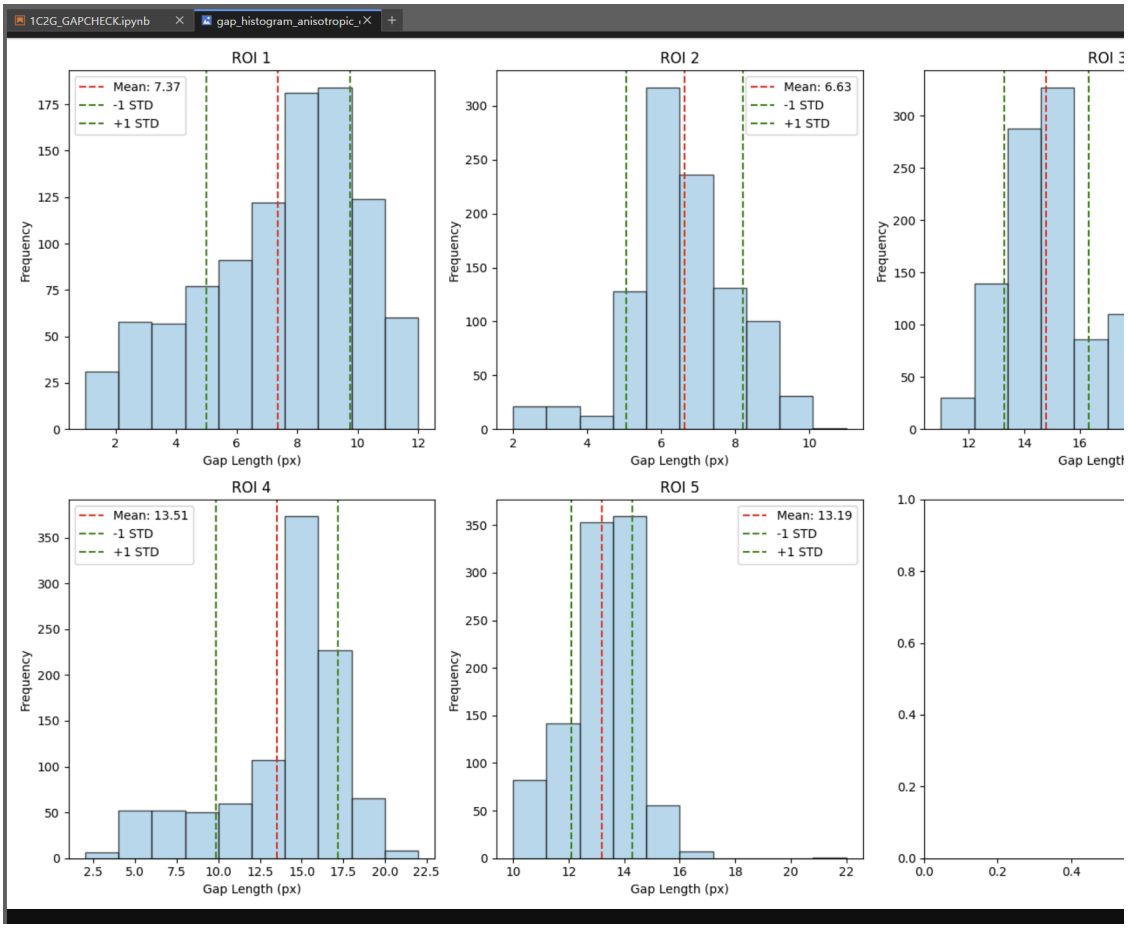
하단 필터

```
def apply_threshold(image, threshold_value=100)

noise_filter = "anisotropic"
edge_filter = "opening_closing"

noise_params = {"radius": 8, "eps": 0.01}
edge_params = {"kernel": 3}
```

결과 roi 1,2(1000장)



상단 하단 각 다른 필터 적용하고 이진화 경계값도 다르게해서 적용하는 방식

상단 anisotropic + threshold=100
하단 wiener + threshold=130
으로 적용 후 이미지 합친 후 갭체크 하는 코드

```
import cv2
import numpy as np
import bm3d
import os
import time
import csv
import skimage.restoration as restoration
from scipy.ndimage import gaussian_filter
import scipy.signal
import scipy.ndimage
import matplotlib.pyplot as plt
import pandas as pd
import matplotlib.font_manager as fm
from skimage.metrics import peak_signal_noise_ratio as psnr
from skimage.metrics import structural_similarity as ssim
```

```
from itertools import product
from skimage.restoration import denoise_wavelet



# 필터별 파라미터 그리드 정의
filter_parameter_grid = {
    "bilateral": [
        {"d": 9, "sigmaColor": 75, "sigmaSpace": 75},
        {"d": 5, "sigmaColor": 50, "sigmaSpace": 50}
    ],
    "fast_global_smoother": [
        {"sigma_s": 60, "sigma_r": 0.4},
        {"sigma_s": 30, "sigma_r": 0.2}
    ],
    "guided_filter": [
        {"radius": 8, "eps": 0.01},
        {"radius": 4, "eps": 0.001}
    ],
    "gaussian": [
        {"ksize": 5, "sigma": 1.0},
        {"ksize": 3, "sigma": 0.5}
    ],
    "anisotropic": [
        {"radius": 8, "eps": 0.01}
    ],
    "wiener": [
        {"h": 10, "hColor": 10, "templateWindowSize": 7, "searchWindowSize": 21}
    ],
    "wavelet": [
        {"method": "BayesShrink", "mode": "soft", "rescale_sigma": True},
        {"method": "VisuShrink", "mode": "soft", "rescale_sigma": True}
    ],
    "opening_closing": [
        {"kernel": 1},
        {"kernel": 3}
    ],
    "clahe": [
        {"clipLimit": 2.0, "tileGridSize": (8, 8)},
        {"clipLimit": 3.0, "tileGridSize": (4, 4)}
    ]
}

# 🔄 모든 이미지에서 ROI별 가장 긴 검정 픽셀 길이 저장할 전역 리스트
global_roi_gaps = {i: [] for i in range(6)}  # ROI 인덱스별 데이터 저장
global_roi_missing_gaps = {i: 0 for i in range(6)}  # ROI별 측정되지 않은 갭 개수 저장

def visualize_gap_distribution(output_dir, selected_filter):
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
```

```
    axes = axes.flatten()

    for i in range(6):
        ax = axes[i]
        if global_roi_gaps[i]:
            data = global_roi_gaps[i]
            mean_val = np.mean(data)
            std_val = np.std(data)

            ax.hist(data, bins=10, color='skyblue', edgecolor='black', alpha=0.7)
            ax.axvline(mean_val, color='red', linestyle='--', label=f"Mean: {mean_val:.2f}")
            ax.axvline(mean_val - std_val, color='green', linestyle='--', label=f"-1 STD")
            ax.axvline(mean_val + std_val, color='green', linestyle='--', label=f"+1 STD")

            ax.set_title(f"ROI {i+1}")
            ax.set_xlabel("Gap Length (px)")
            ax.set_ylabel("Frequency")
            ax.legend()

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, f"gap_histogram_{selected_filter}.png"))
    plt.close()

    # Missing Gap bar chart
    plt.figure(figsize=(10, 5))
    rois = list(global_roi_missing_gaps.keys())
    values = list(global_roi_missing_gaps.values())
    bars = plt.bar(rois, values, color='salmon', alpha=0.8)
    plt.xticks(rois, [f"ROI {i+1}" for i in rois])
    plt.xlabel("ROI")
    plt.ylabel("Missing Count")
    plt.title(f"Missing Gaps per ROI - {selected_filter}")
    for bar, val in zip(bars, values):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(), str(val), ha='center', va='bottom')
    plt.savefig(os.path.join(output_dir, f"missing_gaps_{selected_filter}.png"))
    plt.close()

def plot_black_pixel_distribution(global_roi_gaps, global_roi_missing_gaps, selected_filter):
    """
    ROI별 검정 픽셀 길이 분포 및 평균, 표준편차 시각화
    """

    fig, axes = plt.subplots(2, 3, figsize=(15, 10))  # 2×3 배열로 그래프 배치
    axes = axes.flatten()  # 2D 배열을 1D 리스트로 변환

    for i in range(6):
        ax = axes[i]
        if len(global_roi_gaps[i]) > 0:
            data = global_roi_gaps[i]
```

```
            mean_value = np.mean(data)
            std_dev = np.std(data)

            # 히스토그램 그리기
            ax.hist(data, bins=10, alpha=0.7, color='blue', edgecolor='black')

            # 평균 및 표준편차 선 추가
            ax.axvline(mean_value, color='red', linestyle='dashed', linewidth=2, label=f"Mean: {mean
_value:.2f}")
            ax.axvline(mean_value - std_dev, color='green', linestyle='dashed', linewidth=2, label=f"-
1 STD: {mean_value - std_dev:.2f}")
            ax.axvline(mean_value + std_dev, color='green', linestyle='dashed', linewidth=2, label=
f"+1 STD: {mean_value + std_dev:.2f}")

            ax.set_xlabel("Black Pixel Length (pixels)")
            ax.set_ylabel("Frequency")
            ax.set_title(f"ROI {i+1} - Mean: {mean_value:.2f}, STD: {std_dev:.2f}")
            ax.legend()
            ax.grid(True)

    plt.tight_layout()
    plt.show()  # 즉시 출력

    # ✅ **각 ROI별 Missing Gap 개수 막대 그래프 추가**
    plt.figure(figsize=(10, 6))
    rois = list(global_roi_missing_gaps.keys())  # ROI 인덱스 리스트
    missing_counts = list(global_roi_missing_gaps.values())  # 각 ROI의 Missing Gap 개수

    bars = plt.bar(rois, missing_counts, color='red', alpha=0.7)

    plt.xlabel("ROI Index")
    plt.ylabel("Missing Gap Count")  # 👉 개수로 표시
    plt.title(f"Missing Gaps Count per ROI - {selected_filter}")
    plt.xticks(rois, [f"ROI {i+1}" for i in rois])
    plt.grid(axis='y')

    # ✅ **막대 위에 Missing Gap 개수 표시**
    for bar, count in zip(bars, missing_counts):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(), str(count),
                 ha='center', va='bottom', fontsize=12, color='black')

    plt.show()  # 즉시 출력


def apply_filter(image, noise_filter, edge_filter, params):
    processed_image = image.astype(np.float32)

    if noise_filter == "anisotropic":
        radius = params.get("radius", 8)
```

```
        eps = params.get("eps", 0.01)
        processed_image = cv2.ximgproc.guidedFilter(image, image, radius, eps)

    elif noise_filter == "wiener":
        h = params.get("h", 10)
        h_color = params.get("hColor", 10)
        template_window = params.get("templateWindowSize", 7)
        search_window = params.get("searchWindowSize", 21)
        processed_image = cv2.fastNlMeansDenoisingColored(image, None, h, h_color, template_w
indow, search_window)

    if edge_filter == "opening_closing":
        kernel_size = params.get("kernel", 1)
        kernel = np.ones((kernel_size, kernel_size), np.uint8)
        opened = cv2.morphologyEx(processed_image, cv2.MORPH_OPEN, kernel)
        processed_image = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)

    processed_image = np.clip(processed_image, 0, 255).astype(np.uint8)
    if len(processed_image.shape) == 2:
        processed_image = cv2.cvtColor(processed_image, cv2.COLOR_GRAY2BGR)

    return processed_image


def compare_filters_with_param_grid(folder_path, noise_filters, edge_filters, max_images=10):
    for noise_filter in noise_filters:
        for edge_filter in edge_filters:
            noise_params_list = filter_parameter_grid.get(noise_filter, [{}])
            edge_params_list = filter_parameter_grid.get(edge_filter, [{}])

            for noise_params, edge_params in product(noise_params_list, edge_params_list):
                param_label = f"{noise_filter}_{edge_filter}_" + \
                              f"{str(noise_params)}_{str(edge_params)}".replace(" ", "").replace(":", "").repla
ce(",", "_")
                print(f"\n🔍 Now processing: {param_label}")

                process_images_with_params(
                    folder_path=folder_path,
                    output_prefix=param_label,
                    noise_filter=noise_filter,
                    edge_filter=edge_filter,
                    noise_params=noise_params,
                    edge_params=edge_params,
                    max_images=max_images
                )


def apply_threshold(image, threshold_value=100):
    if len(image.shape) == 3:
```

```
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(image, threshold_value, 255, cv2.THRESH_BINARY)
    return binary


def analyze_and_visualize_on_combined(image, img_a, img_b, binary_a, binary_b, split_y=500, nu
m_samples=20):
    global global_roi_gaps, global_roi_missing_gaps
    global_roi_gaps = {i: [] for i in range(6)}
    global_roi_missing_gaps = {i: 0 for i in range(6)}

    h, w = image.shape[:2]
    upper_half = img_a[:split_y, :]
    lower_half = img_b[split_y:, :]
    combined_img = np.vstack([upper_half, lower_half])

    cv2.line(combined_img, (0, split_y), (w, split_y), (0, 0, 255), 2)
    cv2.putText(combined_img, "Anisotropic (ROI 1,2)", (10, 40), cv2.FONT_HERSHEY_SIMPLEX, 1,
(255, 255, 0), 2)
    cv2.putText(combined_img, "Wiener (ROI 3,4,5)", (10, h - 10), cv2.FONT_HERSHEY_SIMPLEX,
1, (255, 255, 0), 2)

    rois = [
        {"x": 340, "y": 390, "w": 80, "h": 30},
        {"x": 895, "y": 380, "w": 80, "h": 30},
        {"x": 400, "y": 560, "w": 80, "h": 30},
        {"x": 580, "y": 560, "w": 80, "h": 30},
        {"x": 760, "y": 560, "w": 80, "h": 30},
    ]

    found_gaps_per_roi = {}

    for idx, roi in enumerate(rois):
        x, y, w_, h_ = roi["x"], roi["y"], roi["w"], roi["h"]
        binary = binary_a if idx in [0, 1] else binary_b
        gap_lengths = []
        sample_positions = np.linspace(x, x + w_ - 1, num_samples, dtype=int)

        for cx in sample_positions:
            roi_column = binary[y:y + h_, cx]
            target_value = 255 if idx in [0, 1] else 0
            target_pixels_y = np.where(roi_column == target_value)[0] + y

            if len(target_pixels_y) > 1:
                groups = np.split(target_pixels_y, np.where(np.diff(target_pixels_y) > 1)[0] + 1)
                for group in groups:
                    start, end = group[0], group[-1]
                    length = end - start
```

```
                if idx in [0, 1]:
                    above = binary[start - 1, cx] if start > y else None
                    below = binary[end + 1, cx] if end < y + h_ - 1 else None
                    if above != 0 or below != 0:
                        continue
                gap_lengths.append(length)
                cv2.line(combined_img, (cx, start), (cx, end), (0, 0, 255), 1)

        found_gaps_per_roi[idx] = gap_lengths

        if gap_lengths:
            median_gap = int(np.median(gap_lengths))
            global_roi_gaps[idx].append(median_gap)
            cv2.rectangle(combined_img, (x, y), (x + w_, y + h_), (0, 255, 0), 2)
            cv2.putText(combined_img, f"{median_gap}px", (x, y - 10), cv2.FONT_HERSHEY_SIMPLE
X, 0.6, (0, 255, 255), 2)
        else:
            global_roi_missing_gaps[idx] += 1
            cv2.rectangle(combined_img, (x, y), (x + w_, y + h_), (0, 0, 255), 2)

    return combined_img

def visualize_gap_distribution(output_dir, selected_filter):
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    axes = axes.flatten()

    for i in range(6):
        ax = axes[i]
        if global_roi_gaps[i]:
            data = global_roi_gaps[i]
            mean_val = np.mean(data)
            std_val = np.std(data)

            ax.hist(data, bins=10, color='skyblue', edgecolor='black', alpha=0.7)
            ax.axvline(mean_val, color='red', linestyle='--', label=f"Mean: {mean_val:.2f}")
            ax.axvline(mean_val - std_val, color='green', linestyle='--', label=f"-1 STD")
            ax.axvline(mean_val + std_val, color='green', linestyle='--', label=f"+1 STD")

            ax.set_title(f"ROI {i+1}")
            ax.set_xlabel("Gap Length (px)")
            ax.set_ylabel("Frequency")
            ax.legend()

    plt.tight_layout()
    plt.savefig(os.path.join(output_dir, f"gap_histogram_{selected_filter}.png"))
    plt.close()

    # Missing Gap bar chart
    plt.figure(figsize=(10, 5))
```

```
    rois = list(global_roi_missing_gaps.keys())
    values = list(global_roi_missing_gaps.values())
    bars = plt.bar(rois, values, color='salmon', alpha=0.8)
    plt.xticks(rois, [f"ROI {i+1}" for i in rois])
    plt.xlabel("ROI")
    plt.ylabel("Missing Count")
    plt.title(f"Missing Gaps per ROI - {selected_filter}")
    for bar, val in zip(bars, values):
        plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(), str(val), ha='center', va='bottom')
    plt.savefig(os.path.join(output_dir, f"missing_gaps_{selected_filter}.png"))
    plt.close()

def analyze_black_pixels(binary, image_copy, num_samples=20):
    rois = [
        {"x": 340, "y": 390, "w": 80, "h": 30},  # ROI 1
        {"x": 895, "y": 380, "w": 80, "h": 30},  # ROI 2
        {"x": 400, "y": 560, "w": 80, "h": 30},  # ROI 3
        {"x": 580, "y": 560, "w": 80, "h": 30},  # ROI 4
        {"x": 760, "y": 560, "w": 80, "h": 30},  # ROI 5
    ]

    global global_roi_gaps, global_roi_missing_gaps
    binary_colored = cv2.cvtColor(binary, cv2.COLOR_GRAY2BGR)
    found_gaps_per_roi = {}

    for idx, roi in enumerate(rois):
        x, y, w, h = roi["x"], roi["y"], roi["w"], roi["h"]
        sample_positions = np.linspace(x, x + w - 1, num_samples, dtype=int)
        gap_lengths = []

        for cx in sample_positions:
            roi_column = binary[y:y+h, cx]

            target_value = 255 if idx in [0, 1] else 0
            target_pixels_y = np.where(roi_column == target_value)[0] + y

            if len(target_pixels_y) > 1:
                groups = np.split(target_pixels_y, np.where(np.diff(target_pixels_y) > 1)[0] + 1)

                for group in groups:
                    start, end = group[0], group[-1]
                    length = end - start

                    # ROI 1,2: 검정-흰색-검정 패턴만 인정
                    if idx in [0, 1]:
                        above = binary[start - 1, cx] if start > y else None
                        below = binary[end + 1, cx] if end < y + h - 1 else None
                        if above != 0 or below != 0:
                            continue  # 둘 다 검정이 아니면 skip
```

```
                gap_lengths.append(length)
                cv2.line(binary_colored, (cx, start), (cx, end), (0, 0, 255), 1)

        found_gaps_per_roi[idx] = gap_lengths

        if gap_lengths:
            median_gap = int(np.median(gap_lengths))
            global_roi_gaps[idx].append(median_gap)
            cv2.rectangle(binary_colored, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.putText(binary_colored, f"{median_gap}px", (x, y - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)
        else:
            global_roi_missing_gaps[idx] += 1
            cv2.rectangle(binary_colored, (x, y), (x + w, y + h), (0, 0, 255), 2)

    return binary_colored, found_gaps_per_roi




def evaluate_filter_performance(original_image, filtered_image, filter_name):
    """ 필터 성능을 평가하여 메트릭 반환 """

    # 🔹 ROI 검정 픽셀 길이 분석 결과 활용
    mean_gap_values = [np.mean(global_roi_gaps[i]) if global_roi_gaps[i] else 0 for i in range(6)]
    std_gap_values = [np.std(global_roi_gaps[i]) if global_roi_gaps[i] else 0 for i in range(6)]
    missing_gaps = sum(global_roi_missing_gaps.values())

    # 🔹 이상치 갭 개수 계산 (평균 ± 3.5 * 표준편차 기준)
    outlier_gaps = sum([
        sum(1 for gap in global_roi_gaps[i] if gap < np.mean(global_roi_gaps[i]) - 3.5 * np.std(global
_roi_gaps[i]) or
            gap > np.mean(global_roi_gaps[i]) + 3.5 * np.std(global_roi_gaps[i]))
        if len(global_roi_gaps[i]) > 0 else 0
        for i in range(6)
    ])

    # 🔹 PSNR 및 SSIM 계산
    psnr_value = psnr(original_image, filtered_image)
     # **해결 방법: channel_axis=-1 및 win_size 자동 조정**
    try:
        ssim_value = ssim(original_image, filtered_image, channel_axis=-1, data_range=filtered_ima
ge.max() - filtered_image.min())
    except ValueError:
        ssim_value = 0  # SSIM 계산이 불가능할 경우 0으로 설정
    # 🔹 결과 저장
    result = {
        "Filter": filter_name,
```

```
        "Mean Gap": np.mean(mean_gap_values),
        "Std Gap": np.mean(std_gap_values),
        "Missing Gaps": missing_gaps,
        "Outlier Gaps": outlier_gaps,
        "PSNR": psnr_value,
        "SSIM": ssim_value
    }

    return result

def compare_filters_performance(folder_path, noise_filters, edge_filters, max_images=10):
    """
    여러 필터 조합을 적용하여 성능 비교 후 CSV 저장
    - ROI별 평균 갭 및 표준편차 값을 개별 컬럼으로 저장
    """

    performance_results = []

    for noise_filter in noise_filters:
        for edge_filter in edge_filters:
            avg_time, mean_gap_values, std_gap_values, missing_gaps = process_images(
                folder_path, f"{noise_filter}_{edge_filter}", noise_filter, edge_filter, max_images
            )

            # 결과 저장
            result_entry = {
                "Filter": f"{noise_filter} + {edge_filter}",
                "Processing Time (s)": avg_time,
                "Missing Gaps": missing_gaps
            }

            # ROI별 평균 갭 및 표준편차 추가
            for i in range(6):
                result_entry[f"ROI {i+1} Mean Gap"] = mean_gap_values[i]
                result_entry[f"ROI {i+1} Std Gap"] = std_gap_values[i]

            performance_results.append(result_entry)

    # 📄 CSV 파일로 저장
    csv_filename = "1C2G_GAPCHECK_results2.csv"
    with open(csv_filename, mode="w", newline="") as file:
        fieldnames = ["Filter", "Processing Time (s)", "Missing Gaps"] + \
                     [f"ROI {i+1} Mean Gap" for i in range(6)] + \
                     [f"ROI {i+1} Std Gap" for i in range(6)]

        writer = csv.DictWriter(file, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(performance_results)
```

```
    print(f"📁 ROI별 평균 및 표준편차가 포함된 성능 비교 결과가 '{csv_filename}' 로 저장되었습니다.")

def process_images_with_params(folder_path, output_prefix, noise_filter, edge_filter, noise_para
ms, edge_params, max_images=10):
    global global_roi_gaps, global_roi_missing_gaps
    global_roi_gaps = {i: [] for i in range(6)}
    global_roi_missing_gaps = {i: 0 for i in range(6)}

    out_dir = os.path.join("results", output_prefix)
    os.makedirs(out_dir, exist_ok=True)

    image_files = [f for f in os.listdir(folder_path) if f.lower().endswith(('png', 'jpg', 'jpeg'))]
    if max_images:
        image_files = image_files[:max_images]

    csv_path = os.path.join(out_dir, "metrics.csv")
    with open(csv_path, mode="w", newline="") as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(["Image", "PSNR", "SSIM"] + [f"ROI{i+1}_Gap" for i in range(6)])

        for image_file in image_files:
            image_path = os.path.join(folder_path, image_file)
            image = cv2.imread(image_path)
            if image is None:
                continue

            processed = apply_filter(image, noise_filter, edge_filter, params={**noise_params, **edge
_params})
            binary = apply_threshold(processed)
            analyzed_img, found_gaps_per_roi = analyze_black_pixels(binary, binary)

            psnr_val = psnr(image, processed)
            try:
                ssim_val = ssim(image, processed, channel_axis=-1, data_range=processed.max() - pr
ocessed.min())
            except:
                ssim_val = 0

            gaps = [np.mean(global_roi_gaps[i]) if global_roi_gaps[i] else 0 for i in range(6)]
            writer.writerow([image_file, psnr_val, ssim_val] + gaps)

            out_path = os.path.join(out_dir, image_file)
            cv2.imwrite(out_path, analyzed_img)

    visualize_gap_distribution(out_dir, output_prefix)

def process_images_combined_roi_filters(folder_path, output_prefix, max_images=1000):
    global global_roi_gaps, global_roi_missing_gaps
    global_roi_gaps = {i: [] for i in range(6)}
```

```
    global_roi_missing_gaps = {i: 0 for i in range(6)}

    out_dir = os.path.join("results", output_prefix)
    os.makedirs(out_dir, exist_ok=True)

    image_files = [f for f in os.listdir(folder_path) if f.lower().endswith(('png', 'jpg', 'jpeg'))]
    image_files = image_files[:max_images]

    csv_path = os.path.join(out_dir, "roi_gap_metrics.csv")
    with open(csv_path, mode="w", newline="") as csv_file:
        writer = csv.writer(csv_file)
        writer.writerow(["Image"] + [f"ROI{i+1}_Gap" for i in range(6)])

        for image_file in image_files:
            image_path = os.path.join(folder_path, image_file)
            image = cv2.imread(image_path)
            if image is None:
                continue

            img_a = apply_filter(image, "anisotropic", "opening_closing", {"radius": 8, "eps": 0.01, "ker
nel": 3})
            img_b = apply_filter(image, "wiener", "opening_closing", {"h": 10, "hColor": 10, "template
WindowSize": 7, "searchWindowSize": 21, "kernel": 3})

            binary_a = apply_threshold(img_a, 100)
            binary_b = apply_threshold(img_b, 130)

            combined_img = analyze_and_visualize_on_combined(image, img_a, img_b, binary_a, bina
ry_b)
            cv2.imwrite(os.path.join(out_dir, image_file), combined_img)

            row = [image_file]
            for i in range(6):
                if global_roi_gaps[i]:
                    row.append(np.mean(global_roi_gaps[i][-1:]))
                else:
                    row.append(0)
            writer.writerow(row)

    visualize_gap_distribution(out_dir, output_prefix)
    print(f"✅ 완료: 분석 이미지와 ROI 통계 시각화 저장됨 → {out_dir}")


def process_images(folder_path, output_prefix, noise_filter, edge_filter, max_images=None, num
_samples=20):
    global global_roi_gaps, global_roi_missing_gaps
    global_roi_gaps = {i: [] for i in range(6)}
    global_roi_missing_gaps = {i: 0 for i in range(6)}
```

```
    filter_dir = f"1C2G_filtered_images/{noise_filter}_{edge_filter}"
    missing_gaps_dir = f"{filter_dir}/Missing_Gaps"
    outlier_gaps_dir = f"{filter_dir}/Outlier_Gaps"

    os.makedirs(filter_dir, exist_ok=True)
    os.makedirs(missing_gaps_dir, exist_ok=True)
    os.makedirs(outlier_gaps_dir, exist_ok=True)

    image_files = [f for f in os.listdir(folder_path) if f.lower().endswith(('png', 'jpg', 'jpeg'))]

    if max_images:
        image_files = image_files[:max_images]

    if not image_files:
        print(f"🚨 No images found in {folder_path}. Returning default values.")
        return 0, [0] * 6, [0] * 6, 0

    total_time = 0
    processed_count = 0

    for image_file in image_files:
        image_path = os.path.join(folder_path, image_file)
        image = cv2.imread(image_path)

        if image is None:
            print(f"⚠ Warning: Unable to read {image_file}, skipping...")
            continue

        start_time = time.time()

        try:
            filtered_image = apply_filter(image, noise_filter, edge_filter)
            if filtered_image is None:
                print(f"⚠ Warning: {noise_filter} + {edge_filter} returned None for {image_file}. Saving
original.")
                filtered_image = image.copy()
        except Exception as e:
            print(f"❌ Error applying filter ({noise_filter} + {edge_filter}) on {image_file}: {e}")
            filtered_image = image.copy()

        binary_image = apply_threshold(filtered_image)
        analyzed_image, found_gaps_per_roi = analyze_black_pixels(binary_image, num_samples=n
um_samples)

        for idx, roi in enumerate([
            {"x": 340, "y": 390, "w": 80, "h": 30},
            {"x": 895, "y": 380, "w": 80, "h": 30},
            {"x": 400, "y": 560, "w": 80, "h": 30},
            {"x": 580, "y": 560, "w": 80, "h": 30},
```

```
            {"x": 760, "y": 560, "w": 80, "h": 30},
        ]):
            x, y, w, h = roi["x"], roi["y"], roi["w"], roi["h"]

            mean_gap = np.mean(global_roi_gaps[idx]) if global_roi_gaps[idx] else 0
            std_gap = np.std(global_roi_gaps[idx]) if global_roi_gaps[idx] else 0
            lower_bound, upper_bound = mean_gap - 3.5 * std_gap, mean_gap + 3.5 * std_gap
            found_gaps = found_gaps_per_roi[idx]

            color = (0, 255, 0)
            if global_roi_missing_gaps[idx] > 0:
                color = (0, 0, 255)
            elif any(gap < lower_bound or gap > upper_bound for gap in found_gaps):
                color = (0, 165, 255)

            cv2.rectangle(analyzed_image, (x, y), (x + w, y + h), color, 2)

        output_path = os.path.join(filter_dir, f"{output_prefix}_{image_file}")
        cv2.imwrite(output_path, analyzed_image)

        if sum(global_roi_missing_gaps.values()) > 0:
            cv2.imwrite(os.path.join(missing_gaps_dir, f"{output_prefix}_{image_file}"), analyzed_ima
ge)

        processed_count += 1
        total_time += time.time() - start_time

    avg_time = total_time / max(1, processed_count)
    print(f"✅ Processed {processed_count}/{len(image_files)} images for {noise_filter} + {edge_fi
lter}")

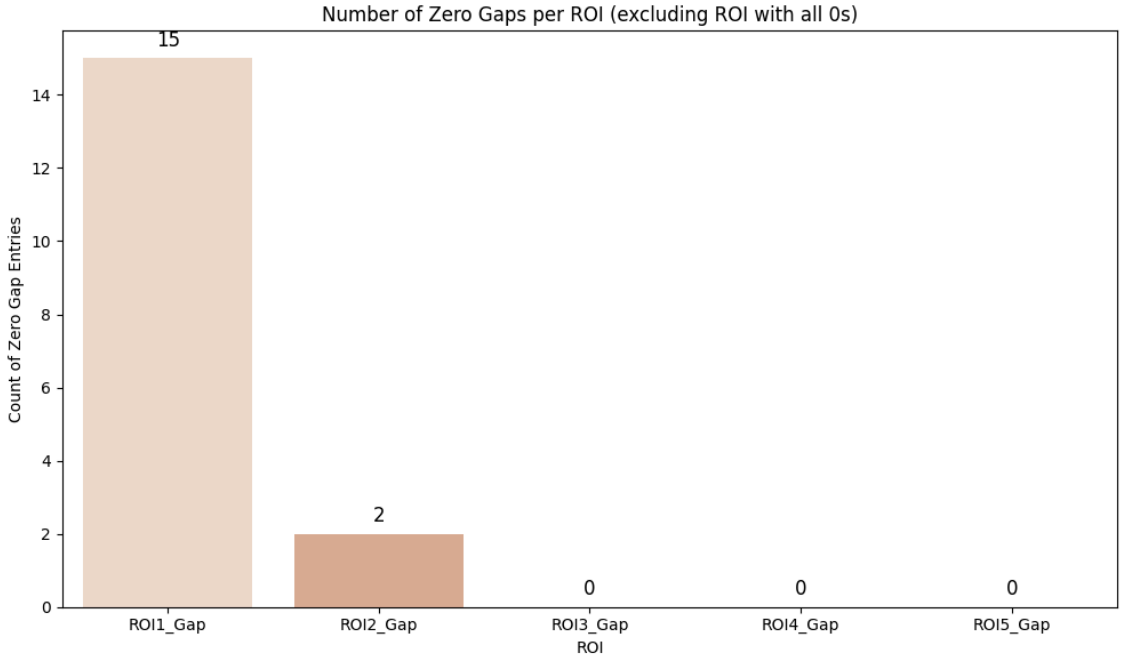
    mean_gap_values = [np.mean(global_roi_gaps[i]) if global_roi_gaps[i] else 0 for i in range(6)]
    std_gap_values = [np.std(global_roi_gaps[i]) if global_roi_gaps[i] else 0 for i in range(6)]
    missing_gaps = sum(global_roi_missing_gaps.values())

    return avg_time, mean_gap_values, std_gap_values, missing_gaps

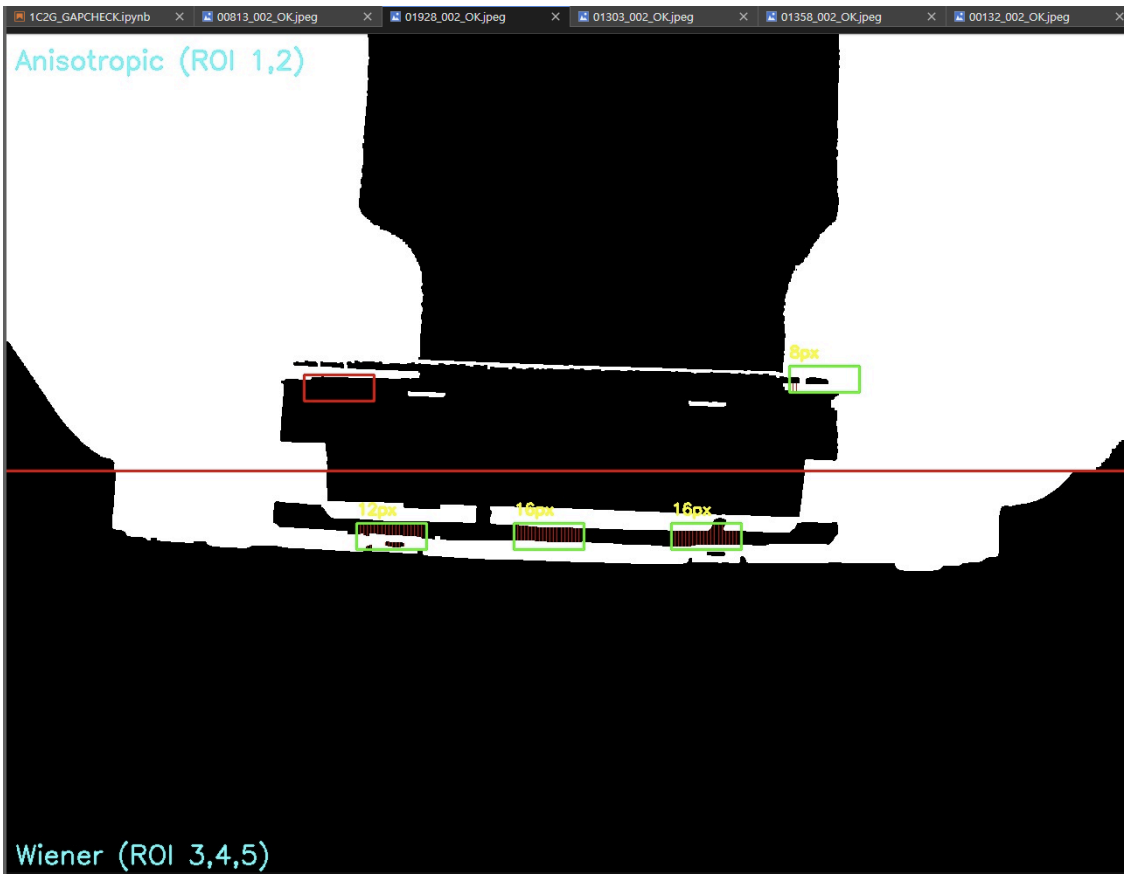

def main():
    folder_path = '1C2G_UNIT2/rotated_translated_images'
    output_prefix = "combined_filtered_with_ROI"
    process_images_combined_roi_filters(folder_path, output_prefix, max_images=10)

main()
```

roi 높이 30이니



roi 내에 검정-흰색-검정 패턴 안들어와 갭이 측정 안되는 경우 존재



→ roi 높이 50으로 변경

최종 roi 위치 정보

```
rois = [
      {"x": 340, "y": 375, "w": 80, "h": 50},
      {"x": 895, "y": 380, "w": 80, "h": 50},
      {"x": 400, "y": 560, "w": 80, "h": 40},
      {"x": 580, "y": 560, "w": 80, "h": 40},
      {"x": 760, "y": 560, "w": 80, "h": 40},
   ]
```